

ZEBRA HANDHELD RFID READER

IoT Connector

MQTT API

RFD40 / RFD90 Series · API v1.1

Document version 1.1.0 · Generated 2026-10-06

<https://zebra-stage.github.io/dcs/rfid/rfd40-90-mqtt/api-reference/mqtt/>

config_endpoint

REFERENCE



API consumer



add, update, or delete an MQTT endpoint configuration on the reader

Overview

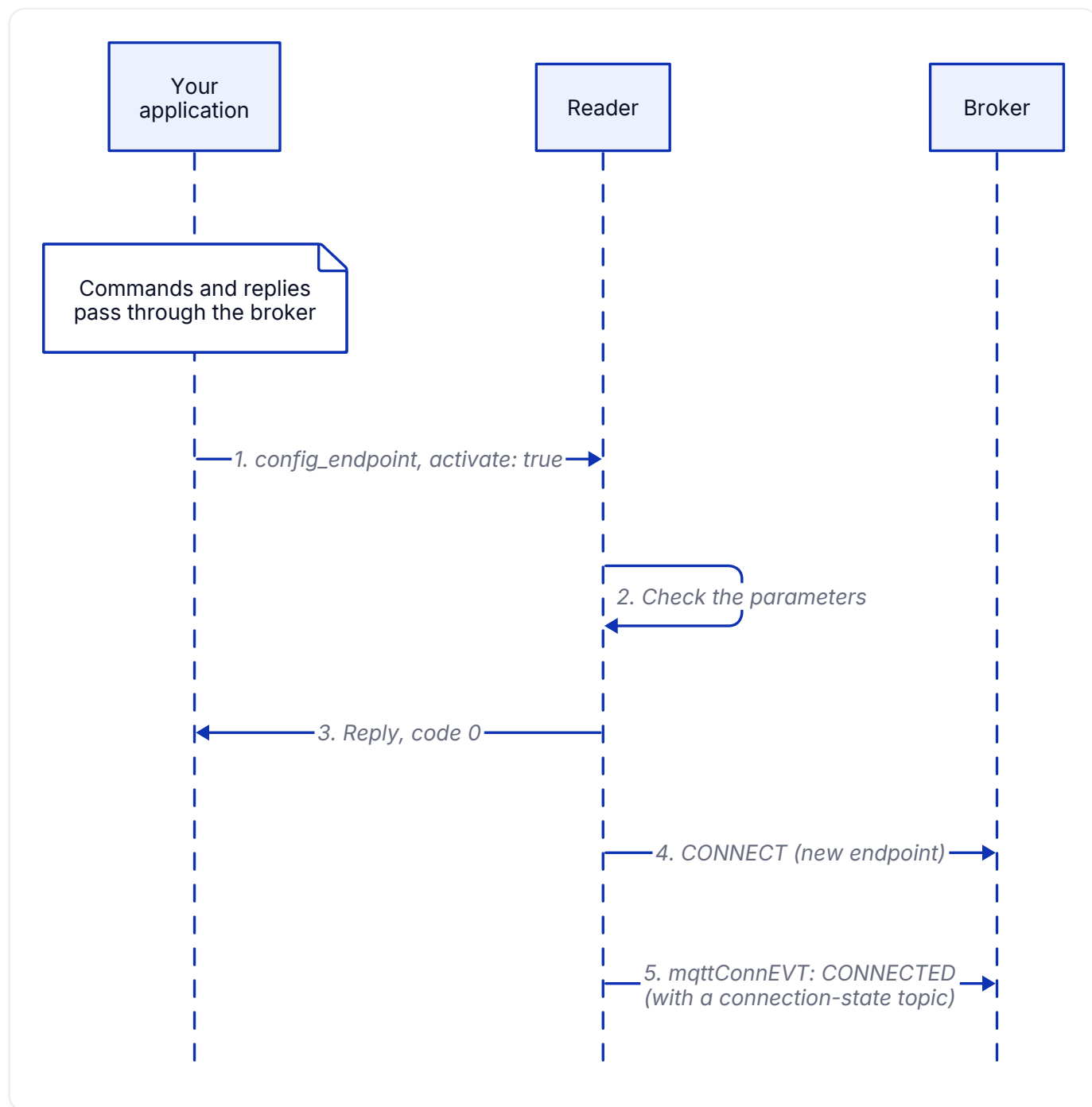
`config_endpoint` provisions the reader's MQTT endpoints — it **adds**, **updates**, or **deletes** an endpoint configuration (chosen by `epConfig.operation`). One payload defines an endpoint's type, protocol, broker `url / port`, TLS `verificationType` and certificates, MQTT `keepAlive` /session/reconnect settings, publish/subscribe topics, optional broker credentials, and per-endpoint event routing. It is a state-changing command on the device-management (MGMT) endpoint; the response is **status-only**. Read the result back with `get_endpoint_config`.

`config_endpoint` is **state-changing and can carry broker credentials** — and reconfiguring the endpoint that carries your own control session can sever it. See the session-sever callout in [Notes and Caveats](#) and the credential/TLS handling in [Request Semantics](#). A saved endpoint is reversible with `operation: delete`.

⚠ SAVING AN ENDPOINT DOES NOT CONNECT IT

An endpoint connects only when it is **active**. `activate: false` saves the endpoint; `activate: true` saves it and marks it active. With `activate: true`, the reader checks the endpoint's parameters. If they are valid, it disconnects from the current configuration and connects to the new endpoint; no reboot is needed. Wait for a `CONNECTED mqttConnEVT` on the endpoint's connection-state topic to confirm the connection. `get_endpoint_config` lists the endpoint under `activeEndpoints` as soon as it is stored as active, which does not show that it connected.

What happens when you add an endpoint with `activate: true`:



Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (writes/removes a saved endpoint; may activate it)
Reversibility	REVERSIBLE (remove with <code>operation: delete</code> ; change with <code>operation: update</code>)
Supported Operations	<code>add</code> (new), <code>update</code> (existing), <code>delete</code> (existing)

Property	Value
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>epConfig</code> (with <code>operation</code> and <code>configuration</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	get endpoint config , config events , install certificate , reboot

When to Use

Use `config_endpoint` to provision or maintain the reader's MQTT wiring after the initial `MDM` endpoint exists (that first endpoint is set up with 123RFID Desktop; all others are added remotely with `config_endpoint`).

- `add` — create a new endpoint (`MGMT`, `CTRL`, `DATA1`, ...). The `endpointName` must not already exist (duplicate → [code 10](#)).
- `update` — modify an existing endpoint (e.g. change `url`, or flip `activate`). The `endpointName` must already exist.
- `delete` — remove an existing endpoint (the contract's example sends only `endpointName` + `epType`).
- **Pre-provision safely** — send `activate: false` to save an endpoint without activating it, then activate later with `update` . A saved endpoint does not connect until it is active.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The `topic` values you configure inside `mqttParams` are **base** topics (e.g. `MGMT/clients/resp`). The reader constructs the full topic at runtime as `<tenantId>/<topic>/<serial>` — never include the tenant or serial in `topic` .

Credentials are presented once at CONNECT (and, for a configured endpoint, via `mqttParams`). The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `epConfig.operation` (request) — `add` / `update` / `delete`; determines the action.
- `configuration.endpointName` (request) — the endpoint's identity (create target / update / delete key).
- `configuration.epType` (request) — the endpoint's role (`MGMT`, `CTRL`, `DATA1`, ...).
- `configuration.activate` (request) — `true` saves the endpoint and marks it active, and the reader connects for it without a reboot if the parameters are valid; `false` only saves it.
- `configuration.securityParams` (request) — for `MQTT_TLS`; the referenced certificates must be pre-installed via [install certificate](#).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The envelope is `command`, `requestId`, and the `epConfig` payload (`operation` + `configuration`). Pick an **operation** / **endpoint** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Operation / endpoint

Add MGMT endpoint for MQTT TLS ▾

Request payload

Example: add, MGMT, MQTT_TLS

```
1  {
2    "command" : "config_endpoint" ,
3    "requestId" : "1233" ,
4    "epConfig" : {
5      "operation" : "add" ,
6      "configuration" : {
7        "endpointName" : "mgmt_tls" ,
8        "epType" : "MGMT" ,
9        "protocol" : "MQTT_TLS" ,
10       "activate" : true ,
11       "url" : "broker.ip.com" ,
12       "port" : 8883 ,
13       "verificationType" : "VERIFY_HOST_AND_PEER" ,
14       "qosCommon" : 1 ,
15       "tenantId" : "ZEBRA" ,
16       "mqttParams" : {
17         "keepAlive" : 300 ,
18         "cleanSession" : true ,
19         "reconnectDelayMin" : 5 ,
20         "reconnectDelayMax" : 60 ,
21         "publishTopics" : [
22           {
23             "topic" : "MGMT/clients/resp" ,
24             "qos" : 1 ,
25             "retain" : false
26           } ,
27           {
28             "topic" : "MGMT/clients/event" ,
29             "qos" : 1 ,
30             "retain" : false
31           } ,
32           {
33             "topic" : "MGMT/clients/rfid" ,
34             "qos" : 0 ,
35             "retain" : true
36           }
37         ] ,
38         "subscribeTopics" : [
39           {
40             "topic" : "MGMT/clients/cmnd" ,
41             "qos" : 0
42           }
43         ]
44       } ,
45       "securityParams" : {
46         "format" : "PEM" ,
47         "caCertificateFile" : "mqtt_ca_cert" ,
48         "clientCert" : "mqtt_client_cert" ,
49         "clientKey" : "mqtt_client_key"
50       }
51     }
52   }
53 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Fixed comma
2	requestId *	string	—	abc123	A unique iden echoed back
3	epConfig *	object	—	—	The endpoint payload. Req which also re and configu
4	operation *	enum	add delete update	add	add a new en existing one, Required with Request Sem
5	configura tion *	object	—	—	The endpoint Required with contract requi epType, prot url, verific port for ever its own delet examples ser Semantics).
6	endpoi ntName *	string	—	mgmt	The endpoint add/update/d
7	epType *	enum	MGMT MGMT_EVT CTRL DATA1 DATA2 SOTI MDM	MGMT	The endpoint is currently n
8	protoc ol *	enum	MQTT MQTT_TLS	MQTT	MQTT (plain, t MQTT_TLS (TI 8883). Both li unchanged vi get_endpoi Request Sem
9	activa te *	boolean	—	true	true saves th marks it activ its parameter valid, connect reboot. false (activate it lat
10	url *	string	—	soti.example.com	Broker addres

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	verification * Type	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_HOST_AND_PEER	TLS verification. Use VERIFY_NONE in production, and VERIFY_HOST_AND_PEER in development.
12	port *	integer	—	1883	Broker port (1883 for MQTT, 883 for MQTTS).
13	qosCommon	integer	—	1	Default QoS level. 0 or 1, the level supported by the broker.
14	tenantId	string	—	ZEBRA	Tenant identifier of every on-warehouse contract. Send it without the contract's name. Bounded at 40 characters, no overlong values.
15	mqttParams	object	—	—	MQTT connection parameters, including topics, and (optional) credentials.
16	keepAlive	integer	max 65535	300	Keep-alive interval in seconds.
17	cleanSession	boolean	—	true	Whether to start a new session.
18	reconnectDelayMin	integer	—	5	Minimum reconnect delay in seconds.
19	reconnectDelayMax	integer	—	60	Maximum reconnect delay in seconds.
20	clientId	string	—	zebra-rfd40-01	MQTT client identifier for connection.
21	username	string	—	mqttuser	Broker username. Requires username if the broker requires it. Redact it in logs.
22	password	string	—	—	Broker password. Requires password if the broker requires it. — a secret; not logged.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	publishTopics	array	—	—	Base topics to — at most code 25). Ent schema exam responses, [: connection ev Will . On DAT/ guidance plac at [1] — see Each item rec retain.
24	topic*	string	—	zebra/rfd40/01/data	Base topic on wire to <tenantId>, Do not includ
25	qos*	integer	—	1	QoS for this p the levels the
26	retain*	boolean	—	false	Whether the l message.
27	subscribeTopics	array	—	—	Base topics th subscribes to returns code , requires topi
28	topic*	string	—	zebra/rfd40/01/data	Base topic on wire to <tenantId>,
29	qos*	integer	—	0	QoS for this s the levels the
30	retain*	boolean	—	—	Retain flag fo

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
31	securityParams	object	—	—	TLS parameters. Required with format. The certificates must be installed via install certificates .
32	format *	enum	PEM PFX	PEM	Certificate format. Supports PEM and PFX, currently supports PEM.
33	caCertificateFile	string	—	cloud-trusted-ca-certs.crt	Installed CA certificate file name (a name).
34	clientCert	string	—	client_cert.pem	Installed client certificate name.
35	clientKey	string	—	client_key.pem	Installed client key name.
36	eventConfiguration	object	—	—	Per-endpoint configuration. Same shape as eventConfiguration , selecting which endpoint emit heartbeat set. Are functional Semantics.
37	antenna	boolean	—	false	set to true to enable antenna events
38	terminalConnection	boolean	—	false	set to true to enable terminal connection/disconnection events
39	firmwareUpdate	boolean	—	true	set to true to enable firmware update events
40	gpi	boolean	—	false	set to true to enable gpi events
41	network	boolean	—	true	set to true to enable network events
42	exceptions	boolean	—	false	set to true to enable exceptions events
43	ntp	boolean	—	true	set to true to enable ntp alerts

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
44	userApp	boolean	—	false	set to true to enable events
45	heartbeat	boolean	—	false	set to true to enable heartbeat
46	power	boolean	—	true	set to true to enable power alerts
47	battery	boolean	—	true	set to true to enable battery alerts
48	temperature	boolean	—	false	set to true to enable temperature alerts
49	fileDownload	boolean	—	true	set to true to enable file download alerts
50	cpuUsage	boolean	—	false	set to true to enable cpu usage alerts
51	flashUsage	boolean	—	false	set to true to enable flash usage alerts
52	ramUsage	boolean	—	false	set to true to enable ram usage alerts
53	heartbeatConfiguration	object	—	—	Heartbeat set heartbeat configuration
54	interval	integer	max 65535	100	Seconds between most 65,535.
55	inventoryStatus	boolean	—	true	set to true to enable inventory status as part of message

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
56	batteryStatus	boolean	—	true	set to true to indicate battery status as part of heartbeat
57	userAppStatus	boolean	—	—	set to true to indicate user app status as part of heartbeat message
58	cpuThreshold	integer	—	0	cpu usage value. alert will be triggered when value exceeds threshold
59	ramThreshold	integer	—	0	ram usage value. alert will be triggered when value exceeds threshold
60	flashThreshold	integer	—	0	flash usage value. alert will be triggered when value exceeds threshold
61	temperatureThreshold	integer	—	0	temperature threshold. alert will be triggered when temperature exceeds threshold
62	batching	object	—	—	Report batching configuration. Optional; when enabled, the reader's built-in batching is used and no source batching is required. A report is published when the batch limit is reached.
63	reportingInterval	integer	1000–60000	5000	Reporting time interval, in milliseconds. Minimum value is 1000.
64	maxPayloadSizePerReport	integer	1024–5120	1024	Maximum payload size in bytes. 0 falls back to default minimum, 1024.

Request Semantics

Modes. `epConfig.operation` selects the action. The contract requires the same seven `configuration` fields for every operation — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType` and `port` — but its own `delete` example sends only `endpointName` and `epType`, and its `update` example omits `verificationType` and `port`. A validator built strictly from the contract rejects both examples; this is a known issue in the source schema:

- `add` — creates a new endpoint. Supply the full `configuration` (the 7 fields `add` requires — `endpointName`, `epType`, `protocol`, `activate`, `url`, `verificationType`, `port` — plus optional `mqttParams` / `securityParams` / `eventConfiguration`). Fails with [code 10](#) if the `endpointName` already exists.
- `update` — modifies an existing endpoint (matched by `endpointName`). The contract's example sends `endpointName` and `epType` plus the fields being changed.
- `delete` — removes an existing endpoint. The contract's example sends only `endpointName` and `epType`.

`add` is **not idempotent**. If a response is lost (e.g. a timeout), the endpoint may already have been created — **read back with `get_endpoint_config` before retrying** rather than blindly re-sending, or the retry will be a duplicate `add`.

- **Payloads can carry broker credentials.** `mqttParams.username` / `password` are supplied in the request when the broker requires them — masked in the example on this page. Redact them in logs, tickets, and screenshots, and send `config_endpoint` only over secured channels. (Certificate fields — `caCertificateFile` / `clientCert` / `clientKey` — are **reference names** of already-installed certificates, not key material.)
- **Optional MQTT connection fields.** `mqttParams.clientId` sets the MQTT client ID used for the broker connection; `cleanSession` defaults to `true`.
- **TLS needs pre-installed certificates.** For `protocol: MQTT_TLS`, the `securityParams` `caCertificateFile` / `clientCert` / `clientKey` must reference certificates already installed via [install_certificate](#). Only `format: PEM` is supported today. The protocol literal is spelled `MQTT_TLS` in both directions — one enum governs the request and the `get_endpoint_config` read-back — so send and match it exactly.
- **Topic slots are positional.** `publishTopics` accepts **at most 3** entries (exceeding → [code 25](#)) and `subscribeTopics` **at most 1** (exceeding → [code 26](#)); both arrays are optional — the contract's own `DATA1` read-back returns `subscribeTopics: []`. The three publish slots carry command **responses** (`.../clients/resp`), **events** — heartbeat, alerts (`.../clients/event`), then the endpoint's **connection-event** topic (`.../clients/rfid`, which carries `mqttConnEVT`); `subscribeTopics[0]` is **command ingress**, where the host publishes and the reader subscribes (`.../clients/cmdnd`). Position is what routes — the reader binds each entry by **index, not by base name** — so keep the configured order, leave omitted indices unused, and match a `get_endpoint_config` read-back by index rather than by topic name. Configure only the base `topic`; the reader wraps it as `<tenantId>/<topic>/<serial>`.
- **DATA endpoints: put the tag-data topic at `publishTopics[1]`.** On `epType DATA1` the reader streams tag data from the **second** publish entry (`DATA2` is currently not supported), so configure an unused placeholder at `[0]` and the tag-data topic (e.g. `DATA/clients/dataevent`) at `[1]`: `[{ "topic": "DATA/clients/unused", "qos": 0, "retain": false }, { "topic": "DATA/clients/dataevent", "qos": 1, "retain": false }]`. Every `DATA1` example on this page uses this two-entry layout. A single-entry array is the trap: the one entry lands in the **reply** slot, the `add` still succeeds, and no tag data ever arrives. A read-

back showing `activate: true` shows only that the endpoint is stored as active; it does not show that the endpoint connected or that tag data is flowing.

- `verificationType`: **use** `VERIFY_HOST_AND_PEER`. It verifies the broker's certificate chain and that the certificate matches the host. `VERIFY_NONE` turns verification off; use it only on an isolated test network, and never on an `MQTT_TLS` endpoint, which rejects it with [code 37](#). `get_endpoint_config` reads back the value you sent, so a client doing read-modify-write can echo it.
- **Event routing.** `configuration.eventConfiguration` uses the same shape as `config_events`; only 7 of its toggles are functional today (the rest are reserved). See the Event routing fields table on the [get_endpoint_config](#) page for the full supported-vs-reserved split.

Topic slots by `epType`. How the positional layout above lands per endpoint type. The contract's examples show the command/response layout; the data-endpoint layout (†) is firmware guidance that the contract does not state:

<code>epType</code>	<code>publishTopics[0]</code>	<code>publishTopics[1]</code>	<code>publishTopics[2]</code>	<code>subscribeTopics[0]</code>
<code>MGMT</code> · <code>MDM</code> · <code>SOTI</code>	command responses (<code>.../resp</code>)	events (<code>.../event</code>)	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>CTRL</code>	command responses (<code>.../resp</code>)	stored, not used for events	connection events (<code>.../rfid</code>)	command ingress (<code>.../cmnd</code>)
<code>MGMT_EVT</code>	not used	not used	Last Will	events are currently published here (a firmware defect)
<code>DATA1</code>	unused placeholder †	tag-data stream (<code>.../dataevent</code>) †	—	— (does not subscribe)

The command/response endpoints (`MGMT` , `MDM` , `SOTI` , `CTRL`) share the `[0]` and `[2]` layout; on `CTRL` , slot `[1]` is stored but not used for events. A `MGMT_EVT` endpoint currently publishes its events on its subscribe topic instead of `publishTopics[1]` , which the API owners record as a firmware defect; see [Endpoints and Topic Slots](#). Match read-back entries by index, not by base name — the `resp` / `event` / `rfid` names are conventions.

† Tag data binds to `publishTopics[1]` , so a data endpoint needs a throwaway entry at `[0]` to push the real topic into slot `[1]` — see the DATA-endpoint bullet above.

Response Payload

`config_endpoint` returns a **status-only** envelope — `command` , `requestId` , `apiVersion` , and `response` . There is no configuration echo; confirm the change with `get_endpoint_config` .

Response payload

```

1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "apiVersion": "V1.1",
5    "response": {
6      "code": 0,
7      "description": "Success"
8    }
9  }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_endpoint	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — config_endpoint echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Response Codes

config_endpoint returns response.code. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the endpoint was added, updated, or deleted.

Code	Description
10	Configuration already exists (<code>IOT_ERROR_CONFIG_ALREADY_EXIST</code>) — an <code>add</code> used an <code>endpointName</code> that already exists. Use <code>update</code> , or delete it first.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>operation</code> , <code>epType</code> , <code>protocol</code> , <code>verificationType</code> , <code>format</code>) is outside its allowed set.
25	Max 3 publish topics exceeded (<code>IOT_ERROR_PUBLISH_TOPICS_EXCEEDED</code>) — more than 3 <code>publishTopics</code> .
26	Max 1 subscribe topic exceeded (<code>IOT_ERROR_SUBSCRIBE_TOPIC_EXCEEDED</code>) — more than 1 <code>subscribeTopics</code> .
27	Invalid tenant ID length (<code>IOT_ERROR_INVALID_TENANTID_LENGTH</code>) — <code>tenantId</code> exceeds the allowed length.
33	Endpoint already configured (<code>IOT_ERROR_ENDPOINT_ALREADY_CONFIGURED</code>) — an endpoint with the same configuration already exists.
35	Can't modify an active endpoint (<code>IOT_ERROR_CANT_MODIFY_ACTIVE_ENDPOINT</code>) — the target endpoint is currently active. Deactivate it before modifying it.
37	Invalid verification type for MQTT TLS (<code>IOT_INVALID_MQTT_TLS_VERIFICATION_TYPE</code>) — <code>protocol: MQTT_TLS</code> was sent with <code>verificationType: VERIFY_NONE</code> . Use <code>VERIFY_HOST_AND_PEER</code> .
101	Default error — generic catch-all. See the duplicate- <code>add</code> note below.

Duplicate `add` may return the generic 101 . The documented code for adding an `endpointName` that already exists is 10 ("Configuration already exists"), but the firmware has been observed returning the generic 101 ("Default error") for a duplicate `add` instead of 10 . Treat a 101 after a re-sent `add` as a probable duplicate and verify with `get_endpoint_config` .

Examples

Publish a `config_endpoint` request; the device replies with the status-only envelope under [Response Payload](#). Every contract example — each `add` , the `delete` and the partial `update` — is in the **Operation / endpoint** dropdown under [Request Payload](#). The [examples](#) below show what the dropdown does not.

A **plain-MQTT** `DATA1` `add` with broker credentials (`activate: true`), the tag-data topic at publish slot [1] with an unused placeholder at [0] — the layout the firmware guidance under [Request Semantics](#) describes — and no TLS:

Request — add (plain MQTT, with credentials)

```
1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "epConfig": {
5      "operation": "add",
6      "configuration": {
7        "endpointName": "dataEP",
8        "epType": "DATA1",
9        "protocol": "MQTT",
10       "activate": true,
11       "url": "broker.example.com",
12       "verificationType": "VERIFY_NONE",
13       "port": 1883,
14       "qosCommon": 1,
15       "tenantId": "zebra",
16       "mqttParams": {
17         "keepAlive": 300,
18         "cleanSession": true,
19         "reconnectDelayMin": 50,
20         "reconnectDelayMax": 500,
21         "username": "*****",
22         "password": "XXXXXXXX",
23         "publishTopics": [
24           {
25             "topic": "DATA/clients/unused",
26             "qos": 0,
27             "retain": false
28           },
29           {
30             "topic": "DATA/clients/datalevent",
31             "qos": 1,
32             "retain": false
33           }
34         ]
35       }
36     }
37   }
38 }
```

A duplicate `add` returns `code 10` (or, on some firmware, the generic `101` — see the note above):

Response — duplicate add (code 10)

```
1  {
2    "command": "config_endpoint",
3    "requestId": "abc123",
4    "apiVersion": "V1.1",
5    "response": {
6      "code": 10,
7      "description": "Configuration already exists"
8    }
9  }
```

Notes and Caveats

⚠ RECONFIGURING THE ACTIVE CONTROL ENDPOINT CAN SEVER YOUR SESSION

Activating a *new* endpoint of a different role coexists with the endpoint carrying your control session (e.g. activating a `DATA1` endpoint does not drop the `MDM` / `MGMT` control channel). But changing the broker `url` / `port` / `protocol` of — or deleting — the endpoint that carries your **own** control session can sever it. When in doubt, save with `activate: false` and switch deliberately.

Because the response is status-only, **verify the result with** `get_endpoint_config` — confirm the endpoint appears in `savedEndpoints.epNames[]` and (if `activate: true`) under `activeEndpoints`. A `CONNECTED mqttConnEVT` on the endpoint's connection-state topic confirms that it connected. For the concept-level view, see [Endpoints and Topic Slots](#).

Related:  [Endpoints and Topic Slots](#) ·  `get_endpoint_config` ·  `config_events` ·  `install_certificate` ·  [Response Codes](#)

config_events

REFERENCE · API consumer · enable/disable the reader's event streams and configure the heartbeat

Overview

`config_events` enables or disables which device-management events the reader emits, sets the heartbeat interval and payload, and configures alert thresholds. You send a single `eventConfiguration` object of boolean flags (plus `heartbeatConfiguration` and thresholds). It is a state-changing management-plane ([MGMT](#)) command and carries no credentials.

Two things make this command distinctive:

- Changes take effect after a reboot.** The configuration is stored immediately (visible via `get_endpoint_config`), but the reader only starts emitting the configured events after a `reboot`. The command's schema states it plainly: *"To apply the changes, a device reboot is required."* Do not assume events are live until the reader has rebooted.
- Accept-but-drop.** The reader accepts the whole payload with `code 0` even if it contains flags it does not support. Of the declared flags, **7 are functional** today; the other 13 are **reserved**: accepted, stored and returned in the read-back, but with no effect on v1.1 firmware. Always read back with `get_endpoint_config` to see what was stored — but note a stored *reserved* flag has still not taken functional effect.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (updates the endpoint's stored event configuration)
Reversibility	REVERSIBLE (re-send <code>config_events</code> with the desired flags to restore a prior configuration)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>eventConfiguration</code> (with the flags)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series

Property	Value
API Version	v1.1 (API Version Reporting)
Related	<code>get_endpoint_config</code> (read back what applied), <code>config_endpoint</code> , <code>reboot</code> , <code>get_status</code>

When to Use

Use `config_events` to control the reader's management-event surface:

- **Turn event streams on/off** — enable `terminalConnection`, `firmwareUpdate`, `network`, `power`, `battery`, `fileDownload`, OR `heartbeat`.
- **Shape the heartbeat** — set `heartbeatConfiguration.interval` and choose whether each heartbeat carries inventory / battery status.
- **Reduce noise** — disable streams you do not consume so the reader emits less telemetry.

Then `reboot` to activate emission, and `get_endpoint_config` to confirm the stored configuration.

Topic Mapping

`config_events` is a **management-plane command** — answered on the MGMT/[MDM](#) endpoint, not the control endpoint.

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmd/<serial></code>	On the management endpoint that carries your session.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	Where the enabled events flow, after a reboot .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`config_events` configures the **event configuration of the management endpoint on which it is received** — other endpoints' event settings (set via `config_endpoint`) are left untouched.

See [Topics and Channels](#).

Key Fields

- **command** (request) — the command name; send exactly `config_events`. The schema does not `enum`-constrain it, so a wrong string is not caught by validation.
- **requestId** (request) — a unique identifier for the request; echoed back in the response to pair reply to request.
- **eventConfiguration** (request) — the event configuration to store; treat it as a **wholesale set** and send the complete configuration you want. The contract requires it.
- **enable flags** (request) — the boolean toggles inside `eventConfiguration` (`terminalConnection`, `firmwareUpdate`, `network`, `heartbeat`, `power`, `battery`, `fileDownload`, plus the reserved flags) that turn each event stream on or off; `heartbeatConfiguration` shapes the heartbeat when `heartbeat: true`.
- **response.code** (response) — `0` = Success (present on success for this management command); non-zero = error. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `eventConfiguration`. Pick an **event set** below; the Example and Schema update together. Use the **Schema** tab for the full field reference (with the supported/reserved status per field) and the **Example** tab for the concrete payload.

Event set

Enable all events

▼

Request payload

Example: enable all supported events

```
1 {
2   "command" : "config_events" ,
3   "requestId" : "abc123" ,
4   "eventConfiguration" : {
5     "terminalConnection" : true ,
6     "firmwareUpdate" : true ,
7     "network" : true ,
8     "heartbeat" : true ,
9     "power" : true ,
10    "battery" : true ,
11    "temperature" : true ,
12    "fileDownload" : true ,
13    "heartbeatConfiguration" : {
14      "interval" : 100 ,
15      "inventoryStatus" : true ,
16      "batteryStatus" : true
17    }
18  }
19 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	comm and *	string	—	config_events	Command name. Send exactly config_events; the schema does not enum-constrain this field (see Request Semantics).
2	requ estI * d	string	—	abc123	A unique identifier for the request; echoed back in the response.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	eventConfiguration *	object	—	—	The event configuration to store. Required by the contract. Treat it as a wholesale set — send the complete configuration you want (see Request Semantics).
4	antenna	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
5	terminalConnection	boolean	—	true	✅ Supported. Terminal (cradle / USB-C) connect and disconnect events.
6	firmwareUpdate	boolean	—	true	✅ Supported. Firmware-update progress and completion events.
7	gpi	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
8	network	boolean	—	true	✅ Supported. Network interface state and connectivity changes.
9	exceptions	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
10	ntp	boolean	—	true	⚠ Reserved (declared for future support; not emitted on V1.1). The device may store/echo it in <u>get_endpoint_config</u> , but it has no effect yet.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
11	use rApp p	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
12	hea rtb eat	boolean	—	true	✅ Supported. Periodic heartbeat; shape it with heartbeatConfiguration.
13	pow er	boolean	—	true	✅ Supported. Power-source changes (battery / external power).
14	bat ter y	boolean	—	true	✅ Supported. Battery-state alerts (low, critical, full).
15	tem per atu re	boolean	—	true	⚠ Reserved — declared in the schema, and the reader accepts and stores it, but no temperature alert is emitted on V1.1, so switching it on has no effect (see Request Semantics).
16	fil eDo wnl oad	boolean	—	true	✅ Supported. File-download progress/completion events (firmware, certificates).
17	cpu Usa ge	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
18	fla shU sag e	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).
19	ram Usa ge	boolean	—	false	⚠ Reserved (declared for future support; not emitted on V1.1).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
20	heartbeatConfiguration	object	—	—	Heartbeat shape; set when heartbeat: true.
21	interval	integer	max 65535	100	✔ Supported. Seconds between heartbeats, at most 65,535 (lower = more telemetry, higher battery cost).
22	inventoryStatus	boolean	—	true	✔ Supported. Include the inventory-status block (rfidStatus, tagCount, scanCount) in each heartbeat.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	b a t t e r y S t a t u s	boolean	—	true	✔ Supported. Include the battery-status block in each heartbeat.
24	u s e r A p p s	boolean	—	—	⚠ Reserved — declared for future support, not functional on current V1.1 firmware.
25	cpu Thr esh old	integer	—	0	⚠ Reserved — CPU-usage percent that would trigger a CPU alert. Not honored on V1.1.
26	ram Thr esh old	integer	—	0	⚠ Reserved — RAM-usage percent. Not honored on V1.1.
27	fla shT hre sho ld	integer	—	0	⚠ Reserved — flash-usage percent. Not honored on V1.1.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
28	temperature	integer	—	0	⚠ Reserved — temperature value that would trigger the alert. Not honored on V1.1, and neither is the temperature flag itself.

When `heartbeat: true`, `heartbeatConfiguration` shapes each heartbeat message — lower `interval` means more telemetry at higher battery cost, and `inventoryStatus` / `batteryStatus` toggle which sub-blocks each heartbeat carries (see the Schema tab above for per-field detail; `userApps` is ⚠ reserved, not functional on V1.1). See [MQTT Connectivity](#) for the heartbeat event shape.

Request Semantics

Field Support. Seven of the flags below are functional on current V1.1 firmware; every flag is declared in the schema. The reader **accepts** a reserved flag (returns `code 0`) but it has no effect. The API developer has declared these for **future support**; treat them as **not currently supported**. Reserved flags are accepted, stored and echoed back in the read-back while remaining **unemitted** — **always read back with** `get_endpoint_config`, remembering that a reserved flag appearing there is stored, not functional; only the 7 supported flags take effect.

Event flag	Status	Stream
<code>terminalConnection</code>	✓ Supported	Terminal (cradle / USB-C) connect / disconnect
<code>firmwareUpdate</code>	✓ Supported	Firmware-update progress / completion
<code>network</code>	✓ Supported	Network interface / connectivity changes
<code>heartbeat</code>	✓ Supported	Periodic heartbeat (shaped by <code>heartbeatConfiguration</code>)
<code>power</code>	✓ Supported	Power-source change (battery / external)

Event flag	Status	Stream
battery	 Supported	Battery state (low, critical, full)
temperature	 Reserved	Temperature alert — declared in the schema but never sent by the reader, so switching it on is accepted and does nothing
fileDownload	 Supported	File-download progress / completion
antenna	 Reserved	Not emitted on V1.1
gpi	 Reserved	Not emitted on V1.1
exceptions	 Reserved	Not emitted on V1.1
ntp	 Reserved	Not emitted on V1.1 (may be stored/echoed)
userApp	 Reserved	Not emitted on V1.1
cpuUsage	 Reserved	Not emitted on V1.1
flashUsage	 Reserved	Not emitted on V1.1
ramUsage	 Reserved	Not emitted on V1.1

The four **threshold** fields (`cpuThreshold` , `ramThreshold` , `flashThreshold` , `temperatureThreshold`) and `heartbeatConfiguration.userApps` are likewise **reserved** — declared but not honored on V1.1. Enabling any reserved field is accepted (returns `code 0`) but has no effect. For the emitted-event catalog, see [Events](#).

- **Send every flag you want set.** Nothing in `eventConfiguration` is required and no flag has a documented default. Send the complete set of flags you intend, then check the

eventConfiguration block in `get_endpoint_config`.

Response Payload

`config_events` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no configuration echo; confirm the stored configuration with `get_endpoint_config`.

Response payload

```
1 {
2   "command" : "config_events" ,
3   "requestId" : "abc123" ,
4   "apiVersion" : "V1.1" ,
5   "response" : {
6     "code" : 0 ,
7     "description" : "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	config_events	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — config_events echoes no configuration.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

The contract's success example **includes** `code: 0`. `command`, `requestId`, `apiVersion` and `response` — with both `code` and `description` — are required on every reply.

Response Codes

The API contract lists **no command-specific error codes** for `config_events` — only the universal codes apply. In practice the command is highly permissive (it accepts unsupported flags with `code_0` rather than rejecting them; see [Request Semantics](#)), so success is by far the common outcome.

Code	Description
0	Success — the configuration was stored (<code>code</code> present, <code>description: "Success"</code>). Emission activates on the next reboot.
101	Default error — generic catch-all for an unhandled failure.

The contract scopes the invalid-payload code (`2`) to other commands, not to `config_events`. For the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Examples

Publish `config_events` over the MGMT endpoint; the device replies with the status envelope under [Response Payload](#), stores the supported subset immediately, and begins emitting after a reboot. The **Event set** dropdown under [Request Payload](#) shows both contract examples: **Enable**

all events, with a 100 s heartbeat, and **Selective events**, which turns some streams off

(`heartbeat: false`, no `heartbeatConfiguration`).

Notes and Caveats

- **Stored immediately, emitted after a reboot.** `config_events` writes the configuration right away — you can confirm it with `get_endpoint_config` before rebooting — but the reader only starts **emitting** the enabled events after a `reboot`. Plan a reboot into any event-configuration change.
- **Global and per-endpoint settings.** An endpoint can also carry its own `eventConfiguration`, set with `config_endpoint`. Which of the two wins when they disagree is not specified in the available sources.

`config_events` carries **no credentials** — only event flags and thresholds, so there is nothing to mask. It is **reversible**: re-send with the desired flags to restore a prior configuration (followed by a reboot). For the concept-level view, see [Event Management](#).

Related:  [get_endpoint_config](#) ·  [config_endpoint](#) ·  [reboot](#) ·  [Events](#) ·  [MQTT Connectivity](#) ·  [Event Management](#) ·  [Response Codes](#)

delete_certificate

REFERENCE

API consumer

remove a certificate (or all of a type) from the reader's trust store

Overview

`delete_certificate` removes a certificate from the reader's trust store, identified by `certificateInfo.name`, `type`. It is **synchronous** — a successful delete returns `response.code 0` and the certificate is gone immediately (confirm with [get installed certificates](#)). It is a state-changing command on the device-management ([MGMT](#)) endpoint.

`delete_certificate` is **destructive and irreversible** — and omitting `certificateInfo.name` deletes every certificate of the given `type`. See the danger callout in [Notes and Caveats](#) before sending.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (removes a certificate, or all certificates of a type)
Reversibility	IRREVERSIBLE (no undo; re-install with install_certificate to restore)
Required Request Fields	<code>command</code> , requestId , <code>certificateInfo</code> (with <code>type</code> ; <code>name</code> optional)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	get installed certificates , install_certificate , config_endpoint , set_wifi

When to Use

Use `delete_certificate` to clean up the reader's trust store:

- **Remove an obsolete or expired certificate** — e.g. before rotating in a replacement.
- **Clear a slot before re-installing** — delete then `install_certificate` with the same `name`.
- **Wipe a whole type** — omit `name` to remove every certificate of a `type` (e.g. all `wifi` certificates) in one call. Use deliberately.

Delete only certificates that are **not in active use**. On a TLS endpoint, deleting the broker/client certificate the reader is currently connected with will break that connection.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`delete_certificate` identifies certificates by `name` / `type` only, so it carries **no credential field** — there is nothing to mask. The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `certificateInfo.type` (request) — the certificate category (`mqtt`, `wifi`, `filestore`, `client`, `server`); required. Confirm exact names/types with `get_installed_certificates` first.
- `certificateInfo.name` (request) — the certificate to delete. ⚠️ **Omitting it deletes every certificate of type** (bulk). Verify the exact name with `get_installed_certificates` (installed entries are named `{name}_{component}`).
- `response.code` (response) — read this to determine success vs failure: `0` (deleted), `21` (no such certificate). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `certificateInfo` object naming the certificate to delete. Pick a **certificate type** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for a single-certificate delete.

Certificate scenario

Delete wifi ca cert

Request payload

Example: wifi / ca_cert

```
1 {
2   "command" : "delete_certificate",
3   "requestId" : "abc123",
4   "certificateInfo" : {
5     "name" : "ca_cert",
6     "type" : "wifi"
7   }
8 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	delete_certificate	Fixed command name.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	certificateInfo *	object	—	—	Identifies the certificate(s) to delete. Required by the contract, and the device rejects a request without it with code 101 .
4	name	string	—	test_soti	⚠ The certificate name to delete. If omitted, ALL certificates of the given type are deleted (bulk, destructive — see Notes).
5	type *	enum	server client mqtt wifi filestore	client	Certificate category. Required within certificateInfo. Selects the certificate (with name) or the whole set (without name).

To delete **every certificate of a type**, omit `name` and send only `type` — a destructive bulk operation:

Request — bulk delete (all wifi certificates)

```
1 {
2   "command" : "delete_certificate" ,
3   "requestId" : "abc123" ,
4   "certificateInfo" : {
5     "type" : "wifi"
6   }
7 }
```

Request Semantics

- `certificateInfo` is required, and so is the `type` inside it. The contract requires the payload outright, and the device agrees — a request without it is rejected with [code 101](#). Always send `certificateInfo` with at least a valid `type`.
- Type-`null` certificates cannot be deleted.** `delete_certificate` matches on `type`, so a certificate that `install_certificate` created **without a** `type` (`type: null`, which the installer accepts) matches no `type` value and returns [code 21](#) for every attempt — it becomes un-deletable through this command. Avoid the situation by always sending `type` to `install_certificate`; to clear an existing orphan, re-install the same `name` with an explicit `type`.

Response Payload

`delete_certificate` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no certificate echo; confirm the removal with [get_installed_certificates](#).

Response payload

```
1 {
2   "command" : "delete_certificate" ,
3   "requestId" : "abc123" ,
4   "apiVersion" : "V1.1" ,
5   "response" : {
6     "code" : 0 ,
7     "description" : "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	delete_certificate	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — delete_certificate echoes no certificate data.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	d e s c r i p t i o n	string	—	Success	Human-readable status.

Response Codes

`delete_certificate` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the matching certificate (or all of the type) was removed.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a backing certificate file could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the <code>name</code> / <code>type</code> is not a saved certificate. See the not-found note below.
101	Default error — generic catch-all (e.g. a malformed payload; see the note below).

Two live-observed behaviors to code against: (1) **A not-found delete returns `code` 21 but with the description text "File not found"** (the wording of `code` 9) — key on the numeric 21, not the string. (2) **A malformed payload** — a missing required `type`, or a missing `certificateInfo` — returns the generic 101 ("Default error"), while an **invalid `type`** value returns 23 ("Invalid enum value"). Gate success strictly on `code == 0`.

Examples

Publish `delete_certificate` naming the certificate to remove; the device replies with the status-only envelope under [Response Payload](#). The bulk-delete request (omit `name`) is shown under [Request Payload](#).

Targeting a `name` / `type` that is **not installed** returns [code](#) `21` (note the `"File not found"` description):

Response — certificate not found (code 21)

```
1  {
2    "command" : "delete_certificate" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 21 ,
7      "description" : "File not found"
8    }
9  }
```

Notes and Caveats

 **IRREVERSIBLE — AND OMITTING `name` WIPES EVERY CERTIFICATE OF THE TYPE**

There is no undo and no confirmation prompt. A `delete_certificate` with `certificateInfo.type` but no `name` deletes **every** certificate of that type. Always include an explicit `name` unless a full type-wipe is intended, and verify the exact name first with `get_installed_certificates`. Never delete a certificate an **active TLS connection** relies on — doing so breaks that connection. Recovery in every case means re-installing with `install_certificate`.

Because the response is status-only, **verify the result with** `get_installed_certificates` — confirm the deleted `name` / `type` no longer appears. For the concept-level view, see [TLS and Authentication](#) and [Certificate Management](#).

Related:  [TLS and Authentication](#) ·  `get_installed_certificates` · 
`install_certificate` ·  `config_endpoint` ·  [Response Codes](#)

delete_wifi_profile

REFERENCE



API consumer



permanently remove a saved Wi-Fi profile from the reader by ESSID

Overview

`delete_wifi_profile` permanently removes a single saved Wi-Fi profile from the reader's profile store. The profile is identified by its `essid` inside `wifiProfileInfo`; on success the matching saved profile — including any stored credentials — is deleted, and the command returns a status-only envelope (no profile echo). It is a state-changing command on the device-management ([MGMT](#)) endpoint. Confirm the result with `get_wifi`.

Deletion is permanent — there is no "undelete"; restoring a profile means re-provisioning with `set_wifi`. The deletion is safe for the control session: the reader refuses to delete the active profile ([code 16](#)). See [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (removes a saved profile and its stored credentials)
Reversibility	IRREVERSIBLE (there is no undo; re-provision with <code>set_wifi</code> to restore a deleted profile)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>wifiProfileInfo</code> (with <code>essid</code> to identify the profile)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_wifi</code> , <code>set_wifi</code> , <code>delete_certificate</code>

When to Use

Use `delete_wifi_profile` to **clean up the reader's saved Wi-Fi profiles** — one profile per call, matched by `ssid`.

- **Remove an obsolete or incorrect profile** — e.g. a network that no longer exists, or one saved with the wrong credentials.
- **Make room before adding a profile** — the device enforces a maximum number of saved profiles; if `set_wifi` `CREATE` returns [code 19](#) (SSID count overflow), delete an unused profile first.
- **Tidy fallback sets** — prune stale entries so the reader only carries the profiles you intend it to roam across.

Delete only a **non-active** profile. To retire the network the reader is currently connected to, first connect it elsewhere (with `set_wifi` `connect: true`) — the device will not delete the profile carrying its live connection ([code 16](#)).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT; `delete_wifi_profile` therefore carries **no credential field**, so there is nothing to mask. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `wifiProfileInfo.essid` (request) — the network name of the saved profile to delete. Case-sensitive; must exactly match a saved profile (verify the exact name with `get_wifi`). Must name a non-active profile; targeting the active profile fails with [code 16](#).
- `response.code` (response) — read this to determine success vs failure: `0` (deleted), `15` (no such saved profile), `16` (that profile is the active connection), `17` (no `essid` supplied). See [Response Codes](#).

Request Payload

The request envelope is `command`, `requestId`, and the `wifiProfileInfo` object naming the profile to delete. Use the **Schema** tab for the full field reference and the **Example** tab for a ready-to-send request.

Request payload

```
1 {
2   "command" : "delete_wifi_profile",
3   "requestId" : "abc123",
4   "wifiProfileInfo" : {
5     "essid" : "TestAP2"
6   }
7 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	comm and	string	—	<code>delete_wifi_profile</code>	Fixed command name.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response so you can pair reply to request.
3	wifiProfileInfo *	object	—	—	Identifies the saved Wi-Fi profile to delete. Required by the contract, and it must name a profile with <code>essid</code> (see Request Semantics).
4	essid *	string	—	JioFi	The network name (ESSID) of the saved profile to remove. Case-sensitive; must exactly match a saved profile.

Request Semantics

- **One profile per call, matched by `essid`.** `delete_wifi_profile` removes a single saved profile identified by its network name. A `code 15` means the `essid` is not saved (typo or wrong case); confirm the exact stored name with `get_wifi` first. A `code 17` means the request reached the device with no `essid` at all.
- `wifiProfileInfo` **is required**. The contract requires it, so a request omitting it is invalid — and it would name no profile and delete nothing in any case. Always send `wifiProfileInfo` with a valid `essid` (which **is required** within `wifiProfileInfo`).

Response Payload

`delete_wifi_profile` returns a **status-only** envelope — `command`, `requestId`, `response` and, optionally, `apiVersion`. The contract's schema for this reply does not declare `apiVersion` at all; the published success example omits it and the error examples carry it. There is no profile echo; to confirm the deletion, follow up with `get_wifi` and check that the `ssid` is gone. Pick a **result** below; the Example and Schema update together — the variants differ only in `response.code` / `response.description` and in whether `apiVersion` is present. Parse `apiVersion` as optional; see the [response envelope](#).

Result

Successful deletion

▼

Response payload

```
1 {
2   "command" : "delete_wifi_profile" ,
3   "requestId" : "abc345" ,
4   "response" : {
5     "code" : 0 ,
6     "description" : "Success"
7   }
8 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	com man * d	string	—	delete_wifi_profile	Echoes the executed command.
2	req ues * tId	string	—	abc345	Echoes the request's requestId — pairs reply to request.
3	res pon * se	object	—	—	Status-only result — delete_wifi_profile echoes no profile data.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	code*	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
5	description*	string	—	Success	Human-readable status.

Response Codes

`delete_wifi_profile` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the matching profile was removed.
2	Invalid payload (<code>IOT_ERROR_INVALID_PAYLOAD</code>) — the JSON is malformed or has invalid field values. Validate against the schema and correct.
15	WIFI Error - SSID not found (<code>IOT_ERROR_SSID_NOT_FOUND</code>) — the <code>ssid</code> is not a saved profile. Verify the exact name (case-sensitive) with <code>get_wifi</code> .
16	WIFI Error - Cannot delete active SSID (<code>IOT_ERROR_DELETE_ACTIVE_SSID</code>) — the target <code>ssid</code> is the currently-connected profile; the reader protects it. Connect elsewhere first, then delete.

Code	Description
17	WIFI Error - SSID missed (<code>IOT_ERROR_SSID_MISSED</code>) — no <code>ssid</code> was supplied in <code>wifiProfileInfo</code> . Include the <code>ssid</code> field.
101	Default error — generic catch-all.

Examples

Publish a `delete_wifi_profile` request naming the `ssid` to remove; the device replies with the status-only envelope. The **Result** dropdown under [Response Payload](#) shows the three replies in the contract:

- **Successful deletion** — `code 0`.
- **Cannot delete active SSID** — `code 16`, unique to `delete_wifi_profile`. The reader refuses to delete the profile it is connected through.
- **SSID not found** — `code 15`, shared with `set_wifi`. The `ssid` is not a saved profile.

Notes and Caveats

⚠ DELETION IS PERMANENT

There is no undo. The saved profile and its stored credentials are removed; restoring it means re-provisioning with `set_wifi` and re-entering the credentials. Double-check the `ssid` before sending — the match is **case-sensitive** and exact.

- **The active profile is firmware-protected.** Unlike `set_wifi` with `connect: true` — which can switch networks and sever the control session — `delete_wifi_profile` **cannot** strand the reader: if you target the currently-connected `ssid`, the device refuses with `code 16` ("Cannot delete active SSID") rather than dropping its own connection. To retire the network the reader manages itself over, first move it to another profile (`set_wifi` `connect: true`), then delete the now-inactive profile.

Because the response is status-only, **verify the result with** `get_wifi` — confirm the deleted `ssid` no longer appears in the saved profiles and the connected profile is unchanged. For the concept-level view, see [Network Configuration](#).

Related: [Network Configuration](#) · `get_wifi` · `set_wifi` · `delete_certificate` · [Response Codes](#)

get_current_region

REFERENCE

API consumer

read the reader's active regulatory region

Overview

`get_current_region` retrieves the reader's **active regulatory / RF region configuration** in a single round trip: the country and governing regulatory standard, the channel plan (the frequencies the radio may use), the transmit-power limits, and whether Listen-Before-Talk (LBT) and frequency hopping are enabled. It is a read-only query on the device-management ([MGMT](#)) endpoint.

The region governs whether — and how — the reader is legally allowed to transmit, so this read is the **compliance gate before any radio operation**. Note that the region can only be **read** over MQTT; it is **set** with a 123RFID application, 123RFID Desktop or 123RFID Mobile, typically at bootstrap. See also [Management](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_status</code> , <code>get_version</code>

When to Use

Use `get_current_region` when you need to **confirm the reader's regulatory configuration before it transmits** — typically in provisioning verification, a pre-inventory check, or a fleet compliance audit.

- **Confirm the region:** verify `country` and `regulatoryStandard` match the deployment location before starting inventory.
- **Validate the RF envelope:** check `maxTxPowerSupported` / `minTxPowerSupported`, `channelData`, `lbtEnabled`, and `frequencyHopping` against the constraints your operation expects.

- **Audit compliance across a fleet:** read the region on each device to confirm consistent, compliant settings.
- **After provisioning or a region change:** run `get_current_region` to confirm the new settings took effect before relying on the radio.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmdnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `currentRegion.country` / `regulatoryStandard` (response) — the configured country and governing standard (e.g. `FCC`, `ETSI`); confirm they match the deployment location.
- `currentRegion.maxTxPowerSupported` (response) — the power ceiling (dBm); validate transmit-power settings against it before inventory.
- `currentRegion.lbtEnabled` / `frequencyHopping` (response) — compliance behaviors that affect how (and whether) inventory can start.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_current_region" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

• — not applicable

• Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command * command	string	—	get_current_region	Fixed command name. Retrieves the current region configuration.
2	requestId * requestId	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command` , `requestId` , `apiVersion` , `response` , and a `currentRegion` object. On the **Schema** tab, expand `currentRegion` to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON. `currentRegion` is optional at the envelope level, but on a successful read it is the operative payload and **all seven of its members are required**. `country` and `regulatoryStandard` are free-form (device-driven) — the schema does not constrain them to a fixed vocabulary.

Response payload

```
1  {
2    "command" : "get_current_region" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "currentRegion" : {
6      "frequencyHopping" : true ,
7      "channelData" : [
8        "915750" ,
9        "915250" ,
10       "903250" ,
11       "926750" ,
12       "926250" ,
13       "904250" ,
14       "927250" ,
15       "920250" ,
16       "919250" ,
17       "909250" ,
18       "918750" ,
19       "917750" ,
20       "905250" ,
21       "904750" ,
22       "925250" ,
23       "921750" ,
24       "914750" ,
25       "906750" ,
26       "913750" ,
27       "922250" ,
28       "911250" ,
29       "911750" ,
30       "903750" ,
31       "908750" ,
32       "905750" ,
33       "912250" ,
34       "906250" ,
35       "917250" ,
36       "914250" ,
37       "907250" ,
38       "918250" ,
39       "916250" ,
40       "910250" ,
41       "910750" ,
42       "907750" ,
43       "924750" ,
44       "909750" ,
45       "919750" ,
46       "916750" ,
47       "913250" ,
48       "923750" ,
49       "908250" ,
```

```

50         "925750" ,
51         "912750" ,
52         "924250" ,
53         "921250" ,
54         "920750" ,
55         "922750" ,
56         "902750" ,
57         "923250"
58     ],
59     "country" : "United States" ,
60     "lbtEnabled" : false ,
61     "maxTxPowerSupported" : 30 ,
62     "minTxPowerSupported" : 0 ,
63     "regulatoryStandard" : "FCC"
64 },
65 "response" : {
66     "code" : 0 ,
67     "description" : "Success"
68 }
69 }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_current_region	Echoes the executed command
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command; emitted per-handler, device-wide.
4	currentRegion	object	—	—	The active regulatory region configuration. Optional at the envelope level; when present, a seven members below are required.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	frequency hopping pin g *	boolean	—	true	Whether the radio hops across channel plan (vs a fixed channel plan). Required in most regions.
6	channel data a *	array	—	["915750", ...]	Channel center frequencies the reader may transmit on, as string in kHz (e.g. 865700 = 865.7 MHz). Count and values are region-specific.
7	country *	string	—	USA	The country the device is configured for. Free-form (device driven) — e.g. India, United States.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
8	l b t E n a b l e d	boolean	—	false	Whether Listen-Before-Talk is enabled (required in some ETSI regions; affects inventory start behavior).
9	m a x T x P o w e r S u p p o r t e d	number	—	30	Maximum transmit power supported in the region. Documented in dBm — confirm scale against your firmware if precise power matters.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	minimum transmit power supported	number	—	0	Minimum transmit power supported in the region (dBm).
11	regulatory standard	string	—	FCC_USA	The regulatory standard governing transmission rules — e.g. FCC, ETSI, and sometimes a more specific plan identifier such as ETSI_EU_800M_900M_3_CHANNEL. Free-form (device-driven).
12	response	object	—	—	Result wrapper.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
13	code	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
14	description	string	—	Success	Human-readable status.

Response Semantics

- `channelData` is a list of channel **center frequencies in kHz**, as strings — e.g. `"865700"` = 865.7 MHz. The count and values depend on the configured region: US / FCC has ~50 channels across 902–928 MHz; ETSI regions typically have fewer, in the 865–868 MHz band. `country` and `regulatoryStandard` are free-form, device-driven strings (no fixed enum).
- `maxTxPowerSupported` / `minTxPowerSupported` are documented in **dBm** (e.g. `30` dBm $\approx$ 1 W). If precise transmit power matters to your integration, confirm the scale against your firmware.

Response Codes

`get_current_region` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — region configuration retrieved and returned.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the region information at this time. Retry after a short delay; if it persists, reboot the device.
101	Default error — generic catch-all.

An **unconfigured region** surfaces as `code 6` (`IOT_STATUS_REGION_NOT_CONFIGURED`) at **connect / bootstrap time** — it is not returned by `get_current_region` itself. If a device has no region set, configure it with 123RFID Desktop or 123RFID Mobile before connecting.

Examples

`get_current_region` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

The response shown throughout is the contract's **United States / FCC** example (50 channels across the 902–928 MHz band). The `channelData` list and the `country` / `regulatoryStandard` / `power` values are region-specific — an ETSI region, for example, typically reports fewer channels, in the 865–868 MHz band.

Notes and Caveats

A WRONG REGION CAN PUT THE READER OUTSIDE LOCAL RADIO REGULATIONS

The **region** is set with 123RFID Desktop or 123RFID Mobile, not by any MQTT command — `get_current_region` only reads it. A reader **will not transmit at all until a region is set**, and a reader set to the wrong region will operate but may violate local regulations. If a freshly provisioned reader reports the wrong `country`, redo region setup in 123RFID Desktop or 123RFID Mobile. See [Management](#) and RF Regions and Regulations.

- **Read-only over MQTT.** `get_current_region` returns the active region and takes no payload. The region is set with 123RFID Desktop or 123RFID Mobile. This documentation covers 123RFID Desktop, which connects over USB-C before the reader joins a broker.

Run `get_current_region` **after device provisioning or a region change** to confirm the regulatory settings are correct before starting inventory operations.

Related:  [Management](#) ·  [get_status](#) ·  [get_version](#) ·  RF Regions and Regulations ·  [Response Codes](#)

get_endpoint_config

REFERENCE · API consumer · read back the reader's MQTT endpoint configuration and saved endpoint names

Overview

`get_endpoint_config` reads back the reader's MQTT endpoint configuration — a faithful view of what was provisioned with `config_endpoint`. It has two modes, chosen by whether you send `endpointDetails.endpointName`:

- All-endpoints (omit `endpointDetails`):** returns every **active** endpoint's full configuration under `endpointResponse.activeEndpoints.epConfig[]`, **plus** the names of **all saved** endpoints under `endpointResponse.savedEndpoints.epNames[]`.
- Single-endpoint (send `endpointDetails.endpointName`):** returns just that one endpoint's configuration.

It is a read-only command on the device-management (MGMT) endpoint.

The response includes broker credentials (`mqttParams.password`, and `username` where set) — treat it as sensitive. See the warning in [Notes and Caveats](#). The examples on this page are the contract's, and carry placeholders rather than real credentials.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (plus optional <code>endpointDetails.endpointName</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	<code>config_endpoint</code> , <code>get_wifi</code> , <code>get_eth</code> , <code>config_events</code>

When to Use

Use `get_endpoint_config` to inspect or audit the reader's MQTT wiring:

- **Before** `config_endpoint` — review the current configuration so you do not overwrite an active endpoint unexpectedly.
- **Confirm the plumbing** — check `url` / `port`, `protocol`, `verificationType`, and the `publishTopics` / `subscribeTopics` a broker will see.
- **List saved endpoints** — read `savedEndpoints.epNames[]` to see everything saved on the device (a superset of the active endpoints) before activating or deleting one.
- **Audit routing & security** — review each endpoint's `eventConfiguration` and `securityParams`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The `publishTopics` / `subscribeTopics` inside the response are **base** topics (e.g. `MDM/clients/cmnd`); on-wire they are wrapped to `<tenantId>/<topic>/<serial>`.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `endpointDetails.endpointName` (request) — omit for all endpoints; supply to target one (case-sensitive).
- `activeEndpoints.epConfig[]` (response) — the full config of each **active** endpoint. Inactive (saved-but-not-activated) endpoints do **not** appear here. An endpoint appears here as soon as it is stored as active, which does not show that it connected: wait for a `CONNECTED` `mqttConnEVT` on the endpoint's connection-state topic. See [Saving an endpoint does not connect it](#).

- `savedEndpoints.epNames[]` (response) — the names of **all** saved endpoints (active *and* inactive) — the active/saved distinction in one field.
- `configuration.mqttParams.password` (response) — ⚠ a broker credential is present in the response; see [Response Payload](#) and the credential warning below.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

To read **all** endpoints, send just `command` and `requestId`. Use the **Schema** tab for the full field reference and the **Example** tab for the all-endpoints request.

Scope

Active endpoint config

Request payload

Example: all endpoints

```
1 {
2   "command" : "get_endpoint_config" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_endpoint_config	Fixed command name.
2	requestId *	string	—	123	A unique identifier for the request; echoed back in the response so you can pair reply to request.
3	endpointDetails	object	—	—	Optional selector. Omit it to retrieve every active endpoint plus all saved names (all-endpoints mode); include it to retrieve one endpoint (single-endpoint mode).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	en dp oi nt Na me	string	—	ctrl	The saved endpoint name to retrieve. Case-sensitive. When present, only that endpoint is returned.

Request Semantics

- **Modes.** All-endpoints (omit `endpointDetails`) returns every active endpoint plus all saved names; single-endpoint (`endpointDetails.endpointName`) returns just that one endpoint.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the endpoint payload under `endpointResponse` — `activeEndpoints.epConfig[]` (full configs) and `savedEndpoints.epNames[]` (all saved names). The **Example** tab shows two active endpoints — an `MDM` endpoint and a `DATA1` endpoint.

`mqttParams.password` and `username` in the example below are the broker credentials a read-back carries. The payload is taken verbatim from the contract, so it follows the contract's own placeholder policy rather than this page's: `password` reads `xxxxxxx` , a placeholder, but `username` reads `mqttuser` — a value the contract ships unmasked. Treat both as placeholders, not as credentials to reuse.

Response payload

Example: all active endpoints

```
1      {
2        "command": "get_endpoint_config",
3        "requestId": "abc123",
4        "apiVersion": "V1.22",
5        "endpointResponse": {
6          "activeEndpoints": {
7            "epConfig": [
8              {
9                "configuration": {
10                  "endpointName": "mdm_ep2",
11                  "epType": "MDM",
12                  "protocol": "MQTT",
13                  "activate": true,
14                  "url": "17L11-DS3362",
15                  "verificationType": "VERIFY_NONE",
16                  "port": 1883,
17                  "qosCommon": 0,
18                  "tenantId": "zebra",
19                  "mqttParams": {
20                    "keepAlive": 96,
21                    "cleanSession": true,
22                    "reconnectDelayMin": 5,
23                    "reconnectDelayMax": 512,
24                    "clientId": "",
25                    "username": "mqttuser",
26                    "password": "xxxxxx",
27                    "publishTopics": [
28                      {
29                        "topic": "MDM/clients/resp",
30                        "qos": 0,
31                        "retain": false
32                      },
33                      {
34                        "topic": "MDM/clients/event",
35                        "qos": 0,
36                        "retain": false
37                      },
38                      {
39                        "topic": "MDM/clients/rfid",
40                        "qos": 0,
41                        "retain": true
42                      }
43                    ],
44                    "subscribeTopics": [
45                      {
46                        "topic": "MDM/clients/cmnd",
47                        "qos": 0
```

```

48         }
49     ]
50 },
51 "securityParams": {
52     "format": "PEM",
53     "caCertificateFile": "",
54     "clientCert": "",
55     "clientKey": ""
56 },
57 "eventConfiguration": {
58     "antenna": false,
59     "terminalConnection": false,
60     "firmwareUpdate": true,
61     "gpi": false,
62     "network": true,
63     "exceptions": false,
64     "ntp": true,
65     "userApp": false,
66     "heartbeat": false,
67     "power": true,
68     "battery": true,
69     "temperature": false,
70     "fileDownload": true,
71     "cpuUsage": false,
72     "flashUsage": false,
73     "ramUsage": false,
74     "cpuThreshold": 0,
75     "ramThreshold": 0,
76     "flashThreshold": 0,
77     "temperatureThreshold": 0
78 }
79 }
80 },
81 {
82     "configuration": {
83         "endpointName": "data1_ep",
84         "epType": "DATA1",
85         "protocol": "MQTT",
86         "activate": true,
87         "url": "17L11-DS3362",
88         "verificationType": "VERIFY_NONE",
89         "port": 1883,
90         "qosCommon": 0,
91         "tenantId": "zebra",
92         "mqttParams": {
93             "keepAlive": 96,
94             "cleanSession": true,
95             "reconnectDelayMin": 5,
96             "reconnectDelayMax": 512,
97             "clientId": "",
98             "username": "mqttuser",
99             "password": "xxxxxxx",

```

```

100         "publishTopics" : [
101             {
102                 "topic" : "DATA/clients/unused" ,
103                 "qos" : 0 ,
104                 "retain" : false
105             },
106             {
107                 "topic" : "DATA/clients/dataevent" ,
108                 "qos" : 1 ,
109                 "retain" : false
110             }
111         ],
112         "subscribeTopics" : [ ]
113     },
114     "securityParams" : {
115         "format" : "PEM" ,
116         "caCertificateFile" : "" ,
117         "clientCert" : "" ,
118         "clientKey" : ""
119     },
120     "eventConfiguration" : {
121         "antenna" : false ,
122         "terminalConnection" : false ,
123         "firmwareUpdate" : true ,
124         "gpi" : false ,
125         "network" : true ,
126         "exceptions" : false ,
127         "ntp" : true ,
128         "userApp" : false ,
129         "heartbeat" : false ,
130         "power" : true ,
131         "battery" : true ,
132         "temperature" : false ,
133         "fileDownload" : true ,
134         "cpuUsage" : false ,
135         "flashUsage" : false ,
136         "ramUsage" : false ,
137         "cpuThreshold" : 0 ,
138         "ramThreshold" : 0 ,
139         "flashThreshold" : 0 ,
140         "temperatureThreshold" : 0
141     },
142     "batching" : {
143         "reportingInterval" : 1000 ,
144         "maxPayloadSizePerReport" : 1024
145     }
146 }
147 }
148 ]
149 },
150 "savedEndpoints" : {
151     "epNames" : [

```

```

152         "mdm_ep" ,
153         "mdm_ep2" ,
154         "ctrl_ep" ,
155         "data1_ep" ,
156         "mgmt_ep"
157     ]
158 }
159 },
160 "response" : {
161     "code" : 0 ,
162     "description" : "Success"
163 }
164 }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	command *	string	—	get_endpoint_config
2	requestId *	string	—	123
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1
4	endpointResponse	object	—	—
5	activeEndpoints	object	—	—
6	epConfig	array	—	—
7	configuration	object	—	—
8	endpointName	string	—	MDM
9	epType *	enum	MGMT MGMT_EVT CTRL DATA1 DATA2 SOTI MDM	MGMT

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
10	protocol *	enum	MQTT MQTT_TLS	MQTT
11	activate *	boolean	—	true
12	url *	string	—	soti.example.com
13	verification * Type	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_HOST_AND_PEER
14	port *	integer	—	1883
15	qosCommon	integer	—	1
16	tenantId	string	—	ZEBRA
17	mqttParams	object	—	—
18	keepAlive	integer	max 65535	300
19	cleanSession	boolean	—	true
20	reconnectDelayMin	integer	—	5
21	reconnectDelayMax	integer	—	512
22	clientId	string	—	zebra-rfd40-01

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
23	username	string	—	mqttuser
24	password	string	—	xxxxxx
25	publishTopics	array	—	—
26	topic	string	—	zebra/rfd40/01/data
27	qos*	integer	—	1
28	retain*	boolean	—	false
29	subscriptions	array	—	—
30	topic*	string	—	zebra/rfd40/01/data
31	qos*	integer	—	0

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
32	reta [*] i n	boolean	—	—
33	securityParams	object	—	—
34	format [*]	enum	PEM PFX	PEM
35	algorithm	enum	RSA256 RSA512	RSA256
36	caCertificateFile	string	—	cloud-trusted-ca-certs.crt
37	clientCert	string	—	client_cert.pem
38	clientKey	string	—	client_key.pem
39	eventConfiguration	object	—	—
40	antenna	boolean	—	false
41	terminalConnection	boolean	—	false
42	firmwareUpdate	boolean	—	true

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
43	gpi	boolean	—	false
44	netwo rk	boolean	—	true
45	excep tions	boolean	—	false
46	ntp	boolean	—	true
47	userA pp	boolean	—	false
48	heart beat	boolean	—	false
49	power	boolean	—	true
50	batte ry	boolean	—	true
51	tempe ratur e	boolean	—	false
52	fileD ownlo ad	boolean	—	true
53	cpuUs age	boolean	—	false
54	flash Usage	boolean	—	false
55	ramUs age	boolean	—	false
56	heart beatC onfig urati on	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
57	interval	integer	max 65535	100
58	inventoryStatus	boolean	—	true
59	batteryStatus	boolean	—	true
60	userApps	boolean	—	—
61	cpuThreshold	integer	—	0
62	ramThreshold	integer	—	0
63	flashThreshold	integer	—	0
64	temperatureThreshold	integer	—	0

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
65	batching	object	—	—
66	reportingInterval	integer	1000–60000	5000
67	maxPayloadSizePerReport	integer	1024–5120	1024
68	savedEndpoints	object	—	—
69	epNames	array	—	["mdm_ep", ...]
70	response *	object	—	—
71	code *	integer	0–101	0
72	description *	string	—	Success

Response Semantics

Field Support. Each endpoint's `configuration.eventConfiguration` selects which events that endpoint emits. The contract declares the full set below, **but only 7 of the boolean toggles are supported today**; the remaining 13 keys are accepted by the firmware and returned in the read-back but **have no effect yet** (reserved for a future release). Enabling a reserved toggle silently does nothing.

Field	Type	Status	Description
<code>terminalConnection</code>	boolean	 Supported	Terminal connect/disconnect events.
<code>firmwareUpdate</code>	boolean	 Supported	Firmware-update events.

Field	Type	Status	Description
<code>network</code>	boolean	 Supported	Network events.
<code>heartbeat</code>	boolean	 Supported	Periodic heartbeat events.
<code>power</code>	boolean	 Supported	Power-source events.
<code>battery</code>	boolean	 Supported	Battery events.
<code>temperature</code>	boolean	 Reserved	Declared in the schema, but no temperature alert is emitted on V1.1 — the toggle is stored and read back, and does nothing.
<code>fileDownload</code>	boolean	 Supported	File-download events.
<code>antenna</code>	boolean	 Reserved	Not yet functional.
<code>gpi</code>	boolean	 Reserved	Not yet functional.
<code>exceptions</code>	boolean	 Reserved	Not yet functional.
<code>ntp</code>	boolean	 Reserved	Not yet functional.
<code>userApp</code>	boolean	 Reserved	Not yet functional.
<code>cpuUsage</code>	boolean	 Reserved	Not yet functional.
<code>flashUsage</code>	boolean	 Reserved	Not yet functional.
<code>ramUsage</code>	boolean	 Reserved	Not yet functional.

Field	Type	Status	Description
<code>cpuThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>cpuUsage</code>).
<code>ramThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>ramUsage</code>).
<code>flashThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>flashUsage</code>).
<code>temperatureThreshold</code>	integer	 Reserved	Not yet functional.
<code>heartbeatConfiguration</code>	object	 Conditional	<code>{ interval, inventoryStatus, batteryStatus, userApps }</code> . Returned only after <code>config_events</code> enables the heartbeat (not by default). <code>userApps</code> is not currently functional.

The **7 supported** booleans are exactly the ones you can rely on today; configure event routing with `config_events`.

- **Active vs saved.** `activeEndpoints.epConfig[]` contains a full configuration only for endpoints with `activate: true`. `savedEndpoints.epNames[]` lists **every** saved endpoint — so a name that appears in `epNames` but not in `epConfig[]` is saved but inactive.
- `verificationType` **reads back as you sent it.** The reader returns the value you set with `config_endpoint`, such as `VERIFY_HOST_AND_PEER`; a plain-MQTT endpoint reads back `VERIFY_NONE`. Because one enum governs both directions, you can echo a read-back value in a later `update`.
- **Topic items and the `protocol` literal.** Each `publishTopics` / `subscribeTopics` entry carries `topic`, `qos`, and `retain`. The contract requires `qos` and `retain` on a publish item, and `topic`, `qos` and `retain` on a subscribe item — but its own read-back examples omit `retain` on every subscribe item, and an MQTT subscription has no retain flag, so do not reject a subscribe entry that lacks it. The TLS protocol literal does not differ by direction: you send `MQTT_TLS` with `config_endpoint` and read the same `MQTT_TLS` back here, so a validator can hold one enum — `MQTT` or `MQTT_TLS` — for both directions.
- **Topic slots read back in configured order.** `publishTopics` entries are returned in the order they were set with `config_endpoint`: `[0]` = command responses (`.../clients/resp`), `[1]` = events (`.../clients/event`), `[2]` = the endpoint's connection-event and **Last Will** topic (`.../clients/rfid`), with `subscribeTopics[0]` = command ingress (`.../clients/cmdnd`). A `DATA1` endpoint configured with a placeholder at `[0]` reads the tag-data topic back at `[1]`, as the contract's `DATA1` read-back shows. A read-back with a single entry is read from index `0` upward,

so that entry sits in the **reply** slot and no tag data is flowing. Match entries by **index**, not base name — the `resp` / `event` / `rfid` names are conventions.

Response Codes

`get_endpoint_config` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the endpoint configuration was retrieved.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the configuration at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

Examples

Publish `get_endpoint_config`; the device replies with the envelope shown under [Response Payload](#). The all-endpoints request (`command` + `requestId`) returns every active endpoint plus the saved-names list; the single-endpoint request (shown under [Request Payload](#)) targets one endpoint by name.

Active vs saved. In the response example, `activeEndpoints.epConfig[]` carries the full config of the two active endpoints (`mdm_ep2` and `data1_ep`), while `savedEndpoints.epNames` lists all five saved (`mdm_ep`, `mdm_ep2`, `ctrl_ep`, `data1_ep`, `mgmt_ep`) — the three that are saved but not active appear **only** as names, with no config block. When more than one endpoint is active, `epConfig[]` holds one full configuration per active endpoint.

Notes and Caveats

⚠ THE RESPONSE RETURNS BROKER CREDENTIALS

`configuration.mqttParams.password` (and `username`, when set) are part of the read-back. Redact them in logs, tickets, and screenshots; transport and store the response only over secured channels. The examples on this page carry placeholders, not real credentials.

Read `get_endpoint_config` **before** `config_endpoint` to review the current wiring and avoid clobbering an active endpoint. For the concept-level view, see [Endpoints and Topic Slots](#).

Related:  [Endpoints and Topic Slots](#) ·  [config_endpoint](#) ·  [config_events](#) ·  [get_wifi](#) ·  [Response Codes](#)

get_eth

REFERENCE

API consumer

read the reader's wired Ethernet interface configuration and status

Overview

`get_eth` retrieves the reader's **wired Ethernet interface configuration and status** — interface state, live link status and speed, and the DHCP-assigned IPv4 addressing. It is a read-only query on the device-management ([MGMT](#)) endpoint, useful for verifying that the reader's wired uplink is up and reaching back-end services.

The response shape depends on interface state: when Ethernet is **enabled** the device returns the full `interfaceDetails` (hostname, MAC, link, IPv4); when it is **disabled** it returns only `interfaceName` and `status` (still a `code 0` success). Note that, per the command's own contract, **static IP configuration and IPv6 are not available in the current API version**, and the 802.1X `security` block is not populated (see [Response Semantics](#) below).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_wifi</code> , <code>get_status</code>

When to Use

Use `get_eth` when you need to **verify the reader's wired network state** — during commissioning, provisioning verification, or troubleshooting a wired uplink. Because it is read-only and idempotent, it is safe to poll.

- **Confirm connectivity:** check `interfaceDetails.status` (enabled?), `linkStatus.status` (cable connected?), and `linkStatus.linkSpeed` (negotiated speed).
- **Validate addressing:** check `ipv4Configuration.ipAddress`, `gateway`, and `enableDhcp` to confirm the reader has a working IPv4 route to the broker / back-end.
- **Troubleshoot wired issues:** an unexpected `linkSpeed`, a missing IP, or a `Disabled` interface each point to a specific cable/switch/config problem.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `ethConfig.interfaceDetails.status` (response) — `Enabled` or `Disabled`. When `Disabled`, only `interfaceName` and `status` are returned.
- `ethConfig.interfaceDetails.linkStatus` (response) — `status` (`Connected` / `Disconnected`) and `linkSpeed`; the live carrier state.

- `ethConfig.interfaceDetails.ipv4Configuration` (response) — the DHCP-assigned IPv4 addressing (reported read-only).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_eth" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_eth	Fixed command name. Retrieves the Ethernet configuration.
2	requestId *	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and an `ethConfig` object. On the **Schema** tab, expand `ethConfig` → `interfaceDetails` (and its `linkStatus`, `ipv4Configuration`, `security` sub-objects) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full (enabled-interface) JSON. Within `interfaceDetails`, only `interfaceName` and `status` are required; the rest are returned when the interface is **enabled**. The `security` block is reserved (not available in the current API version), and `ipv4Configuration` is reported read-only.

Interface state

Ethernet config info ▾

Response payload

Example: Interface enabled — full config

```
1  {
2    "command" : "get_eth" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "ethConfig" : {
6      "interfaceDetails" : {
7        "interfaceName" : "eth0" ,
8        "status" : "Enabled" ,
9        "hostname" : "RFD40-211545201D0020" ,
10       "macAddress" : "AC:3F:A4:00:2C:89" ,
11       "linkStatus" : {
12         "status" : "Connected" ,
13         "linkSpeed" : "100Mbps"
14       } ,
15       "ipv4Configuration" : {
16         "ipAddress" : "192.168.0.101" ,
17         "subnetMask" : "255.255.255.0" ,
18         "gateway" : "192.168.0.1" ,
19         "enableDhcp" : true ,
20         "dnsServer" : "0.0.0.0" ,
21         "domainName" : "zebra.com"
22       }
23     } ,
24   } ,
25   "response" : {
26     "code" : 0 ,
27     "description" : "Success"
28   }
29 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_eth	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command; emitted per-handler, not device-wide.
4	ethConfig	object	—	—	Ethernet configuration container. Present on a successful read; optional at the envelope level.
5	interface Details	object	—	—	Details of the Ethernet interface. <code>interfaceName</code> and <code>status</code> are always present; the rest are present only when the interface is enabled.
6	interfaceName *	string	—	eth0	OS-level interface name.
7	status *	enum	Enabled Disabled	enabled	Interface enable state. When Disabled, the device returns only <code>interfaceName</code> and <code>status</code> .
8	hostname	string	—	RFD40AB03	Hostname bound to the interface (present when enabled).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
9	macAddress	string	—	E0-D0-45-3D-38-1D	MAC address of the interface (present when enabled).
10	linkStatus	object	—	—	Live carrier state and negotiated speed (present when enabled).
11	status *	enum	Connected Disconnected	Connected	Physical link state (required within linkStatus).
12	linkSpeed *	string	—	100Mbps	Negotiated link speed (required within linkStatus).
13	ipv4Configuration	object	—	—	IPv4 addressing (present when enabled). Reported read-only — DHCP-assigned; static IP is not settable via this API (see Response Semantics).
14	ipAddress	string	ipv4	192.168.1.81	IPv4 address of the interface.
15	subnetMask	string	ipv4	255.255.255.0	Subnet mask.
16	gateway	string	ipv4	192.168.1.144	Default gateway.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
17	enableDhcp	boolean	—	true	Whether DHCP is enabled for IPv4 assignment.
18	dnsServer	string	ipv4	192.168.1.78	DNS server address.
19	domainName	string	—	test.soti.com	Domain name.
20	security	object	—	—	802.1X security block. Reserved — not available in the current API version (planned for a future release); not populated. See Response Semantics.
21	securityStatus	boolean	—	true	Whether 802.1X security is enabled.
22	EAP_802_1X	object	—	—	802.1X EAP parameters (reserved).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
23	authentication	enum	EAP-MD5 LEAP EAP-SIM EAP-AKA EAP-TLS EAP-TTLS EAP-FAST	PEAP	Outer EAP authentication method.
24	inner authentication	enum	EAP-MS-CHAPv2 EAP-TLS EAP-GTC	EAP-MS-CHAPv2	Inner authentication method.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
25	certificate	string	—	client45	Certificate name used for EAP-TLS.
26	response *	object	—	—	Result wrapper.
27	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
28	description *	string	—	Success	Human-readable status.

Response Semantics

- **Not available in the current API version.** Per this command's contract, **static IP configuration and IPv6 are not available in the current API version.** The response schema models a `security` / `EAP802_1X` block and a full `ipv4Configuration`, but:
 - the `security` block is **reserved** (planned for a future release) and is **not populated**;
 - `ipv4Configuration` is **reported read-only** — the reader's addressing is DHCP-assigned, and you **cannot set a static IP** through this API;
 - **no IPv6** addressing is returned.
- **Disabled interface.** When Ethernet is disabled, the response contains only `ethConfig.interfaceDetails.interfaceName` and `status (Disabled)` — `linkStatus`, `ipv4Configuration`, `hostname`, and `macAddress` are absent. This is normal behavior and a `code 0` success, not an error.

Response Codes

`get_eth` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — Ethernet configuration retrieved and returned. A disabled interface is also a 0 success (minimal payload), not an error.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the interface details at this time. Retry after a short delay; if it persists, reboot the device.
101	Default error — generic catch-all.

Examples

`get_eth` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The **Interface state** dropdown under [Response Payload](#) shows both replies: the full reply for an enabled interface, and the minimal one for a disabled interface, which carries only `interfaceName` and `status` and is still a `code 0` success.

Notes and Caveats

`get_eth` is a read-only getter — safe to poll during commissioning or troubleshooting. For the concept-level view of wired networking, see [Network Configuration](#); for Wi-Fi, use `get_wifi`.

Related:  [Network Configuration](#) ·  `get_wifi` ·  `get_status` ·  Network and Connectivity ·  [Response Codes](#)

get_installed_certificates

REFERENCE

API consumer

list the certificates installed in the reader's trust store

Overview

`get_installed_certificates` lists the certificates in the reader's trust store, returning per-certificate metadata — name, type, issuer/subject, serial, public-key algorithm, and validity dates — under `installedCerts.certInfo[]`. It is read-only and returns **no private-key material**. Use it to audit the certificate inventory and to confirm a certificate exists (and is valid) before referencing it from `config_endpoint` or `set_wifi`.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	install certificate , delete certificate , config_endpoint

When to Use

Use `get_installed_certificates` to audit or verify the reader's certificate inventory:

- **Before configuring TLS** — confirm the `ca_cert` / `client_cert` referenced by `config_endpoint` (`MQTT_TLS`) or `set_wifi` (Enterprise) is installed and of the right `type`.
- **Rotation planning** — check `validTill` (expiry) and `validFrom` (not-yet-valid) to schedule renewals before TLS handshakes start failing.
- **Trust audit** — review `issuerName` / `subjectName` / `serial` to confirm the expected CA chain.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `certInfo[].type` (response) — the certificate's scope (`mqtt`, `wifi`, `filestore`, `client`, `server`); a certificate installed for one type is not usable by another.
- `certInfo[].name` (response) — the certificate's name. Published examples show component identifiers (`ca_cert`, `client_cert`), not the logical name given to `install_certificate`; which the reader reports is unresolved.
- `certInfo[].validTill` / `validFrom` (response) — the validity window; an expired or not-yet-valid certificate causes TLS handshake failures.
- `certInfo[].issuerName` / `subjectName` / `serial` (response) — identify the certificate and its CA chain (note `serial` is truncated — see [Response Semantics](#)).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request is a minimal envelope — just `command` (the plural literal) and `requestId`. The command returns the full inventory; there are no filter parameters.

Request payload

```
1 {
2   "command" : "get_installed_certificates" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_installed_certificates	Fixed command name — plural get_installed_certificates (see the command-name below).
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.

Request Semantics

- **Command literal.** Send `command: "get_installed_certificates"`. An unknown or bogus command value is silently dropped (no error, no reply), so a missing reply usually means a wrong command string.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the inventory under `installedCerts.certInfo[]` . The **Example** tab shows a five-certificate trust store: CA certificates for `filestore` , `wifi` and `mqtt` , and client certificates for `mqtt` and `wifi` .

Response payload

```
1  {
2    "command" : "get_installed_certificates" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.21" ,
5    "installedCerts" : {
6      "certInfo" : [
7        {
8          "name" : "ca_cert" ,
9          "type" : "filestore" ,
10         "issuerName" : "SotiConnectRoot" ,
11         "publicKeyAlg" : "RSA" ,
12         "publicKey" : "RSA" ,
13         "serial" : "ed:83:dd:4a:47:dd:07:98" ,
14         "subjectName" : "SotiConnectRoot" ,
15         "validFrom" : "2024-07-01T07:00:00Z" ,
16         "validTill" : "2034-02-28T15:18:47Z"
17       },
18       {
19         "name" : "ca_cert" ,
20         "type" : "wifi" ,
21         "issuerName" : "CA" ,
22         "publicKeyAlg" : "RSA" ,
23         "publicKey" : "RSA" ,
24         "serial" : "1e:e0:12:f6:75:68:39:f1" ,
25         "subjectName" : "CA" ,
26         "validFrom" : "2024-07-01T07:00:00Z" ,
27         "validTill" : "2025-04-16T07:26:35Z"
28       },
29       {
30         "name" : "ca_cert" ,
31         "type" : "mqtt" ,
32         "issuerName" : "SOTI-mqtt-ca" ,
33         "publicKeyAlg" : "RSA" ,
34         "publicKey" : "RSA" ,
35         "serial" : "7d:1b:c5:9f:ea:ff:0f:56" ,
36         "subjectName" : "SOTI-mqtt-ca" ,
37         "validFrom" : "2024-07-01T07:00:00Z" ,
38         "validTill" : "2038-01-19T03:14:07Z"
39       },
40       {
41         "name" : "client_cert" ,
42         "type" : "mqtt" ,
43         "issuerName" : "RootCA" ,
44         "publicKeyAlg" : "RSA" ,
45         "publicKey" : "RSA" ,
46         "serial" : "1" ,
47         "subjectName" : "Client" ,
48         "validFrom" : "2024-07-01T07:00:00Z" ,
49         "validTill" : "2030-05-06T08:06:57Z"
```

```
50     },
51     {
52         "name" : "client_cert" ,
53         "type" : "wifi" ,
54         "issuerName" : "CA" ,
55         "publicKeyAlg" : "RSA" ,
56         "publicKey" : "RSA" ,
57         "serial" : "26:00:16:e0:c4:a1:b5:00" ,
58         "subjectName" : "CLIENT" ,
59         "validFrom" : "2024-07-01T07:00:00Z" ,
60         "validTill" : "2025-04-16T07:27:52Z"
61     }
62 ]
63 },
64 "response" : {
65     "code" : 0 ,
66     "description" : "Success"
67 }
68 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_installed_certificates	Echoes the ex command (plu
2	requestId *	string	—	abc123	Echoes the re requestId — reply to requ
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.21	API version re this command per-handler, n wide.
4	installedCerts	object	—	—	The certificate Present on suc (Not in the env required set.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	certInfo	array	—	—	One entry per certificate. Each requires type, issuerName, subjectName, validFrom, validTill.
6	name	string	—	Reader Main Certificate	The name of the certificate. Even published examples carry a certificate component identifier here (ca_certificate, client_certificate) than the logical <u>install_certificate</u> accepts; which the reader reports unresolved.
7	type*	enum	client server mqtt wifi filestore	server	Certificate scope. Certificates are categorized by type — one for mqtt is not for wifi.
8	installTime	string	date-time	2022-07-01T03:01:31.464Z	Install timestamp. Optional — the reader often omits it.
9	issuerName	string	—	FX9600EE5729	Issuer common name

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	publicKeyAlg	enum	RSA RSAMD5 RSA_SHA_1 RSA_SHA_256 RSA_SHA_384 RSA_SHA_512	RSA	Public-key algorithm reported by the device (e.g. RSA).
11	publicKeyField	string	—	091234acgbe12	Public-key field device current populates this algorithm name than key bytes Response Semantics
12	serial	string	—	ed:83:dd:4a:47:dd:07:98	Certificate serial number in colon-hex — the first 6 bytes of the full certificate (the device true serial number see Response Semantics).
13	subjectName	string	—	FX9600EE572	Subject common name
14	validFrom	string	date-time	2024-07-01T07:00:00Z	Start of the valid time window.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
15	validity	string	date-time	2034-02-28T15:18:47Z	End of the validity window.
16	response	object	—	—	Status envelope read.
17	code	integer	0-101	0	0 = Success; non-zero = error. Requires contract. See Codes below.
18	description	string	—	Success	Human-readable

Response Semantics

- serial** is truncated to 8 bytes. The device returns the certificate serial as colon-delimited hex, but only the **first 8 bytes** of the (typically 20-byte) real serial — e.g. `7d:ef:d4:c3:2d:39:4a:0b`. Combine **serial** with **name** + **type** + **issuerName** to identify a certificate uniquely; **serial** alone is not a globally unique identifier.
- Private keys are never listed.** The inventory returns public certificate metadata only. A **client_key** installed via [install_certificate](#) does **not** appear in `certInfo[]` — only its paired **ca_cert** / **client_cert** do. There is no way (by design) to read back private-key material.
- publicKey** and **publicKeyAlg** quirks. The device currently populates **publicKey** with the algorithm name (e.g. `RSA`) rather than actual key bytes — treat **publicKey** as informational, not as usable key material. **publicKeyAlg** reports the certificate's public-key algorithm — an `RSA` variant (see the allowed values under [Response Payload](#)).

Response Codes

`get_installed_certificates` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the inventory was retrieved (an empty trust store still returns 0 with an empty/absent <code>certInfo</code>).
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the certificate list at this time. Retry after a short delay; if persistent, reboot.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a backing certificate file could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the requested certificate is not installed.
101	Default error — generic catch-all.

[Codes](#) 3 / 9 / 21 are this command's mapped failure codes. The request takes no parameters; send exactly `command` + `requestId`.

Examples

Publish the plural `get_installed_certificates` with a `requestId`; the device replies with the inventory shown under [Response Payload](#). The command always returns the full list and takes no parameters (there are no request variants).

Notes and Caveats

Read-only, but still sensitive. The inventory exposes no secrets (no private keys), but a certificate list reveals the device's PKI/trust topology — redact it in shared logs, tickets, and screenshots. For the concept-level view, see [TLS and Authentication](#) and [Certificate Management](#).

Related:  [TLS and Authentication](#) ·  `install certificate` ·  `delete certificate` ·  `config_endpoint` ·  `set_wifi` ·  [Response Codes](#)

get_status

REFERENCE

API consumer

request an on-demand device-health snapshot

Overview

`get_status` retrieves a point-in-time health and status snapshot of the reader. It is a read-only query on the device-management ([MGMT](#)) endpoint: the app publishes the command and the device publishes a correlated reply containing power, radio, clock, terminal-link, and battery information under `deviceStatus`.

Because it is read-only, `get_status` is safe to call at any time. It complements the asynchronous `heartBeatEVT` push: `heartBeatEVT` carries only a lightweight inventory-status or battery-alert summary, so use the event stream for continuous monitoring and `get_status` for an immediate, full snapshot.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , requestId
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	V1.21 (API Version Reporting)
Related	get version , get current region , reboot , set operating mode

When to Use

Use `get_status` when you need an **immediate, correlated snapshot of reader health** — power source, radio activity/connection, system clock, terminal link, and battery condition — typically in a management workflow over the MGMT endpoint. It confirms device state on demand rather than waiting for the next asynchronous heartbeat.

- **Get:** the current `deviceStatus` snapshot (power, radio, clock, terminal connection, battery).
- **Use it for:** pre-flight safety checks before a state-changing command, spot health checks, and diagnostics. (`deviceStatus.radioActivity` in the response reports the activity status of the

device's radio — see [Key Fields](#).)

- **Prefer a different command when:** you want continuous monitoring rather than a single read — subscribe to `heartBeatEVT` instead for periodic pushes, though it carries only an inventory-status or battery-alert summary, not the full power/radio/clock/terminal/battery snapshot `get_status` returns.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes. To watch a whole fleet rather than one reader, put the MQTT single-level wildcard `+` in the serial slot — `<tenantId>/MGMT/clients/resp/+` — as explained under [Topics and Channels](#).

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `deviceStatus.radioActivity` (response) — the activity status of the device's radio (`INACTIVE` / `ACTIVE`).
- `deviceStatus.powerSource` (response) — how the reader is currently powered (`WALLCHARGER` / `USB` / `CRADLE` , the contract's enum); current firmware has also been reported to return `NONE` , which the enum does not allow.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_status" ,
3   "requestId" : "abcd123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_status	Fixed command name. Used to get the reader status information.
2	requestId *	string	—	abcd123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command` , `requestId` , `apiVersion` , `response` , and an optional `deviceStatus` snapshot. On the **Schema** tab, expand `deviceStatus` (and its `terminalConnection` , `batteryStatus` sub-objects) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON. `deviceStatus` is optional at the envelope level; which fields appear depends on device state and model — the **Example** tab shows a representative snapshot.

Response payload

```
1  {
2    "command": "get_status",
3    "requestId": "abcd123",
4    "apiVersion": "V1.21",
5    "deviceStatus": {
6      "powerSource": "USB",
7      "radioActivity": "INACTIVE",
8      "radioConnection": "DISCONNECTED",
9      "systemTime": "2026-06-15T09:26:00.443Z",
10     "ntp": {
11       "offset": 0,
12       "reach": 0
13     },
14     "terminalConnection": {
15       "status": "CONNECTED",
16       "type": "USB"
17     },
18     "batteryStatus": {
19       "mfgDate": "22DEC2024",
20       "cycleCount": 8,
21       "fullChargeCapacity": 5739,
22       "temperature": 24,
23       "designCapacity": 6400,
24       "batteryType": "LI-ION",
25       "capacity": 2467,
26       "stateOfHealth": "GOOD",
27       "chargePercentage": 46,
28       "chargeStatus": 1
29     }
30   },
31   "response": {
32     "code": 0,
33     "description": "Success"
34   }
35 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_status	Echoes the executed command
2	requestId *	string	—	abc123	Echoes the request's request ID — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.21	API version the device reports
4	deviceStatus	object	—	—	Status snapshot; optional at envelope level. Which deviceStatus fields appear depends on device state and model — see the Example.
5	powerSource *	enum	WALLCHARGER USB CRADLE	CRADLE	The source of power for the device. The contract's enum WALLCHARGER, USB or CRADLE. Current firmware has also been reported to return NONE (no external power source attached which is outside the enum).
6	radioActivity *	enum	INACTIVE ACTIVE	ACTIVE	The activity status of the device radio.
7	radioConnection *	enum	CONNECTED DISCONNECTED	CONNECTED	Connection status of the RF radio.
8	hostname	string	—	RFD40-212735201D0053	Device hostname on the network. Optional; may be absent.
9	systemTime *	string	date-time	2022-07-01T03:01:31.464Z	Device system time in ISO 8601

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	temperature	integer	—	32	Device temperature (distinct from the battery temperature below).
11	ntp	object	—	—	Details about the Network Time Protocol (NTP) synchronization.
12	offset	integer	—	120	The offset in milliseconds between the device's clock and the NTP server. Declared in schema, but current firmware does not populate it — the value shown is the schema's own example, not a device reading. See Response Semantics .
13	reach	integer	—	321	The reachability score of the NTP server. Declared in the schema, but current firmware does not populate it — the value shown is the schema's own example, not a device reading. See Response Semantics.
14	terminalConnection	object	—	—	Host-terminal link to the selected terminal.
15	status	enum	CONNECTED DISCONNECTED	CONNECTED	Terminal connection status.
16	type	enum	BLUETOOTH CIB USB	USB	Terminal transport type.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
17	bat ter ySt atu s	object	—	—	Battery status information.
18	m f g D a t e	string	—	22DEC2024	Battery manufacture date.
19	c y c l e C o u n t	integer	—	120	Completed battery charge cycles.
20	f u l l C h a r g e C a p a c i t y	integer	—	3200	Current full-charge capacity.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
21	temperature	integer	—	32	Battery temperature (distinct from the device temperature above).
22	design capacity	integer	—	7000	Designed battery capacity.
23	battery type	string	—	LI-ION	Battery chemistry / type.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
24	capacity	integer	—	5000	Remaining battery capacity.
25	stateOfHealth	enum	GOOD CRITICAL MODERATE	GOOD	Long-term battery capacity / health. The same value set the <code>batteryAlert.stateOfHealth</code> carried by alerts and heartBeatEVT — one shared vocabulary, so nothing need converting. See Response Semantics.
26	chargePercentage	integer	—	23	Percentage of charge remain

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
27	chargingStatus	integer	—	1	Charging status: 0 = charge connected; 1 = charger connected, charging; 2 = charger connected, battery at 100%.
28	response *	object	—	—	Result wrapper.
29	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
30	description *	string	—	Success	Human-readable status.

Response Semantics

- Which `deviceStatus` fields appear depends on device state and model — absent fields are normal behavior, not an error. Note that `deviceStatus` reports two distinct temperatures: `deviceStatus.temperature` (device) and `deviceStatus.batteryStatus.temperature` (battery) — do not conflate them.
- The reader has no RTC backup battery. After a cold start or factory reset, `systemTime` starts at a baseline default; the device syncs its clock from **network time** once it has a network connection. There is currently no API to change the NTP configuration — set the NTP server details (and, if NTP is unreachable, the device time itself) from the **123RFID Desktop** application. Time-sensitive operations (TLS certificate validity, log/event timestamps) may use the default time until the clock has synced.
- `stateOfHealth` is one grade, on two surfaces. `deviceStatus.batteryStatus.stateOfHealth` here and the `batteryAlert.stateOfHealth` carried by `alerts` and `heartBeatEVT` use the **same** enum — `GOOD` / `CRITICAL` / `MODERATE` — so shared battery logic needs no conversion. Keep it distinct

from the event's `status`, which is a charge band (`LOW` / `CRITICAL` / `CHARGING` / `FULL` / `HIGH`), not a health grade. See Interpret `stateOfHealth`.

- `ntp.offset` and `ntp.reach` **are not reported by the reader**. The schema declares both fields; the values on the **Schema** tab (`120` and `321`) are the schema's own field examples, and the contract's message example shows `0` for each. Current firmware has been reported not to populate either one, so treat their absence as normal and do not build clock-health logic on them. Removing them from the schema is an upstream request to the API owners; until then the fields remain declared.

Response Codes

`get_status` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — status retrieved and returned.
3	Not able to retrieve information — the reader could not gather the requested status at this moment. Retry; if persistent, reboot after stopping any active inventory.
101	Default error — generic catch-all.

Examples

`get_status` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abcd123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

Notes and Caveats

- `get_status` is the on-demand poll; `heartBeatEVT` and related alerts push a lighter periodic summary (an inventory-status or battery-alert block), not the full `get_status` snapshot. Prefer the events for continuous monitoring and `get_status` for a complete point-in-time check. Cross-command `apiVersion` reporting can differ (for example `mqttConnEVT` emits a numeric `apiVersion`); that is a firmware note, not a `get_status` schema difference.

Related:  [Management](#) ·  [Response Codes](#) ·  [MQTT API](#)

get_version

REFERENCE

API consumer

read the reader's identity and software versions

Overview

`get_version` retrieves the reader's **identity and software-version metadata** in a single round trip: model, serial number, SKU, top-level firmware version, and a component-level breakdown (scanner, radio, and IoT Connector versions). It is a read-only query on the device-management ([MGMT](#)) endpoint — the app publishes the command and the device publishes a correlated reply containing the `readerVersion` object.

Because it is idempotent, read-only, and independent of radio state, `get_version` can be issued **at any point once the connection is active**, and it is the natural first command to send after any connect, reconnect, or bootstrap — to confirm which device you are talking to and which command set it supports. Where `get_version` reports *identity*, `get_status` reports *live runtime health*.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.21 (API Version Reporting)
Related	<code>get_status</code> , <code>get_current_region</code> , <code>set_os</code>

When to Use

Use `get_version` when you need to **establish device identity and the software baseline** — typically in a management or fleet workflow over the MGMT endpoint. It answers "is this the

device I expect, and what command set does it support?"

- **Identify the device:** `readerVersion.model`, `serialNumber`, and `sku` confirm the exact model and regional/hardware variant, and key asset-tracking, support tickets, and device registration to the physical label.
- **Gate on firmware / capabilities:** compare `readerVersion.firmwareVersion` and `detailedVersions` against your expected baseline before an update or when diagnosing an issue; read `detailedVersions.iotcVersion` to decide which MQTT API commands and features are available.
- **Confirm a firmware update applied:** `set os` returns only an *accepted* acknowledgment (`code 1`) with no terminal success/failure on the response topic — poll `get_version.readerVersion.firmwareVersion` to verify the new firmware actually took effect.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `readerVersion.model` (response) — `RFD40` or `RFD90`; branch model-specific configuration on it.
- `readerVersion.firmwareVersion` (response) — the reader firmware build; compare against your baseline and use it to confirm a `set os` update applied.
- `readerVersion.detailedVersions.iotcVersion` (response) — the IoT version, which determines the available command/feature set.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_version" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_version	Fixed command name. Retrieves the reader's identity and version information.
2	requestId *	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and a `readerVersion` object. On the **Schema** tab, expand `readerVersion` (and its `detailedVersions` sub-object) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON.

`readerVersion` is optional at the envelope level, but when it is present its six identity members (`firmwareVersion`, `model`, `serialNumber`, `sku`, `companyName`, `manufacturerName`) are all required; `detailedVersions` and its members are optional.

Response payload

```
1      {
2          "command" : "get_version" ,
3          "requestId" : "abc123" ,
4          "apiVersion" : "V1.21" ,
5          "readerVersion" : {
6              "firmwareVersion" : "PAAFKS00-013-N05" ,
7              "model" : "RFD40" ,
8              "serialNumber" : "211545201D0034" ,
9              "sku" : "RFD4031-G10B700-US" ,
10             "companyName" : "Zebra Technologies" ,
11             "manufacturerName" : "Zebra Technologies" ,
12             "detailedVersions" : {
13                 "scannerFirmware" : "PAAEOC20-003-R01" ,
14                 "radioFirmware" : "2.0.53.0" ,
15                 "iotcVersion" : "V1.22"
16             }
17         } ,
18         "response" : {
19             "code" : 0 ,
20             "description" : "Success"
21         }
22     }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_version	Echoes the executed command.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abcd123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API/IoTC protocol version reported by this command. See the caveat in Response Semantics — it is emitted per-handler, not device-wide.
4	readerVersion	object	—	—	Reader identity and software-version details. Optional at the envelope level; when present, all six identity members below are required.
5	firmwareVersion *	string	—	PAAFKS00-006-R02	The firmware version of the reader.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	model	enum	RFD40 RFD90	RFD40	The reader model.
7	serialNumber	string	—	23053520102096	The reader serial number (matches the device label).
8	sku	string	—	RFD4031-G10B700-US	The stock-keeping unit — identifies the regional / hardware variant.
9	companyName	enum	Zebra Technologies	Zebra Technologies	The company name associated with the reader.
10	manufacturerName	enum	Zebra Technologies	Zebra Technologies	The manufacturer name associated with the reader.
11	details	object	—	—	Component-level version breakdown. Optional.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
12	s c a n n e r F i r m w a r e	string	—	PAAEOC20-003- R01	Firmware version of the scanner component.
13	r a d i o F i r m w a r e	string	—	2.0.42.0	Firmware version of the RFID radio component.
14	i o t c v e r s i o n	enum	V1.1 V1.2 V1.21	V1.1	The version of the IoT Connector (IoTc) software.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
15	response *	object	—	—	Result wrapper.
16	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
17	description *	string	—	Success	Human-readable status.

Response Semantics

- **Version fields.** Use `readerVersion.detailedVersions.iotcVersion` as the IoT C software version, and compare firmware builds via `readerVersion.firmwareVersion`. The envelope `apiVersion` is per-handler — see [API Version Reporting](#).
- `iotcVersion` **can be newer than its enum.** The contract's example reports `v1.22`, but the field's enum lists only `v1.1`, `v1.2` and `v1.21`, so the contract's own example does not validate against it — a known issue in the source schema. Parse `iotcVersion` as a string and compare versions numerically rather than rejecting an unlisted value.
- `companyName` and `manufacturerName` are constrained to `Zebra Technologies`. `readerVersion.detailedVersions` is optional and may be absent on some builds; the example values shown (firmware, SKU, serial) are illustrative — real values vary by unit and region.

Response Codes

`get_version` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — identity and version details retrieved and returned.
101	Default error — generic catch-all.

Examples

`get_version` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

Notes and Caveats

- Confirming a firmware update: `set_os` returns an **acceptance response only** — `code 1` ("Command payload is accepted") on the response topic, with no terminal success or failure. While the update runs, the reader emits live progress events: a `FIRMWARE_UPDATE` `alert` carrying `alertDetails.fwUpdateStatus` (`updateStatus` , `overallProgress` , `stage`), provided firmware-update events are enabled with `config_events` (`eventConfiguration.firmwareUpdate: true`). To confirm the update applied, watch those events, or poll `get_version` and check that `readerVersion.firmwareVersion` advanced to the expected build.

`get_version` is idempotent and independent of radio state, so it can be issued at any time on an active connection and is the right **first command after any connect, reconnect, or bootstrap** — confirm the device identity and IoT version before relying on newer commands or fields. See [Management](#).

Related:  [Management](#) ·  `get_status` ·  `set_os` ·  [Response Codes](#)

get_wifi

REFERENCE

API consumer

read the reader's saved Wi-Fi profiles and connection status

Overview

`get_wifi` retrieves the **Wi-Fi profiles saved on the reader** together with their current connection status and configuration. The response is a list (`wifiProfiles.wifiConfig[]`) of profiles; each describes the wireless interface, the associated access point (ESSID, security type, connection status), the IPv4/IPv6 addressing, and a `securityDetails` block keyed by the security type (`WPA2Personal` , `WPA3Personal` , `WPA2Enterprise` , `WPA3Enterprise`). It is a read-only query on the device-management ([MGMT](#)) endpoint.

The device **masks** stored credentials — passwords, passphrases, and identities are returned as a mask, never the real secret. Use `get_wifi` to verify which SSID a reader is connected to and confirm its addressing before relying on the wireless uplink.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	set_wifi , delete_wifi_profile , get_eth

When to Use

Use `get_wifi` when you need to **verify the reader's wireless configuration** — during commissioning, a connectivity check before inventory, or a fleet audit of which SSID each device is on.

- **Confirm the connection:** check `accessPoint.status` (CONNECTED?), `accessPoint.essid` (correct network?), and `accessPoint.isPreferred`.

- **Validate security:** check `accessPoint.securityType` matches the AP; the matching `securityDetails` block shows the (masked) credential/protocol configuration.
- **Validate addressing:** check `ipv4Configuration.ipAddress`, `gateway`, and `enableDhcp` to confirm the reader has a working route to the broker / back-end.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `wifiProfiles.wifiConfig[]` (response) — the array of saved profiles; iterate it to find the connected one.
- `...accessPoint.status` / `essid` (response) — the association state and network name of each profile.
- `...accessPoint.securityType` (response) — selects which `securityDetails` key is populated.
- `...ipv4Configuration.ipAddress` (response) — the DHCP-assigned address the reader uses to reach the broker.

- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_wifi" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command * and	string	—	get_wifi	Fixed command name. Retrieves the Wi-Fi profiles saved on the device.
2	requestId * d	string	—	abc123	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and a `wifiProfiles` object whose `wifiConfig` is the **array of profiles**. On the **Schema** tab, expand `wifiProfiles` → `wifiConfig` → `interfaceDetails` (and its `accessPoint`, `ipv4Configuration`, `ipv6Configuration`, `securityDetails` sub-objects) to see every field. Within each profile's `interfaceDetails`, only `interfaceName`, `hostname`, and `macAddress` are required; the interface-level `status` is optional. `securityDetails` carries only the key matching `accessPoint.securityType`, with secrets masked. The **Example** tab shows a WPA2-Personal profile; enterprise variants are in [Examples](#).

Connection scenario

WPA2 Personal connection ▼

Response payload

Example: WPA2-Personal

```
1  {
2    "command" : "get_wifi" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "wifiProfiles" : {
6      "wifiConfig" : [
7        {
8          "interfaceDetails" : {
9            "interfaceName" : "wlan0" ,
10           "status" : "" ,
11           "hostname" : "RFD40-211545201D0034" ,
12           "macAddress" : "AC:3F:A4:00:2B:15" ,
13           "accessPoint" : {
14             "essid" : "IOT_TESTAP" ,
15             "status" : "CONNECTED" ,
16             "securityType" : "WPA2Personal" ,
17             "isPreferred" : false ,
18             "autoConn" : false
19           },
20           "ipv4Configuration" : {
21             "ipAddress" : "192.168.0.110" ,
22             "subnetMask" : "255.255.255.0" ,
23             "gateway" : "192.168.0.1" ,
24             "enableDhcp" : true ,
25             "dnsServer" : "192.168.0.1" ,
26             "domainName" : "zebra.com"
27           },
28           "securityDetails" : {
29             "WPA2Personal" : {
30               "password" : "XXXXXXXX"
31             }
32           }
33         }
34       ]
35     },
36     "response" : {
37       "code" : 0 ,
38       "description" : "Success"
39     }
40   }
41 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMP
1	command *	string	—	get_v
2	requestId *	string	—	abc12
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1
4	wifiProfiles	object	—	—
5	wifiConfig	array	—	—
6	interfaceDe tails	object	—	—
7	interfa ceName *	string	—	wlan0
8	status	enum	Enabled Disabled	Enabl
9	hostnam e *	string	—	RFD40
10	macAddr ess *	string	—	E0-D0

#	FIELD	TYPE	ENUM / UNITS	EXAMP
11	accessPoint	object	—	—
12	essid *	string	—	Zwire
13	status *	enum	CONNECTED DISCONNECTED	CONN
14	securityType *	enum	WPA2Personal WPA3Personal WPA2Enterprise WPA3Enterprise	WPA2I
15	isPreferred *	boolean	—	false
16	autoConn	boolean	—	false
17	ipv4Configuration	object	—	—
18	ipAddress	string	ipv4	192.1
19	subnetMask	string	ipv4	255.2
20	gateway	string	ipv4	192.1
21	enableDhcp	boolean	—	true
22	dnsServer	string	ipv4	192.1
23	domainName	string	—	test.
24	ipv6Configuration	object	—	—
25	ipAddress	string	ipv6	fe80:

#	FIELD	TYPE	ENUM / UNITS	EXAMP
26	prefix	integer	—	64
27	gateway	string	ipv6	2a02
28	enableAuto	boolean	—	true
29	dnsServer	string	ipv6	2001:
30	domainName	string	—	test.
31	securityDetails	object	—	—
32	WPA2Personal	object	—	—
33	password	string	password	test0
34	WPA3Personal	object	—	—
35	password	string	password	test0
36	WPA2Enterprise	object	—	—
37	authentication	enum	tls ttls peap	tls

#	FIELD	TYPE	ENUM / UNITS	EXAMP
38	innerAuthentication	enum	none mschapv2	mscha
39	identity	string	—	test
40	anonymousIdentity	string	—	test
41	password	string	password	test
42	phrase	string	—	test
43	certificate	array	—	—
44	key	enum	ca_cert client_key client_cert cert_key_password	ca_ce
45	value	string	—	wifi_

#	FIELD	TYPE	ENUM / UNITS	EXAMP
46	pro toc ol	enum	WPA2_Enterprise_CCMP	WPA2_
47	WPA3En terpri se	object	—	—
48	aut hen tic ati on	enum	tls ttls peap	tls
49	inn erA uth ent ica tio n	enum	none mschapv2	mscha
50	ide nti ty	string	—	test
51	ano nym ous Ide nti ty	string	—	test
52	pas swo rd	string	password	test
53	pas sph ras e	string	—	test

#	FIELD	TYPE	ENUM / UNITS	EXAMP
54	cert t i f i c a t e	array	—	—
55	k e y	enum	ca_cert client_key client_cert cert_key_password	ca_ce
56	v a l u e	string	—	wpa3_
57	pro to c o l	enum	WPA3_Enterprise_CCMP WPA3_Enterprise_CCMP_256 WPA3_Enterprise_GCMP_128 WPA3_Enterprise_GCMP_256_SHA256 WPA3_Enterprise_GCMP_256_SUITEB_192	WPA3_
58	response *	object	—	—
59	code *	integer	0–101	0
60	description *	string	—	Succ

Response Semantics

- **Credentials are masked.** The device returns stored passwords, passphrases, and identities as a mask (never the real secret). Treat every credential field in the response as opaque — you cannot read back the actual value with `get_wifi`.
- **Security type selects the credential block.** `securityDetails` contains only the one key that matches `accessPoint.securityType` — `WPA2Personal` / `WPA3Personal` (a single `password`) or `WPA2Enterprise` / `WPA3Enterprise` (EAP `authentication`, optional `innerAuthentication` / `identity` / `anonymousIdentity`, a `certificate` array, and a `protocol`).
- `ipv6Configuration` is schema-defined but not returned by the current firmware.

- **Wi-Fi-disabled response shape.** When Wi-Fi is disabled, the contract's own example returns the profile under a **top-level** `wifiConfig` object (rather than `wifiProfiles.wifiConfig[]`), carrying only `interfaceName`, `status` (`Disabled`), `hostname` and `macAddress`. The response schema does not declare a top-level `wifiConfig`, so parse both shapes; the connected shape documented here is `wifiProfiles.wifiConfig[]`.

Response Codes

`get_wifi` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — Wi-Fi profiles retrieved and returned.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the Wi-Fi configuration at this time. Retry after a short delay; if it persists, reboot the device.
101	Default error — generic catch-all.

For the full code list see the [Response Codes](#). (Code `20`, `Wifi is not supported`, applies to the Wi-Fi **setters** `set_wifi` / `set_config`, not to `get_wifi`.)

Examples

`get_wifi` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abc123`) is echoed back so you can pair reply to request. The **Connection scenario** dropdown under [Response Payload](#) shows each contract example:

- **WPA2 Personal connection** — a single masked `password`.
- **WPA2 Enterprise connection** — an EAP `authentication` method, a `certificate` array of `{key, value}` references and a `protocol`, with credentials masked.
- **WPA3 Enterprise connection** — additionally `innerAuthentication`, `identity` and `anonymousIdentity`, and a WPA3 `protocol` suite.
- **WiFi interface disabled** — the minimal reply for a disabled interface.

Notes and Caveats

`get_wifi` is a read-only getter — safe to poll during commissioning or troubleshooting. Configure profiles with `set_wifi` / `delete_wifi_profile`; for the concept-level view, see [Network Configuration](#). Enterprise profiles depend on installed certificates — see [TLS and Authentication](#).

Related:  [Network Configuration](#) ·  `get_eth` ·  Network and Connectivity ·  [Response Codes](#)

get_wifi

2 of 4

install_certificate

REFERENCE

API consumer

install a certificate or key into the reader's trust store

Overview

`install_certificate` **installs a certificate (or key) into the reader's trust store** — for MQTT-TLS brokers, Wi-Fi Enterprise (EAP), the file store, or client/server auth. Content is supplied one of two ways, chosen by `certDetails.certSource`:

- `HTTP` (default) — the device **downloads** each component from the URLs in `url[]`.
- `DIRECT` — you provide the content **inline** in `certificateBundle`.

It is a state-changing command on the device-management ([MGMT](#)) endpoint. Installed certificates are then referenced by name from `config_endpoint` (`MQTT_TLS`) and `set_wifi` (Enterprise).

`install_certificate` **is asynchronous: the reply `code 1` is an accept, not an install confirmation.** Verify with `get_installed_certificates` (and, for `HTTP`, the `FILE_DOWNLOAD` alerts). Do not gate success on `code == 0`. See [Lifecycle](#).

Details

Property	Value
Interaction Pattern	Asynchronous Request / Accept (device acks with code 1 ; completion verified out-of-band)
State Change	STATE-CHANGING (writes to the trust store; asynchronous)
Reversibility	REVERSIBLE (remove with <code>delete_certificate</code>)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>certDetails</code> (with <code>type</code> and <code>authenticationType</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.21</code> (API Version Reporting)
Related	<code>get_installed_certificates</code> , <code>delete_certificate</code> , <code>config_endpoint</code> , <code>set_wifi</code>

When to Use

Use `install_certificate` to provision the certificates other commands depend on:

- **Before MQTT-TLS** — install the `ca_cert` / `client_cert` / `client_key` (type `mqtt`) that `config_endpoint` references in `securityParams`.
- **Before Wi-Fi Enterprise** — install `wifi`-type certificates that `set_wifi` references in its `certificate` array.
- **File-store trust** — install a `filestore` CA used to authenticate HTTPS downloads.
- **Choose the source:** use `DIRECT` when you can embed the content (no network dependency); use `HTTP` when the device can reach your file server (watch the `FILE_DOWNLOAD` alerts).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The <code>code 1</code> accept. Correlated by the in-payload <code>requestId</code> .
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	For <code>HTTP</code> installs, the device emits asynchronous <code>FILE_DOWNLOAD</code> alerts (per-file download/save) here. See <code>alerts</code> .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `certDetails.certSource` (request) — `HTTP` (use `url`) or `DIRECT` (use `certificateBundle`).
- `certDetails.type` (request) — the certificate category (`mqtt`, `wifi`, `filestore`, ...); scopes where the certificate can be used.

- `certDetails.name` (request) — the logical name you choose for the entry. It is optional at install time, but `name` together with `type` is what identifies an installed certificate afterward: `delete_certificate` takes exactly that pair, and a delete that omits `name` removes every installed certificate of the matching `type`.
- `certDetails.authenticationType` (request) — `NONE` or `CERTIFICATE` (certificate-authenticated download).
- `certificateBundle.client_key` / `cert_key_password` (request) — ⚠ secrets you send inline (DIRECT); mask them. In `HTTP` mode a `url` entry with `key: cert_key_password` likewise carries a plaintext password in its `value`. See the credential warning below.
- `response.code` (response) — `1` means *accepted*, not *installed*; confirm with `get_installed_certificates`. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `certDetails` payload. Pick a **certificate scenario** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

⚠ YOU TRANSMIT PRIVATE KEYS

In `DIRECT` mode `certificateBundle.client_key` (private key) and `cert_key_password` (password) are secrets in the request — mask them as `*****`. In `HTTP` mode, a `url` entry with `key: cert_key_password` also carries a plaintext password in its `value` — treat it the same way. Send `install_certificate` only over secured channels. The device never echoes them, and the read-back never exposes private-key material.

Certificate scenario

MQTT certificates with certificate authentication



Request payload

Example: mqtt · HTTP · CERTIFICATE auth

```
1      {
2        "command" : "install_certificate" ,
3        "requestId" : "cert-test-001" ,
4        "certDetails" : {
5          "type" : "mqtt" ,
6          "name" : "testMqttCert" ,
7          "authenticationType" : "CERTIFICATE" ,
8          "certSource" : "HTTP" ,
9          "verificationType" : "VERIFY_HOST_AND_PEER" ,
10         "-----BEGIN CERTIFICATE-----
MIIDPTCCAiWgAwIBAgIUHtj1WDceNR9Nf+vZBB1Ne7s
BQAwLjEPMA0GA1UEAwwGVGVzdENBMQ4wDAYDVQQKDAV
VVMwHhcNMjYwNTEwMTMxNDQ3WhcNMzYwNTA4MTMxNDQ
ZXN0Q0ExDjAMBgNVBAoMBVplYnJhMQswCQYDVQQGEwJ
AQEBBQADggEPADCCAQoCggEBANR8E/mxOQ5uTPW/muW
dMLIQ+TVgb9mcOz80VnpgkQqomhbZaIG4CtnBufyIJv
lGoPC4M469Rm+0oEN9GkUHgtgF2N4Qt9bYomySux3yy
XE/a7XWdXeJkpIB0Mzzf7MoNq14AfX431RKgvpMgAqZ
aKIAJV2avJd87dF8x+K80kLBErDnKtuNFpXN7KoC7BB
eL9MY5Ik7POMR0AM/TxRBLAXyoZHSPdEpEoaQZjSzJ7
VR00BBYEFJHms5XPEyFYnVcf2++X9N8UlkahMB8GA1U
nVcf2++X9N8UlkahMA8GA1UdEwEB/wQFMAMBAf8wDQY
AL4HEEMhuQ9sQLFE9wiGw4BIILXnX30FbphWM8/xpHK
+QUhaU9hruBEajT7mYjKvZ9NtUumcAmAh3s29KD4Npj
zCmqlAqGVzK3ve/Fuurgdbk3kApOZTTyqFw629dmmRh
Y6ZtwwAJygC4/WCfWg/7XaeQ+6kqKtY6ijSJFFCNwEP
rvm5UUPRMGCs20JOucntSF8uPOICNWUER3DbqAJ4jXK
Vqh5mX6k2PZe0eylcukyu/s=
-----END CERTIFICATE-----"
        "caCertificateFileContent" :
        "url" : [
          {
            "key" : "ca_cert" ,
            "value" : "https://192.168.0.104:8443/mqtt_ca_cert.pem"
          },
          {
            "key" : "client_cert" ,
            "value" : "https://192.168.0.104:8443/mqtt_client_cert.pem"
          },
          {
            "key" : "client_key" ,
            "value" : "https://192.168.0.104:8443/mqtt_client_key.pem"
          }
        ]
      }
    }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	comm and *	string	—	install_certificate
2	requ estI *	string	—	abc123
3	cert Deta *	object	—	—
4	nam e	string	—	sotiAgent
5	ty pe *	enum	client server mqtt wifi filestore	client
6	au th en ti ca *	enum	NONE CERTIFICATE	CERTIFICATE
7	cer tSo urc e	enum	HTTP DIRECT	HTTP

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
8	certFormat	enum	PEM BASE64	BASE64
9	url	array	—	—
10	key	enum	ca_cert client_key client_cert cert_key_password	ca_cert
11	url	string	—	https://192.168.0.104:8443/mqtt_pem
12	verificationType	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_NONE

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
13	certificateBundle	object	—	—
14	certificate	string	—	-----BEGIN CERTIFICATE-----...
15	clientIdentity	string	—	LS0tLS1CRUdJTiBSU0Eg...
16	clientCertificate	string	—	LS0tLS1CRUdJTiBDRVJU...

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
17	cert-key-password	string	—	—
18	caCertificateContent	string	—	-----BEGIN CERTIFICATE-----...

Request Semantics

Modes. `certDetails.certSource` selects how the certificate content is supplied — `HTTP` (download) or `DIRECT` (inline):

- **HTTP (default).** Supply `url[]` with a `{key, value}` entry per component (`ca_cert`, `client_cert`, `client_key`, `cert_key_password`). The device downloads each; for `authenticationType: CERTIFICATE` over HTTPS, supply `caCertificateFileContent` (or rely on an installed filestore CA). Requires **device** → **file-server reachability**; missing `url` returns [code 29](#). The device emits `FILE_DOWNLOAD` alerts as it fetches.
- **DIRECT.** Supply `certificateBundle` with inline PEM content for one `type`. No network dependency; missing `certificateBundle` returns [code 30](#).

Invalid enum values (e.g. `type`, `certSource`) return [code 23](#). The contract does not forbid unrecognized fields, so a misspelled field still passes schema validation. How the reader responds to such a field is not specified in the available sources, so check field names client-side before you send.

- **Choosing `type`**. The enum accepts `client`, `server`, `mqtt`, `wifi`, and `filestore`. Current deployments need `wifi` (Enterprise EAP, referenced by [set wifi](#)), `filestore` (the CA that authenticates HTTPS downloads), and `mqtt` (broker TLS, referenced by [config endpoint](#)). `client` and `server` are the generic client-side and server-side categories; the schema gives them no further gloss beyond "the certificate category that identifies where the certificate is used", and its own field examples pair `type: client` with an agent certificate (`name: sotiAgent`).
- **One install, one `type` — and file-store trust must be typed `filestore`**. `certDetails.type` is a single string with a five-member enum, not a list, and `certificateBundle` supplies certificate content "for one certificate `type`", so each `install_certificate` call installs exactly one category. A file-store CA is therefore typed `filestore` and cannot also be typed `client` or `server` — send a second `install_certificate` if you need a second category. This matters for HTTPS downloads: with `authenticationType: CERTIFICATE` and an empty `caCertificateFileContent`, the schema names an already-installed `filestore` certificate as the trust anchor the device falls back to, so installing the same CA under `client` or `server` leaves no `filestore` entry for that fallback to use.
- **`type` and `authenticationType` are required but not firmware-enforced**. The schema requires both within `certDetails`, but the device accepts a payload omitting them (still returns [code 1](#)) and installs a malformed entry (e.g. `type: null`). Always send both; do not rely on the device to reject their absence.
- **Certificate size limit — 4 KB**. Certificate content is capped at **4 KB (4096 bytes) per certificate file**, not per request; keep each certificate within the limit.
- **PEM, or base64-encoded PEM**. The device supports certificates in **PEM** format, or as PEM text base64-encoded again (`certFormat: BASE64`) — PEM being the base64 encoding delimited by header/footer markers, e.g. `-----BEGIN CERTIFICATE-----` / `-----END CERTIFICATE-----`, `-----BEGIN RSA PRIVATE KEY-----` / `-----END RSA PRIVATE KEY-----`, and `-----BEGIN ENCRYPTED PRIVATE KEY-----` / `-----END ENCRYPTED PRIVATE KEY-----`.
- **PKCS1 (RSA) key encoding**. The reader requires private keys in **PKCS1** format — the public-key-cryptography standard for RSA.

Response Payload

The immediate reply is an **accept** — `command`, `requestId`, `apiVersion`, and `response` with [code 1](#). It confirms receipt, not installation.

Result

Certificate installed



Response payload

Example: accepted (code 1)

```
1  {
2    "command" : "install_certificate",
3    "requestId" : "18996",
4    "apiVersion" : "V1.1",
5    "response" : {
6      "code" : 1,
7      "description" : "Command payload is accepted"
8    }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	install_certificate	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Accept/status envelope. This is an asynchronous accept, not a terminal install result (see below).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	c o d e *	integer	0–101	0	1 = accepted (the success/accept code for this command — not 0). Non-1 = rejected. See Response Codes below.
6	d e s c r i p t i o n *	string	—	Success	Human-readable status.

Lifecycle

Acknowledgment

Because the accept (`code 1`) is not a terminal result, confirm the install actually landed. There is **no** terminal success/failure message on the response topic after the `code 1` accept. A consumer that treats `code == 0` as the success condition will misread every `install_certificate` reply.

Progress

For `HTTP`, watch `FILE_DOWNLOAD` alerts on the event topic — per-file download/save progress. These confirm download + save only, not trust-store persistence; still verify with the read-back.

Confirmation

Read back with `get installed certificates` — the installed components appear in `installedCerts.certInfo[]` (named `{name}_{component}`). This is the authoritative confirmation. (Note: private keys are never listed, and serials read back truncated to 8 bytes.)

Response Codes

`install_certificate` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the success/accept code for this command . The install proceeds asynchronously; verify with <code>get_installed_certificates</code> .
0	Success — terminal success; in practice <code>install_certificate</code> acks with 1, and 0 is what the <code>get_installed_certificates</code> read-back returns.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — a referenced file/URL could not be found.
21	Certificate not found (<code>IOT_ERROR_CERT_NOT_FOUND</code>) — the named certificate is absent.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>type</code> , <code>certSource</code> , <code>authenticationType</code> , <code>verificationType</code>) is outside its allowed set.
29	Url missing for HTTP source (<code>IOT_ERROR_URL_MISSING</code>) — <code>certSource: HTTP</code> (or defaulted) with no/empty <code>url</code> .
30	Certificate content missing for direct source (<code>IOT_ERROR_CERT_CONTENT_MISSING</code>) — <code>certSource: DIRECT</code> with no <code>certificateBundle</code> .
34	Certificate base64 decode failed (<code>IOT_ERROR_CERT_BASE64_DECODE_FAILED</code>) — the base64-encoded certificate content could not be decoded. Re-encode it and resend.
36	Missing client key or client certificate (<code>IOT_MISSING_CLIENT_KEY_OR_CERT</code>) — <code>client_cert</code> and <code>client_key</code> were not sent together. Always install a client certificate with its key.
101	Default error — generic catch-all.

Examples

Publish `install_certificate`; the device acks with the `code 1` envelope under [Response Payload](#). Every contract request example is in the **Certificate scenario** dropdown under [Request Payload](#), and the **Result** dropdown under [Response Payload](#) shows the accepted reply and the two source-specific errors (29 for a missing `url`, 30 for missing `certificateBundle` content). The [examples](#) below show what the dropdowns do not.

A **DIRECT** **install** embeds the content inline — the private key and any key password are masked:

Request — DIRECT (inline, secrets masked)

```
1  {
2    "command" : "install_certificate",
3    "requestId" : "cert-test-003",
4    "certDetails" : {
5      "type" : "mqtt",
6      "name" : "testMqttCert",
7      "authenticationType" : "NONE",
8      "certSource" : "DIRECT",
9      "verificationType" : "VERIFY_HOST_AND_PEER",
10     "certificateBundle" : {
11       "-----BEGIN CERTIFICATE-----"
12       "ca_cert" : ... ,
13       "-----END CERTIFICATE-----"
14       "-----BEGIN CERTIFICATE-----"
15       "client_cert" : ... ,
16       "-----END CERTIFICATE-----"
17       "-----BEGIN RSA PRIVATE KEY-----"
18       "client_key" : ***** ,
19       "-----END RSA PRIVATE KEY-----"
20       "cert_key_password" : "XXXXXXXXXX"
21     }
22   }
23 }
```

A minimal **filestore** **HTTP install** downloads a single CA:

Request — filestore (HTTP, single CA)

```
1  {
2    "command" : "install_certificate",
3    "requestId" : "19001",
4    "certDetails" : {
5      "name" : "root_certs",
6      "type" : "filestore",
7      "authenticationType" : "NONE",
8      "certSource" : "HTTP",
9      "verificationType" : "VERIFY_HOST_AND_PEER",
10     "url" : [
11       {
12         "key" : "ca_cert",
13         "value" : "https://192.0.2.107:8080/filestore_ca_cert.pem"
14       }
15     ]
16   }
17 }
```

Notes and Caveats

Installing a certificate leaves the active [MDM/MGMT](#) control session intact. After installing, reference the certificate by its logical `name` from `config_endpoint` or `set_wifi`. For the concept-level view, see [TLS and Authentication](#) and [Certificate Management](#).

Related: [TLS and Authentication](#) · [get installed certificates](#) · [delete certificate](#) · [config_endpoint](#) · [set_wifi](#) · [Response Codes](#)

install_certificate

2 of 3

reboot

REFERENCE



API consumer



perform a warm reset of the reader

Overview

`reboot` performs a **warm reset** of the reader. It requires only `command` and `requestId`. The device **accepts** the request (`response.code 1`), reboots, and then **auto-reconnects to the previously-connected server** — no manual reconnect needed. It is a state-changing command on the device-management ([MGMT](#)) endpoint.

`reboot` is **session-severing** — the reply is `code 1` ("Command payload is accepted"), returned before the reset, and the control session drops for ~30 s–2 min. See the warning in [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Asynchronous Request → Accept → Warm reset → Auto-reconnect
State Change	STATE-CHANGING (warm reset; clears volatile state)
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set os</code> , <code>get version</code> , <code>get status</code> , <code>control operation</code>

When to Use

Use `reboot` to **warm-reset the reader**:

- **Recover from a bad state** — clear a transient fault when a targeted command will not.
- **Apply/settle changes** that take effect on restart.

- Clear the [operating mode](#) — a reboot resets the operating mode to default (see the volatility note below).

Do **not** reboot during an active RFID inventory — the device refuses with [code 5](#). Stop the inventory first with [control operation](#).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The code 1 accept, returned before the reset. Correlated by the in-payload requestId .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`reboot` carries **no credential field** — there is nothing to mask. The `requestId` you send is echoed in the accept so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `command` / `requestId` (request) — the entire request; there is no reboot payload.
- `response.code` (response) — `1` means *accepted* (the reset then proceeds); `5` means the device refused the reboot because an inventory is running. See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` and `requestId`.

Request payload

```
1 {
2   "command" : "reboot" ,
3   "requestId" : "123abcd"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	reboot	Fixed command name.
2	requestId *	string	—	123abcd	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

The reply is an **accept** — `command`, `requestId`, `apiVersion`, and `response` with code 1, returned *before* the reset begins.

Response payload

Example: accept

```
1 {
2   "command" : "reboot" ,
3   "requestId" : "123abcd" ,
4   "apiVersion" : "V1.1" ,
5   "response" : {
6     "code" : 1 ,
7     "description" : "Command payload is accepted"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	reboot	Echoes the executed command.
2	requestId *	string	—	123abcd	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Accept/status envelope. This is an accept returned before the reset — not a "reboot complete" result (see below).
5	code *	integer	0–101	0	1 = the payload was accepted (the reboot success-ack — not 0). See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Lifecycle

A single `reboot` drives the device through a clean warm-reset cycle.

Acknowledgment

The device replies `response.code 1` ("Command payload is accepted"), *before* it resets. This is the success-ack — there is no later "reboot complete" message on success. If the reboot fails, the device sends a failure notification.

Session Impact

- **Down** — the reader drops its MQTT connections and warm-resets. It is unreachable for the duration.
- **Auto-reconnect** — the device auto-reconnects to the **previously-connected server on its own**, with no manual intervention. Recovery time **varies** (observed ~44 s and ~110 s across two runs — roughly ~30 s to ~2 min); do not hard-code a fixed wait.

Persistence

State after reset:

- **Endpoint configuration persists** — saved and active endpoints are restored/re-activated across the reset.
- **Operating Mode is volatile** — the reboot resets the operating mode to default (e.g. `BALANCED_PERFORMANCE`, any armed `accessOperations` cleared). See `set operating mode`; this is the documented "on reboot the set configuration is lost" behavior.

Response Codes

`reboot` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the reboot success-ack , returned before the reset. The device then reboots and auto-reconnects.
0	Success — the generic success code; in practice <code>reboot</code> acks with 1, not 0.

Code	Description
5	Can't reboot device, inventory in progress (<code>IOT_ERROR_IN_REBOOT_INVENTORY_IN_PROGRESS</code>) — unique to <code>reboot</code> ; the device refuses the reset while an RFID inventory is running. Stop the inventory with <code>control_operation</code> , then retry.
101	Default error — generic catch-all.

[Code 1](#) is the acceptance ack, not a `reboot` -only code. The contract scopes code 1 ("Command payload is accepted") to `reboot` , `set_os` and `install_certificate` alike, and scopes [code 5](#) to `reboot` alone. Accept 0 and 1 as success-equivalents here.

Examples

Publish `reboot` ; the device replies with the [code 1](#) accept under [Response Payload](#), then runs the [reboot cycle](#). If an inventory is active, the device refuses the reset with [code 5](#) :

Response — refused during inventory (code 5)

```

1  {
2      "command" : "reboot" ,
3      "requestId" : "123abcd" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 5 ,
7          "description" : "Can't reboot device, inventory in progress"
8      }
9  }
```

Notes and Caveats

⚠ SESSION-SEVERING — SEND ONCE, NEVER AUTO-RETRY

The warm reset drops the control session; the device auto-reconnects to the previously-connected server, but recovery time **varies** (~30 s–2 min observed). Because a reboot cannot be un-done and interrupts service, issue it deliberately (ideally after operator confirmation) and **never auto-retry** it — the reboot may already have taken effect even if the accept was lost.

- **Endpoint config persists; operating mode does not.** Across the reset, saved and active MQTT endpoints are restored and re-activated, but the **operating mode reverts to default** (any tuned profile or armed `accessOperations` is cleared). Re-apply the operating mode after a reboot if you were relying on a non-default configuration — see `set_operating_mode`.

Sequence a reboot as: (optionally) stop any inventory with `control_operation` → send `reboot` once → wait out the reconnect (poll, do not hard-code the delay) → confirm with `get_version` / `get_status`. For the concept-level view, see [Device Lifecycle](#) and [Broker Connection](#).

Related:  [Device Lifecycle](#) ·  `set_os` ·  `get_version` ·  `control_operation` ·  [Response Codes](#)

set_os

REFERENCE

API consumer

initiate an over-the-air firmware update on the reader

Overview

`set_os` initiates an over-the-air (OTA) firmware update — it points the reader at a firmware `.DAT` image at a URL and triggers the download-and-flash cycle. It is a state-changing command on the device-management ([MGMT](#)) endpoint, and it is both **asynchronous** and **session-severing**: the device accepts the payload, downloads and flashes the firmware, and **reboots** — dropping off the network for the duration. The authoritative confirmation of the outcome is an out-of-band `get_version` read-back.

`set_os` **reflashes and reboots the reader**. The reply `code 1` is an accept (returned for success and failure alike), and the reader rejects every other command with `code 4` while the update runs. See the danger callout in [Notes and Caveats](#); confirm the result with `get_version`.

Details

Property	Value
Interaction Pattern	Asynchronous Request → Accept → Reboot → Verify (completion confirmed out-of-band)
State Change	STATE-CHANGING (downloads and flashes firmware; reboots the reader)
Reversibility	IRREVERSIBLE (no single-step undo; re-flash a prior build with another <code>set_os</code> to roll back)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>OSUpdateDetails</code> (with <code>url</code> and <code>authenticationType</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_version</code> , <code>install_certificate</code> , <code>reboot</code> , <code>get_status</code>

When to Use

Use `set_os` to **update the reader's firmware** — during commissioning, a fleet migration, or a security/feature rollout.

- **Host the image** — place the firmware `.DAT` at a URL the reader can reach.
- **Choose authentication** — `NONE` for an open/internal server, or `CERTIFICATE` (with a CA reference or inline PEM) plus a `verificationType` for a secured HTTPS download.
- **Check preconditions** — ensure adequate battery and flash first (low battery → [code 14](#), insufficient flash → [code 8](#)).
- **Verify afterward** — always confirm the new build with `get version`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The code 1 accept, and the code 4 lock returned to <i>other</i> commands during the install.
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	The <code>FIRMWARE_UPDATE</code> notifications (start, progress, outcome). See alerts .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `OSUpdateDetails.url` (request) — the firmware `.DAT` download URL (a bad/unreachable URL → [code 9](#)).
- `OSUpdateDetails.authenticationType` (request) — `NONE` or `CERTIFICATE`; for `CERTIFICATE`, supply `caCertificateFile` (installed reference) or `caCertificateFileContent` (inline PEM).

- `OSUpdateDetails.verificationType` (request) — TLS verification of the download server; we recommend setting it for HTTPS downloads (see [Request Semantics](#)).
- `response.code` (response) — 1 means *accepted*, not *updated*; confirm with `get version`. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `OSUpdateDetails` payload (`url` + `authenticationType`). Pick an **update source** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Update source

No authentication HTTP

Request payload

Example: authenticationType NONE · HTTP

```
1 {
2   "command" : "set_os" ,
3   "requestId" : "abc123" ,
4   "OSUpdateDetails" : {
5     "url" : "http://192.168.29.39:8000/Build-3.10.27/Firmware/PAAFKS00-012-R
6     "authenticationType" : "NONE" ,
7     "verificationType" : "VERIFY_NONE"
8   }
9 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	command *	string	—	set_os
2	requestId *	string	—	abc123
3	OSUpdateDetails *	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
4	url*	string	uri	https://192.168.29.39:8000/Build-3.10.27/Firmware
5	authentication*	enum	NONE CERTIFICATE	CERTIFICATE
6	verificationType	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_NONE

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
7	certificate	string	—	<pre> -----BEGIN CERTIFICATE----- \nMIIDPTCCAiWgAwIBAgIUHtj1WDceNR9 1Ne7sFmHkwDQYJKoZIhvcNAQEL\nBQAwI A1UEAwwGVGVzdENBMQ4wDAYDVQQKDAVaZ MAkGA1UEBhMC\nVVMwHhcNMjYwNTEwMTM cNMzYwNTA4MTMxNDQ3WjAuMQ8wDQYDVQQ ZXN0Q0ExDjAMBGNVBAoMBVplYnJhMQswC EwJVUzCCASIwDQYJKoZIhvcN\nAQEBBQA CCAQoCggEBANR8E/mxoQ5uTPW/muWnGoW lIiJiCM/n92\ndMLIQ+TVgb9mcOz80Vnpg ZaIG4CtnBufyIJvNKid+mgrrrFh18HtMnE oPC4M469Rm+0oEN9GkUHgtgF2N4Qt9bYo yPlFuUOC+uaYRCCn3Hm8ko\nXE/a7XWdX Mzzf7MoNq14AfX431RKgvpMgAqZTnDWtm q0t64tP7\naKIAJV2avJd87dF8x+K80kL uNFpXN7KoC7BBwdTzbG8VWYNUZeG8eCqB Y5Ik7POMR0AM/TxRBLAXyoZHSPdEpEoaQ Hv0CAwEAAaNTMFEwHQYD\nVR0OBBYEFJH FYnVcf2++X9N8UlkahMB8GA1UdIwQYMBa XPeyFY\nnnVcf2++X9N8UlkahMA8GA1UdE MAMBAf8wDQYJKoZIhvcNAQELBQADggEB\ MhuQ9sQLFE9wiGw4BIILxnX30FbphWM8/ sH/NOWpjulG+ZAxe8n\n+QUhaU9hruBEa vZ9NtUumcAmAh3s29KD4NpjwglqUYfKQp MXuW\nnzCmqlAqGVzK3ve/Fuurgdbk3kAp w629dmmRhoP+2sdCYMAuky348/4G1\nnY ygC4/WCfWg/7XaeQ+6kqKtY6ijSJFFCNw 3XAqP+kZ0e2taL5a\nnrvm5UUPRMGCS20J 8uPOICNWUER3DbqAJ4jXKhSHdsG6fNDyJ 0j\nVqh5mX6k2PZe0eylcukyu/s=\n--- CERTIFICATE----- </pre>
8	certificate	string	—	/apps/ca_cert.pem

Request Semantics

Modes. `authenticationType` selects how the reader authenticates to the firmware source: `NONE` for an unauthenticated download (an open or internal server), or `CERTIFICATE` to authenticate the HTTPS download against a CA — supplied either as a **pre-installed** reference (`caCertificateFile` , installed via `install_certificate`) or **inline** (`caCertificateFileContent`), and paired with a `verificationType` for TLS server verification. Both `CERTIFICATE` forms are in the [Request Payload](#) selector: **Pre-installed CA certificate HTTPS** and **Inline CA certificate HTTPS**.

- **Authentication & transport security.** For `authenticationType: CERTIFICATE` , supply the CA either as an installed reference (`caCertificateFile` , pre-installed via `install_certificate`) or inline (`caCertificateFileContent`); if `caCertificateFileContent` is empty, the reader uses an already installed filestore certificate. `authenticationType: NONE` means no authentication is needed to access the URL. `verificationType` sets how far the reader validates the download server's TLS certificate: `VERIFY_HOST_AND_PEER` checks the certificate chain and that the certificate matches the host; `VERIFY_NONE` does no verification and must not be used in production.
- `OSUpdateDetails` **is required**. The contract requires it, so a request without it is invalid rather than merely unable to update. A bad or unreachable `url` returns [code 9](#) .

Response Payload

The immediate reply is an **accept** — `command` , `requestId` , `apiVersion` , and `response` with [code 1](#) . It confirms receipt, not a completed update.

Response payload

Example: *accept*

```
1  {
2      "command" : "set_os" ,
3      "requestId" : "abc1234" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 1 ,
7          "description" : "Command payload is accepted"
8      }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	set_os	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	v1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response	object	—	—	Accept/status envelope. Not in the contract's required set for this reply, although every published example carries it. This is an accept, not a terminal result (see the lifecycle below).
5	code *	integer	0–101	0	1 = the payload was accepted (the accept code for this command — not 0). See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Lifecycle

`set_os` is an accept-then-verify cycle, not a single request/response.

Acknowledgment

The device replies `response.code 1` ("Command payload is accepted") on the response topic. This confirms receipt only, and is **identical for success and failure**.

Progress

A `FIRMWARE_UPDATE` notification appears on the event topic as the download/flash begins. The block below is the example the schema declares for it, not an observed one:

Event — FIRMWARE_UPDATE start (schema example)

```
1  {
2    "type": "NOTIFICATION",
3    "timestamp": "2026-06-18T11:30:50.387Z",
4    "state": "ONESHOT",
5    "id": "FIRMWARE_UPDATE",
6    "priority": "HIGH",
7    "alertDetails": {
8      "fwUpdateStatus": {
9        "updateStatus": "Starting FW update"
10     }
11  }
12 }
```

Notifications continue on the same topic while the install runs. Each of the five recorded during one update observed on an RFD40 carried `alertDetails.fwUpdateStatus.overallProgress` — percent of the update completed — beside a human-readable `updateStatus`. `overallProgress` is optional and the start example above carries none, so treat its absence as unknown rather than as zero. Three of those five notifications are shown here and under [Confirmation](#) below, across the 15.265 s in which `overallProgress` ran from 87% to 100%; the earlier part of that update was not recorded.

Event — FIRMWARE_UPDATE in progress, 87% (live)

```
1  {
2    "type": "NOTIFICATION",
3    "timestamp": "2026-07-07T11:49:50.628Z",
4    "state": "ONESHOT",
5    "id": "FIRMWARE_UPDATE",
6    "priority": "HIGH",
7    "alertDetails": {
8      "fwUpdateStatus": {
9        "updateStatus": "In progress",
10       "overallProgress": 87
11     }
12  }
13 }
```

The identical `In progress` value arrived twice — at 87%, then at 95% 10.014 s later — so only the first is shown. The notification after it carried a status string naming a component file inside the firmware package:

Event — FIRMWARE_UPDATE in progress, 98% (live)

```
1  {
2    "type" : "NOTIFICATION" ,
3    "timestamp" : "2026-07-07T11:50:04.424Z" ,
4    "state" : "ONESHOT" ,
5    "id" : "FIRMWARE_UPDATE" ,
6    "priority" : "HIGH" ,
7    "alertDetails" : {
8      "fwUpdateStatus" : {
9        "updateStatus" : "Updating file PAAFKS00-013-R04L0.DAL" ,
10       "overallProgress" : 98
11     }
12   }
13 }
```

- `updateStatus` is display text, not a machine token. It carries whatever the firmware shows an operator, file names included, and four of the five recorded notifications carried a value that is not one of the five `fwUpdateStatus.updateStatus` declares — only the terminal `FW update success` matched. The declared lowercase values `updating`, `skipping` and `failed` appeared in none of them: the reader sent sentences such as `Updating file ...` and `Skipping file ...` instead, so the obvious test `updateStatus === 'updating'` matches nothing this reader sends. Render the string as opaque text and drive logic from `overallProgress` and from the terminal literal under Confirmation below. The schema remains normative; this is an observed device deviation, cataloged in [Known device deviations from the schema](#).
- **A status string can arrive incomplete.** The notification at 99% reported `Skipping file ...`, naming a path inside the firmware package, and the value ended mid-name, its final character recorded as an encoding-error artifact rather than as text. Whether the reader truncated the string or the recording chain corrupted it cannot be told from a single sample, and no maximum length can be inferred from one either: do not assume a length limit, and do not substring-match. `overallProgress` in that same notification was a well-formed `99`, which is the practical case for branching on the number rather than on the text.
- **The stream's timing is not a contract.** Across the recorded window the gap between notifications ran from 10.014 s down to 0.284 s while `overallProgress` advanced by 8, then 3, then 1, then 1 percentage points — neither a fixed interval nor a fixed increment, so do not derive a silence timeout from it. Emission is gated by `eventConfiguration.firmwareUpdate` in [config_events](#) and can be switched off altogether. The stream therefore gives you no deadline, so bound the wait on your own clock: set a wall-clock budget for the whole update from your own fleet's measurements, and on expiry poll [get_version](#) until the reader answers. Treat silence as unknown, never as failure.

Session Impact

While the update runs, the reader rejects **every other command** on the response topic with `code 4` ("Firmware update in [progress](#)"), echoing the blocked command's name:

Response — another command blocked during the update (code 4)

```
1  {
2    "command" : "get_endpoint_config" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 4 ,
7      "description" : "Firmware update in progress"
8    }
9  }
```

The reader reflashes and reboots, and is briefly off the network.

Confirmation

The reader also reports the outcome on the event topic. The terminal notification carries `updateStatus: FW update success` with `overallProgress: 100`:

Event — FIRMWARE_UPDATE success, 100% (live)

```
1  {
2    "type" : "NOTIFICATION" ,
3    "timestamp" : "2026-07-07T11:50:05.893Z" ,
4    "state" : "ONESHOT" ,
5    "id" : "FIRMWARE_UPDATE" ,
6    "priority" : "HIGH" ,
7    "alertDetails" : {
8      "fwUpdateStatus" : {
9        "updateStatus" : "FW update success" ,
10       "overallProgress" : 100
11      }
12    }
13  }
```

That literal is the one `updateStatus` value worth an equality test — it is a declared enum member and the observed notification matched it exactly — with `overallProgress: 100` as corroboration rather than as the test, because a single observation cannot establish that 100 always means success. Every notification recorded, the terminal one included, carried `state: ONESHOT`, so the terminal signal sits in `fwUpdateStatus` and not in the envelope: **do not wait for a** `CLEAR`.

Gate on the read-back, not on the notification. Run `get_version` and compare `readerVersion.firmwareVersion` against the target build (see the version note below). The notification tells you when to stop waiting; the read-back tells you whether the firmware changed. A failed update leaves the prior version in place; terminal failure is reported as `fwUpdateStatus.updateStatus: failed` in the `FIRMWARE_UPDATE` event (which carries no response code), not on the `set_os` accept. `failed` is the value the schema declares for that case; no failure arose in the traffic behind the [examples](#) above, and because two of the three bare-stem values the schema declares arrive as sentences on this firmware, a real failure may well arrive as a sentence too — so detect failure by the read-back, not by matching `failed`.

Response Codes

`set_os` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the accept code for this command . The update proceeds asynchronously; verify with <code>get_version</code> .
0	Success — terminal success; in practice <code>set_os</code> acks with 1, and completion is confirmed via the <code>get_version</code> read-back.
4	Firmware update in progress (<code>IOT_STATUS_FW_UPDATE_IN_PROGRESS</code>) — returned to other commands while an update is running (the device-wide lock). Pause orchestration until the device returns.
8	Insufficient flash size (<code>IOT_ERROR_LOW_FLASH_SIZE</code>) — not enough free flash to download the firmware. Free space or use a smaller package.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — the firmware <code>url</code> is wrong or unreachable.
13	<code>Firmware update Failed</code> (<code>IOT_ERROR_FW_UPDATE_FAIL</code>) — the defined failure code. In practice <code>set_os</code> acks only with 1; a failed update is reported as <code>fwUpdateStatus.updateStatus: failed</code> in the <code>FIRMWARE_UPDATE</code> event (which carries no response code), not on the <code>set_os</code> response — but do not equality-test that literal; see Confirmation .
14	Battery is low, Cannot update firmware (<code>IOT_ERROR_FW_UPDATE_FAIL_LOW_BATTERY</code>) — charge or connect external power, then retry.

Code	Description
101	Default error — generic catch-all.

Gate success strictly on the `get_version` read-back, not on the `set_os` response code — the reply `code 1` acknowledges only that the payload was accepted, and is returned for both success and failure.

Examples

Publish `set_os`; the device acks with the `code 1` envelope under [Response Payload](#), then runs the [Lifecycle](#). The `CERTIFICATE` variants are in the [Request Payload](#) selector; the `FIRMWARE_UPDATE` notifications — start, in-flight progress, and terminal success — and the `code 4` lock are shown under the [Lifecycle](#) above.

Notes and Caveats

⚠ REFLASHES AND REBOOTS THE READER — PLAN FOR UNREACHABILITY

The device goes dark and locks out other commands. While the update runs, the reader rejects **every** other command with `code 4` ("Firmware update in **progress**") and is then unreachable across the reboot. Do not retry-storm — pause orchestration until the device re-attaches. The reply `code 1` is returned for **both** success and failure, and there is no terminal result on the response topic: confirm the outcome with `get_version` and compare the firmware version.

- **Preconditions.** Ensure the device is ready before issuing: low battery → `code 14`; insufficient flash → `code 8`.
- **Confirm by full version string.** `readerVersion.firmwareVersion` is a free-form string. Compare the **whole** value against the target build — the build-variant prefix can change (e.g. `PAAFKS00-013-R02` → `SAAFKS00-013-R02`), so match the full string rather than a substring. See `get_version`.

Sequence a firmware rollout as: pre-check with `get_status` (power/battery) → `set_os` → wait out the reboot (expect `code 4` on other commands) → confirm with `get_version`. For the concept-level view, see [Device Lifecycle](#).

Related:  [Device Lifecycle](#) ·  [get_version](#) ·  [install_certificate](#) ·  [reboot](#) ·  [Response Codes](#)

set_wifi

REFERENCE

API consumer

create or modify a Wi-Fi profile on the reader

Overview

`set_wifi` writes a Wi-Fi profile to the reader — it **creates** a new saved profile or **modifies** an existing one (selected by `wifiConfig.operation` = `CREATE` / `MODIFY`). One payload configures the interface, the access point (ESSID, preference), the security type and its credentials (WPA2/WPA3 Personal or Enterprise), and DHCP IPv4. It is a state-changing command on the device-management ([MGMT](#)) endpoint. The response is status-only — it echoes **no configuration and no credentials**; confirm the result with [get_wifi](#).

`set_wifi` **is state-changing**: `accessPoint.connect: true` **switches networks and can sever the MQTT control session**. Use `connect: false` to save a profile without switching. See the warning callout in [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (writes a saved profile; may switch the active connection)
Reversibility	REVERSIBLE (remove a created profile with delete_wifi_profile ; re-send <code>set_wifi</code> <code>MODIFY</code> to change settings)
Supported Operations	<code>CREATE</code> (new profile), <code>MODIFY</code> (update existing, matched by <code>ssid</code>)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>wifiConfig</code> (with <code>operation</code> and <code>accessPoint</code> ; inside it, <code>ssid</code> and <code>isPreferred</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	get_wifi , delete_wifi_profile , install_certificate

When to Use

Use `set_wifi` to **provision or update the reader's Wi-Fi** — during onboarding, when a network changes, or to mark a profile preferred.

- `CREATE`: first-time provisioning of a new ESSID. Fails with [code 18](#) if the ESSID already exists.
- `MODIFY`: update an existing profile (password, security, preference), matched by `ssid`. Fails with [code 15](#) if the ESSID is not found. A `MODIFY` can be a partial payload — e.g. just `operation` + `accessPoint.{isPreferred, ssid}` to set the preferred flag.
- Pre-provision safely**: send `connect: false` to save a profile without switching the active connection — the recommended pattern when managing a reader over the same Wi-Fi that carries the [MDM](#) session.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at `CONNECT`. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `wifiConfig.operation` (request) — `CREATE` or `MODIFY`; determines whether a new profile is added or an existing one (matched by `ssid`) is updated.

- `wifiConfig.accessPoint.essid` (request) — the network name; the create target / modify key (case-sensitive).
- `wifiConfig.accessPoint.connect` (request) — ⚠️ `true` switches the active connection (can sever the MDM session); `false` just saves. See the notes below.
- `wifiConfig.accessPoint.security.securityType` (request) — selects which `securityDetails` key you populate (WPA2/WPA3 Personal or Enterprise).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is `command`, `requestId`, and the `wifiConfig` payload. Use the **Schema** tab for the full field reference and the **Example** tab for a ready-to-send WPA2-Personal `CREATE` (enterprise variants are in [Examples](#)).

Credentials in the examples are the contract's sample values, not real secrets — passwords, passphrases, and identities are write-only (see [Request Semantics](#)).

Pick a **profile scenario** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Profile scenario

Create WPA2 personal



Request payload

Example: WPA2-Personal CREATE

```
1  {
2    "command" : "set_wifi" ,
3    "requestId" : "abc123" ,
4    "wifiConfig" : {
5      "interfaceName" : "wlan0" ,
6      "enableInterface" : true ,
7      "operation" : "CREATE" ,
8      "accessPoint" : {
9        "connect" : false ,
10       "isPreferred" : false ,
11       "autoConn" : true ,
12       "essid" : "TestAP1" ,
13       "enableSecurity" : true ,
14       "security" : {
15         "securityType" : "WPA2Personal" ,
16         "securityDetails" : {
17           "WPA2Personal" : {
18             "password" : "test@123"
19           }
20         }
21       }
22     }
23   }
24 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	command *	string	—	set_v
2	requestId *	string	—	abc12
3	wifiConfig *	object	—	—
4	interfaceName	string	—	wlan0

#	FIELD	TYPE	ENUM / UNITS	EXAMP
5	enableInterfac e	boolean	—	true
6	operation *	enum	CREATE MODIFY	CREAT
7	accessPoint *	object	—	—
8	connect	boolean	—	false
9	isPreferre d *	boolean	—	false
10	autoConn	boolean	—	true
11	essid *	string	—	ZWire
12	enableSecur ity	boolean	—	true

#	FIELD	TYPE	ENUM / UNITS	EXAMP
13	ipv4Configu ration	object	—	—
14	ipAddress	string	ipv4	192.1
15	subnetMas k	string	ipv4	255.2
16	gateway	string	ipv4	192.1
17	enableDhc p	boolean	—	true
18	dnsServer	string	ipv4	192.1
19	domainNam e	string	—	exam
20	security	object	—	—
21	securit yType *	enum	WPA2Personal WPA3Personal WPA2Enterprise WPA3Enterprise	WPA2P
22	securit yDetail * s	object	—	—
23	WPA2Pe rsonal	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
24	password	string	—	test()
25	WPA3Personal	object	—	—
26	password	string	—	—
27	WPA2Enterprise	object	—	—
28	authentication	enum	tls ttls peap	peap
29	innerAuthentication	enum	none mschapv2	mscha
30	identity	string	—	test
31	anonymousIdentity	string	—	test

#	FIELD	TYPE	ENUM / UNITS	EXAMP
32	password	string	—	test0
33	password	string	—	test0
34	certificate	array	—	—
35	key	enum	ca_cert client_key client_cert cert_key_password	ca_ce
36	value	string	—	wifi_
37	protocol	enum	WPA2_Enterprise_CCMP	WPA2_
38	WPA3Enterprise	object	—	—
39	authentication *	enum	tls ttls peap	peap

#	FIELD	TYPE	ENUM / UNITS	EXAMP
40	innerAuthentication	enum	none mschapv2	mscha
41	identity	string	—	test
42	anonymousIdentity	string	—	test
43	password	string	—	test
44	phrase	string	—	test
45	certificate	array	—	—
46	key	enum	ca_cert client_key client_cert cert_key_password	ca_ce
47	value	string	—	wifi_

#	FIELD	TYPE	ENUM / UNITS	EXAMP
48	profile	enum	WPA3_Enterprise_CCMP WPA3_Enterprise_CCMP_256 WPA3_Enterprise_GCMP_128 WPA3_Enterprise_GCMP_256_SHA256 WPA3_Enterprise_GCMP_256_SUITEB_192	WPA3_

Request Semantics

- **CREATE vs MODIFY.** `CREATE` adds a new profile (fails `18` if the `ssid` exists); `MODIFY` updates an existing profile matched by `ssid` (fails `15` if not found). The device enforces a maximum number of saved profiles — exceeding it returns `19`; delete unused profiles with `delete_wifi_profile`. For Enterprise TLS, the `ca_cert` / `client_cert` / `client_key` referenced in `certificate` must already be installed via `install_certificate`.
- **Credentials are write-only — mask them.** Supply passwords, passphrases, and identities in the request (the examples carry sample values); the device never echoes them — `set_wifi`'s response carries no credentials, and `get_wifi` returns stored secrets masked.
- **Enterprise inner authentication.** For `ttls` and `peap`, `innerAuthentication` must be `mschapv2` and defaults to `mschapv2` when omitted; for `tls` it is not applicable and can be omitted, as the `tls` variants on this page do.
- **IPv4 only, DHCP only.** In the current API version, `set_wifi` supports **IPv4 with DHCP assignment only**. The `ipv4Configuration` static fields (`ipAddress` / `subnetMask` / `gateway` / `dnsServer`) are modeled in the schema but **not supported yet** (planned for a future release), and **IPv6 is not available**. Use `enableDhcp: true`.

Response Payload

`set_wifi` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no configuration echo and no credential in the response; to confirm the saved profile, follow up with `get_wifi`.

Response payload

```

1  {
2    "command" : "set_wifi" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	set_wifi	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — set_wifi echoes no configuration and no credentials.
5	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Response Codes

`set_wifi` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the profile was created or modified.
2	Invalid payload (<code>IOT_ERROR_INVALID_PAYLOAD</code>) — the JSON is malformed or has invalid field values. Validate against the schema and correct.
15	WIFI Error - SSID not found (<code>IOT_ERROR_SSID_NOT_FOUND</code>) — a <code>MODIFY</code> targeted an <code>essid</code> that is not saved. Verify the name or <code>CREATE</code> it first.
17	WIFI Error - SSID missed (<code>IOT_ERROR_SSID_MISSED</code>) — the required <code>essid</code> field is missing from the payload.
18	WIFI Error - SSID already exist (<code>IOT_ERROR_SSID_ALREADY_EXIST</code>) — a <code>CREATE</code> used an <code>essid</code> that already exists. Use <code>MODIFY</code> , or delete the existing profile first.
19	WIFI Error - SSID count overflow (<code>IOT_ERROR_SSID_LIMIT_OVERFLOW</code>) — the saved-profile limit is reached. Delete an unused profile with delete_wifi_profile .
20	Wifi is not supported (<code>IOT_ERROR_WIFI_INTER_NOT_SUPPORTED</code>) — the device has no Wi-Fi interface; use Ethernet.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>operation</code> , <code>securityType</code> , <code>authentication</code>) is outside its allowed set.
101	Default error — generic catch-all.

Examples

Publish a `set_wifi` request; the device replies with the status-only envelope shown under [Response Payload](#). Every contract example is in the **Profile scenario** dropdown under [Request Payload](#):

- **Create WPA2 personal and Modify WPA2 personal** — a pre-shared key.
- **Set preferred AP** — a partial `MODIFY` that needs only `operation` plus `accessPoint`.
{isPreferred, essid}.
- **Create WPA2 enterprise TLS / TTLS / PEAP and Create WPA3 enterprise TLS / TTLS / PEAP** — installed certificates referenced as {key, value}, an EAP `authentication` method and a `protocol`; the TTLS and PEAP variants add `innerAuthentication`.

Notes and Caveats



`connect: true` CAN SEVER THE CONTROL SESSION

`accessPoint.connect: true` disconnects the currently connected profile and connects to the target ESSID. If the reader manages itself over the same Wi-Fi that carries its MQTT/MDM session, switching to a different network **drops the control channel** and can strand the device. When managing a reader over Wi-Fi, use `connect: false` (save without switching), and undo an unwanted profile with `delete_wifi_profile` rather than connecting onto it.

Because the response is status-only, **verify the result with** `get_wifi` — check that the profile was saved (and its `isPreferred` / `accessPoint.status`). For the concept-level view, see [Network Configuration](#).

Related: [Network Configuration](#) · `get_wifi` · `delete_wifi_profile` · `install_certificate` · [Response Codes](#)

control_operation

REFERENCE

API consumer

start or stop the RFID radio (or the scanner) on demand

Overview

`control_operation` **starts or stops** a reader subsystem on demand. You pick a `controlType` (`RFID` — the radio inventory engine, or `SCANNER` — the barcode scanner) and an `operation` (`START` or `STOP`). For `RFID`, `START` begins an inventory cycle **using the currently configured [operating mode](#)** and the device streams tag reads as `dataEVT` events until you `STOP` (or the operating-mode stop conditions are met). It is a state-changing **control-plane ([CTRL](#)) command**, and it is **reversible** — a `START` is undone by a `STOP`.

This is a **CTRL-endpoint command**, published on the **control** endpoint

(`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel).

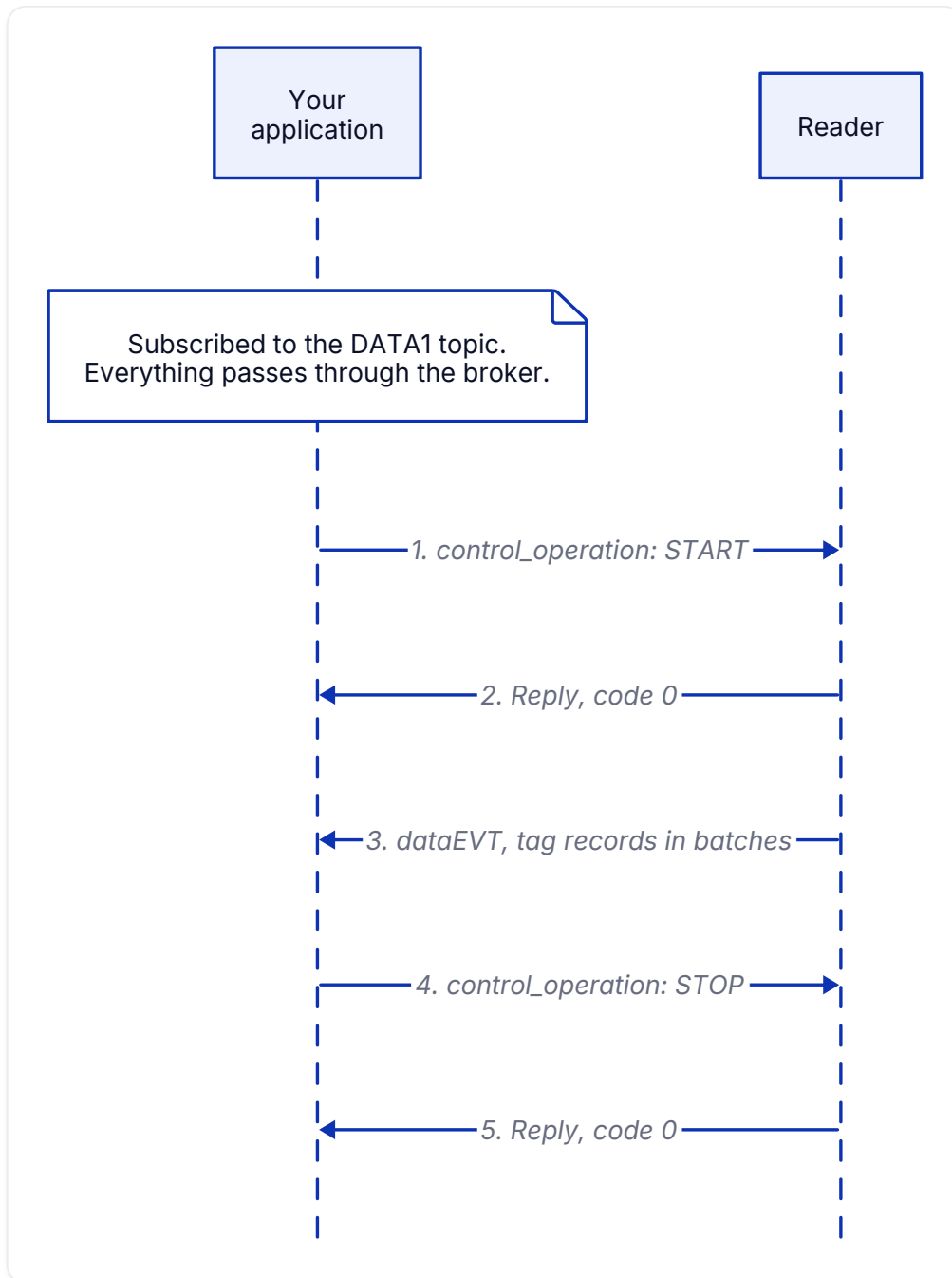
Returning a response requires an **active CTRL endpoint** (`config_endpoint`, `epType: CTRL`, `activate: true`). Receiving the tag stream that `START` triggers also requires an **active `DATA1` endpoint**

(`<tenantId>/DATA/clients/dataevent/<serial>`); see the [data endpoint topic mapping](#)).

Set the [regulatory region](#) first. Before you start an RFID inventory, make sure the reader's regulatory region is configured: the reader does not transmit until it is, and the region is set outside MQTT with 123RFID Desktop or 123RFID Mobile. Use `get_current_region` to verify the current region.

A `START` **executes whatever the operating mode has armed** — including destructive access operations. See the danger callout in [Notes and Caveats](#).

A first inventory, once the region is set, the operating mode is configured and you have subscribed to the `DATA1` topic:



Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (starts / stops the radio or scanner)
Reversibility	REVERSIBLE (a <code>START</code> is undone by a <code>STOP</code>)
Supported Operations	<code>START</code> , <code>STOP</code>

Property	Value
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>ctrlOprPayload</code> is optional in the contract; inside it, <code>controlType</code> and <code>operation</code> are required)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	<code>set operating mode</code> , <code>get operating mode</code> , <code>get status</code> , <code>reboot</code>

When to Use

Use `control_operation` to drive the radio (or scanner) on demand:

- **Start an inventory** — `controlType: RFID`, `operation: START` begins reading tags with the current operating mode; the device streams `dataEVT` events over the DATA endpoint.
- **Stop an inventory** — `operation: STOP` halts the radio; the tag stream ceases.
- **Control the scanner** — `controlType: SCANNER` starts / stops the barcode scanner instead of the radio.

Configure *how* the reader reads with `set operating mode` **first**, then `START` — `control_operation` itself has no tuning parameters. If the inventory state is uncertain, check it with `get status` before sending `START`.

Topic Mapping

`control_operation` is the one command that drives **both** planes: the command travels over CTRL, while the device publishes the tag events a `START` triggers over DATA.

Plane	Message	Direction	Topic	Note
CONTROL	Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
CONTROL	Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Plane	Message	Direction	Topic	Note
DATA	Event	Reader → App	<tenantId>/DATA/clients/dataevent/<serial>	The <code>dataEVT</code> stream a <code>RFID</code> <code>START</code> triggers. Requires an active <code>DATA1</code> endpoint to receive it.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The command is answered **only on the CTRL endpoint**; the tag stream is published on the `DATA1` endpoint's configured publish topic (`dataevent` by default). `DATA2` is currently not supported.

See [Topics and Channels](#).

Key Fields

- `ctrlOprPayload.controlType` (request) — `RFID` (radio inventory engine) or `SCANNER` (barcode scanner).
- `ctrlOprPayload.operation` (request) — `START` (begin) or `STOP` (halt).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `ctrlOprPayload` (`controlType` + `operation`). Pick an **operation** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Operation

Start inventory

▼

Request payload

Example: RFID START

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abcd1432" ,
4    "ctrlOprPayload" : {
5      "controlType" : "RFID" ,
6      "operation" : "START"
7    }
8  }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	enum	control_operation	control_operation	Fixed command name (pinned by the schema enum).
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	ctrlOprPayload	object	—	—	The control operation to perform. Optional in the contract; when present, the contract requires both controlType and operation within it.
4	controlType *	enum	RFID SCANNER	RFID	Which reader subsystem to control — RFID (the radio inventory engine) or SCANNER (the barcode scanner). Required within the payload.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	operation *	enum	START STOP	START	START begins the operation for the selected subsystem; STOP halts it. Required within the payload.

Request Semantics

Modes. `ctrlOprPayload.operation` selects the action for the chosen `controlType`:

- `START` — begin the operation. For `RFID`, this starts an inventory cycle using the current operating mode and streams `dataEVT` events over the DATA endpoint.
- `STOP` — halt the active operation. For `RFID`, the tag stream ceases (a single in-flight read may still be flushed shortly after — see Notes).

A `STOP` when no operation is active returns [code 12](#) ("No Radio Operation in Progress") — this is **not a failure**, the device is already idle. A `START` while an inventory is already running returns [code 11](#) ("Inventory in progress").

- `SCANNER` uses the same request envelope as `RFID`. `ctrlOprPayload` declares one `controlType` (`RFID` | `SCANNER`) and one `operation` (`START` | `STOP`), both required, with no scanner-specific field and no separate payload — a scanner request differs from an RFID request only in the `controlType` value. Decoded barcodes are declared on the DATA plane, not in this command's response: the `dataEVT` data object carries `barcodeData[]` (`decodedBarcode`, plus `symbology`, the barcode type as a hexadecimal string such as `0x03`) alongside `tagData[]`. Codes 11 and 12 above are radio-scoped — the contract describes them as an RFID inventory already running and no radio operation active — and the schema defines no scanner-specific response code, leaving [code 23](#) for a `controlType` or `operation` outside its enum. Physical-trigger behavior and per-model scanner coverage are outside the schema and are not documented here — see Scan Barcodes.

Response Payload

`control_operation` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. The tag reads arrive separately as `dataEVT` events on the DATA endpoint; the response carries no data echo.

Response payload

```
1 {
2   "command": "control_operation",
3   "requestId": "abc123",
4   "apiVersion": "V1.1",
5   "response": {
6     "code": 0,
7     "description": "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	control_operation	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — control_operation echoes no configuration.
5	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.
6	description *	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful `START` or `STOP` can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`control_operation` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the radio (or scanner) started or stopped. (Readers have been reported to omit <code>code</code> on this reply — see above.)
11	Inventory in progress (<code>IOT_ERROR_INVENTORY_IN_PROGRESS</code>) — a <code>START</code> was sent while an RFID inventory is already running. Stop the current inventory first; if the state is uncertain, check <code>get_status</code> .
12	No Radio Operation in Progress (<code>IOT_ERROR_NO_RADIO_OP_IN_PROGRESS</code>) — a <code>STOP</code> was sent while the radio is already idle. Not a failure — no action required. Unique to this command.
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — <code>controlType</code> or <code>operation</code> is outside its allowed set.
101	Default error — generic catch-all.

Examples

Publish `control_operation` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). The **Operation** dropdown under [Request Payload](#) shows the `START` and `STOP` requests. The [examples](#) below show what the dropdown does not: the tag stream that `START` triggers, and the two common error responses.

A `RFID` `START` triggers a `dataEVT` **stream** on the DATA endpoint. Each event's `type` echoes the active operating-mode profile, and the mode's `tagMetadataToEnable` flags gate which `tagData` fields appear (here the default mode reports `EPCid` / `eventNum` / `peakRssi` / `seenCount` only):

DATA plane — dataEVT triggered by START (abridged)

```
1  {
2    "type": "BALANCED_PERFORMANCE",
3    "timestamp": "2026-06-16T06:03:36.894Z",
4    "data": {
5      "tagData": [
6        {
7          "EPCid": "E28011B0A506007D3DD07C0F",
8          "eventNum": 41,
9          "peakRssi": -73,
10         "seenCount": 256
11       },
12       {
13         "EPCid": "311100000000000000000000",
14         "eventNum": 42,
15         "peakRssi": -50,
16         "seenCount": 256
17       },
18       {
19         "EPCid": "11113333880600890630DBEC",
20         "eventNum": 43,
21         "peakRssi": -53,
22         "seenCount": 256
23       },
24       {
25         "EPCid": "000011C1A5000062F7952926",
26         "eventNum": 44,
27         "peakRssi": -65,
28         "seenCount": 256
29       },
30       {
31         "EPCid": "E200001B880600870630DBEB",
32         "eventNum": 45,
33         "peakRssi": -44,
34         "seenCount": 256
35       },
36       {
37         "EPCid": "1111333300004001B043ACB2",
38         "eventNum": 46,
39         "peakRssi": -36,
40         "seenCount": 256
41       },
42       {
43         "EPCid": "11113333333344445555AAAA",
44         "eventNum": 47,
45         "peakRssi": -32,
46         "seenCount": 256
47       },
48       {
49         "EPCid": "11113333A5000062F798E5B6",
50         "eventNum": 48,
51         "peakRssi": -34,
52         "seenCount": 256
53       },
54     ]
55   }
```

```

55     "EPCid" : "1111333333344445555AAAA" ,
56     "eventNum" : 49 ,
57     "peakRssi" : -36 ,
58     "seenCount" : 256
59 },
60 {
61     "EPCid" : "11115555222233334444A9B6" ,
62     "eventNum" : 50 ,
63     "peakRssi" : -43 ,
64     "seenCount" : 256
65 },
66 {
67     "EPCid" : "111133332222333344445555" ,
68     "eventNum" : 51 ,
69     "peakRssi" : -44 ,
70     "seenCount" : 256
71 },
72 {
73     "EPCid" : "111133332222333344445550" ,
74     "eventNum" : 52 ,
75     "peakRssi" : -43 ,
76     "seenCount" : 256
77 },
78 {
79     "EPCid" : "FFFFFFFF3435363742426666" ,
80     "eventNum" : 53 ,
81     "peakRssi" : -51 ,
82     "seenCount" : 256
83 },
84 {
85     "EPCid" : "11113333333344445555AAAA" ,
86     "eventNum" : 54 ,
87     "peakRssi" : -46 ,
88     "seenCount" : 256
89 },
90 {
91     "EPCid" : "11113333880600900630DBF1" ,
92     "eventNum" : 55 ,
93     "peakRssi" : -52 ,
94     "seenCount" : 256
95 },
96 {
97     "EPCid" : "E200001B880600840630DBE2" ,
98     "eventNum" : 56 ,
99     "peakRssi" : -57 ,
100     "seenCount" : 256
101 },
102 {
103     "EPCid" : "111133332222333344445555" ,
104     "eventNum" : 57 ,
105     "peakRssi" : -63 ,
106     "seenCount" : 256
107 },
108 {
109     "EPCid" : "111133332222333344445555" ,
110     "eventNum" : 58 ,

```

```

111         "peakRssi" : -63 ,
112         "seenCount" : 256
113     },
114     {
115         "EPCid" : "11113333222333344445555" ,
116         "eventNum" : 59 ,
117         "peakRssi" : -62 ,
118         "seenCount" : 256
119     },
120     {
121         "EPCid" : "E200001B880600870630DBEB" ,
122         "eventNum" : 60 ,
123         "peakRssi" : -44 ,
124         "seenCount" : 256
125     },
126     {
127         "EPCid" : "000011C1A5000062F7952926" ,
128         "eventNum" : 61 ,
129         "peakRssi" : -66 ,
130         "seenCount" : 256
131     },
132     {
133         "EPCid" : "11113333B22FB22FB22FB22F" ,
134         "eventNum" : 62 ,
135         "peakRssi" : -65 ,
136         "seenCount" : 256
137     },
138     {
139         "EPCid" : "AAAABBBB11112222CCCCDDDD" ,
140         "eventNum" : 63 ,
141         "peakRssi" : -60 ,
142         "seenCount" : 256
143     },
144     {
145         "EPCid" : "BEDD11112222333344445555" ,
146         "eventNum" : 64 ,
147         "peakRssi" : -45 ,
148         "seenCount" : 256
149     },
150     {
151         "EPCid" : "311100000000000000000000" ,
152         "eventNum" : 65 ,
153         "peakRssi" : -51 ,
154         "seenCount" : 256
155     },
156     {
157         "EPCid" : "11113333880600890630DBEC" ,
158         "eventNum" : 66 ,
159         "peakRssi" : -53 ,
160         "seenCount" : 256
161     },
162     {
163         "EPCid" : "1111FB63AC1F3841DADA00000000CAFEDADACAFEDADACAFE00001111" ,
164         "eventNum" : 67 ,
165         "peakRssi" : -57 ,
166         "seenCount" : 256

```

```

167     },
168     {
169         "EPCid" : "E28011B0A506007D3DD07D01" ,
170         "eventNum" : 68 ,
171         "peakRssi" : -72 ,
172         "seenCount" : 256
173     },
174     {
175         "EPCid" : "E28011B0A506007D3DD07C0F" ,
176         "eventNum" : 69 ,
177         "peakRssi" : -69 ,
178         "seenCount" : 256
179     },
180     {
181         "EPCid" : "E28011B0A506007D3DD07D00" ,
182         "eventNum" : 70 ,
183         "peakRssi" : -70 ,
184         "seenCount" : 256
185     },
186     {
187         "EPCid" : "11115555222233334444A9B6" ,
188         "eventNum" : 71 ,
189         "peakRssi" : -41 ,
190         "seenCount" : 256
191     },
192     {
193         "EPCid" : "111133332222333344445550" ,
194         "eventNum" : 72 ,
195         "peakRssi" : -45 ,
196         "seenCount" : 256
197     },
198     {
199         "EPCid" : "311100000000000000000000" ,
200         "eventNum" : 73 ,
201         "peakRssi" : -65 ,
202         "seenCount" : 256
203     },
204     {
205         "EPCid" : "111133332222333344445555" ,
206         "eventNum" : 74 ,
207         "peakRssi" : -61 ,
208         "seenCount" : 256
209     },
210     {
211         "EPCid" : "A1B2222BBBB" ,
212         "eventNum" : 75 ,
213         "peakRssi" : -65 ,
214         "seenCount" : 256
215     }
216 ]
217 }
218 }

```

A `STOP` issued while the radio is **already idle** returns [code 12](#) (not a failure):

Response — STOP while idle (code 12)

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 12 ,
7      "description" : "No Radio Operation in Progress"
8    }
9  }
```

A `START` issued while an inventory is already running returns [code 11](#):

Response — START while inventory running (code 11)

```
1  {
2    "command" : "control_operation" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 11 ,
7      "description" : "Inventory in progress"
8    }
9  }
```

Notes and Caveats

⚠️ ARMED ACCESS OPERATIONS EXECUTE ON `START`

`control_operation` has no tuning parameters; a `START` runs whatever `set_operating_mode` last configured. If destructive access operations (`WRITE` / `LOCK` / `KILL`) are **armed**, `START` executes them on every tag in range — `KILL` and `PERMANENT_LOCK` are **irreversible**. Confirm the operating mode is read-only (no armed `accessOperations`) before starting a production inventory. See `set_operating_mode` \$. [Notes and Caveats](#).

- `START` **drives two planes; the tag stream requires a DATA endpoint**. The command round-trip is on the CTRL endpoint, but the tag reads a `RFID` `START` produces are published as `dataEVT` events on the **DATA** endpoint (`<tenantId>/DATA/clients/dataevent/<serial>` for `DATA1_EP`). To receive them, a `DATA1` endpoint must be active — activate one with `config_endpoint` . `dataEVT.type` echoes the active operating-mode profile, and the mode's `tagMetaDataSetToEnable` flags gate which `tagData` fields appear (`EPCid` is always present).
- `STOP` **is immediate at the radio, but a trailing read may still flush**. A `STOP` stops the radio right away, yet a single in-flight / buffered read may still be delivered on the DATA plane a couple of seconds after the `STOP` (≈ 2.9 s observed on RFD40) before the stream fully ceases. Do not treat one trailing `dataEVT` after a `STOP` as continued inventory.
- **Configure the mode before you `START`** . Set profiles, power, triggers, `select` filters, and access operations with `set_operating_mode` first, then `START` ; `control_operation` itself sets none of these. The operating

mode is **volatile**: it resets to the default (`BALANCED_PERFORMANCE` , no armed access operations) on `reboot` , so re-apply it after a reboot before starting.

- **An attached terminal has priority.** Apps on a terminal attached to the reader, whether Android, iOS or Windows, always have priority over MQTT commands, and a user can start an inventory from the terminal while the reader is connected to the broker. What the reader replies to a `control_operation` that a terminal app overrides is not specified in the available sources. See [The Reader Is Its Own MQTT Client](#).

`control_operation` is **reversible** (`START` ↔ `STOP`) and carries **no credentials** — the tag `EPCid` values in the resulting `dataEVT` stream are identifiers read off physical tags, not secrets. `SCANNER` control follows the same request/response shape as `RFID` (verified paths are `RFID` `START` / `STOP` on RFD40). For the concept-level view, see [Inventory Control](#).

Related:  [set_operating_mode](#) ·  [get_operating_mode](#) ·  [get_status](#) ·  [dataEVT](#) ·  [Inventory Control](#) ·  [Scan Barcodes](#) ·  [Response Codes](#)

get_operating_mode

REFERENCE

API consumer

read back the reader's current RFID [operating-mode](#) configuration

Overview

`get_operating_mode` reads back the reader's current RFID operating-mode configuration — the active profile, radio start/stop conditions, Gen2 `query` parameters, select pre-filters, armed access operations, and tag-metadata reporting flags — under `operatingMode.operatingModes`. It is read-only. It is a **control-plane (CTRL) command** — configure the mode with `set_operating_mode`.

This is a **CTRL-endpoint command**: it is published on the **control** endpoint (`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint** (configured with `config_endpoint`, `epType: CTRL`, `activate: true`). See [Topic Mapping](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set_operating_mode</code> , <code>control_operation</code> , <code>config_endpoint</code> , <code>reboot</code>

When to Use

Use `get_operating_mode` to inspect or audit the reader's RFID configuration:

- Before an inventory** — confirm the profile, `query` parameters, and trigger conditions are what you expect.
- After `set_operating_mode`** — verify the mode was applied (there is no configuration echo on the set command's response).

- **After a reboot** — the operating mode reverts to default on reboot (see the volatility note); read it back to confirm the current state.
- **Check reporting** — see which `tagMetadataToEnable` fields the reader will include on tag reads.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Unlike the MGMT commands, control commands are answered **only on the CTRL endpoint** — publishing `get_operating_mode` to the MGMT/MDM command topic produces no reply.

If no CTRL endpoint is active, add or activate one with `config_endpoint` (`epType: CTRL`, `activate: true`) before sending control commands. The `requestId` you send is echoed in the response.

See [Topics and Channels](#).

Key Fields

- `operatingMode.operatingModes.profiles` (response) — the active profile (e.g. `BALANCED_PERFORMANCE`); **ADVANCED** surfaces `advancedConfigurations`.
- `query` (response) — the Gen2 session / inventory-state / SL-flag / tag-population parameters driving inventory.
- `radioStartConditions` / `radioStopConditions` (response) — the trigger semantics for starting/stopping inventory.
- `tagMetadataToEnable` (response) — which metadata fields (RSSI, EPC, TID, ...) the reader reports on each tag read.
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission. See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` (pinned to `get_operating_mode`) and `requestId`. There are no parameters.

Request payload

```
1 {
2   "command" : "get_operating_mode" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	enum	get_operating_mode	get_operating_mode	Fixed command name (pinned by the schema enum).
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the current mode under `operatingMode.operatingModes` . The device returns only the configured sub-objects — `advancedConfigurations` , `accessOperations` , and `select` appear only when configured. The **Example** tab shows a default `BALANCED_PERFORMANCE` read-back.

Response payload

```
1  {
2    "command": "get_operating_mode" ,
3    "requestId": "abc123" ,
4    "apiVersion": "V1.1" ,
5    "operatingMode": {
6      "operatingModes": {
7        "profiles": "BALANCED_PERFORMANCE" ,
8        "inventoryProtocol": "GEN2" ,
9        "radioStartConditions": {
10         "trigger": "IMMEDIATE" ,
11         "startDelay": 0 ,
12         "repeat": false
13       },
14       "radioStopConditions": {
15         "trigger": "IMMEDIATE"
16       },
17       "query": {
18         "session": "SESSION_1" ,
19         "inventoryState": "STATE_A" ,
20         "slFlag": "ALL" ,
21         "tagPopulation": 30
22       },
23       "tagMetaDataToEnable": {
24         "RSSI": true ,
25         "PHASE": false ,
26         "SEENCOUNT": true ,
27         "CHANNEL": false ,
28         "PC": false ,
29         "XPC": false ,
30         "CRC": false ,
31         "EPC": false ,
32         "TID": false ,
33         "USER": false ,
34         "MAC": false ,
35         "HOSTNAME": false
36       }
37     }
38   },
39   "response": {
40     "code": 0 ,
41     "description": "Success"
42   }
43 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMP
1	command *	string	—	get_c
2	requestId *	string	—	abc12
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1
4	operatingMode	object	—	—
5	operatingModes	object	—	—
6	profiles	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	BALAN
7	advancedConfigurations	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
8	transmitPower	number	0-300	270
9	linkProfile	enum	M4_256K M2_240K M2_256K M2_320K M4_240K M4_320K FM0_0K FM0_320K M8_240K M8_256K M8_320K	M2_24
10	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
11	dynamicPower	boolean	—	true
12	inventoryProtocol	enum	GEN2 GEN2X HYBRID	GEN2
13	accessOperations	array	—	—
14	operationType	enum	READ WRITE ACCESS LOCK KILL	READ
15	config	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
16	m e m o r y B a n k	enum	EPC TID USER RESERVED	EPC
17	o f f s e t	integer	—	1
18	l e n g t h	integer	—	2
19	d a t a	string	—	FFFFE
20	p a s s w o r d	string	—	0000C

#	FIELD	TYPE	ENUM / UNITS	EXAMP
21	lockMemoryBank	enum	EPC ACCESS_PWD KILL_PWD TID USER ALL	TID
22	lockAccession	enum	READ_AND_WRITE PERMANENT_LOCK PERMANENT_UNLOCK UNLOCK	UNLOCK
23	radioStartConditions	object	—	—
24	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEDIATE
25	startDelay	integer	—	3000
26	repeat	boolean	—	false
27	radioTopConditions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
28	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEI
29	tag Count	integer	—	10
30	stopTimeOut	integer	—	10000
31	inventory Count	integer	—	0
32	query	object	—	—
33	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
34	inventory State	enum	STATE_A STATE_B STATE_AB	STATE
35	slF lag	enum	ALL ASSERTED DEASSERTED	ALL
36	tag Population	integer	min 0	30
37	select	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
38	enable *	boolean	—	true
39	tagParameter *	string	—	2222E
40	memoryBank *	enum	EPC TID USER	EPC
41	offset *	integer	—	32
42	action *	enum	INV_A_NOT_INV_B_OR_ASRT_SL_NOT_DSRT_SL INV_A_OR_ASRT_SL NOT_INV_B_OR_NOT_DSRT_SL INV_A2BB2A_NOT_INV_A_OR_NEG_SL_NOT_ASRT_SL INV_B_NOT_INV_A_OR_DSRT_SL_NOT_ASRT_SL INV_B_OR_DSRT_SL NOT_INV_A_OR_NOT_ASRT_SL NOT_INV_A2BB2A_OR_NOT_NEG_SL	INV_E
43	target *	enum	SESSION_SL SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
44	length *	integer	—	16
45	tagMetadataEnable	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
46	RSSI	boolean	—	true
47	PHASE	boolean	—	false
48	SEENCOUNT	boolean	—	true
49	CHANNEL	boolean	—	false
50	PC	boolean	—	false
51	XPC	boolean	—	false
52	CRC	boolean	—	false
53	EPC	boolean	—	false
54	TID	boolean	—	false
55	USER	boolean	—	false
56	MAC	boolean	—	false
57	HOSTNAME	boolean	—	false
58	response *	object	—	—
59	code *	integer	0-101	0
60	description *	string	—	Success

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful read can arrive with `code` omitted — the same envelope and `operatingMode` payload. This form does not validate against the contract; the reply with `code: 0` above is the documented form. The mode object is abridged here:

Response — success reported without code (not accepted by the contract; operatingMode abridged)

```
1  {
2    "command" : "get_operating_mode" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "operatingMode" : {
6      "operatingModes" : {
7        "profiles" : "BALANCED_PERFORMANCE"
8      }
9    },
10   "response" : {
11     "description" : "Success"
12   }
13 }
```

Response Semantics

- **The read-back masks access passwords.** When access operations are armed, `get_operating_mode` returns their structure but echoes `config.password` as empty (`"`) — the stored credential is never returned. The real password is carried only in the `set operating mode` request (mask it there). A default read-back has no `accessOperations` at all.
- **FAST_READ is enumerated but not supported.** The `profiles` enum lists `FAST_READ`, but it is **currently not supported** by the firmware (per the schema's own description). Use `CYCLE_COUNT`, `DENSE_READERS`, `OPTIMAL_BATTERY`, `BALANCED_PERFORMANCE`, or `ADVANCED`.
- **Conditional sub-objects.** `advancedConfigurations` appears only when `profiles` is `ADVANCED`; `accessOperations` and `select` appear only when configured. A default read-back typically returns just `profiles`, `inventoryProtocol`, `radioStartConditions`, `radioStopConditions`, `query`, and `tagMetadataToEnable`.

Response Codes

`get_operating_mode` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the operating mode was retrieved.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the operating mode at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

The contract requires `response.code`; readers have been reported to omit it on success — see [Success without a code](#).

Examples

Publish `get_operating_mode` over the CTRL endpoint (with an active CTRL endpoint); the device replies with the snapshot shown under [Response Payload](#). There are no request variants — the command takes no parameters.

Notes and Caveats

- **Operating mode is volatile.** The operating mode resets to default (`BALANCED_PERFORMANCE` , armed `accessOperations` cleared) on `reboot` . Read it back after a reset to confirm the current state; re-apply with `set_operating_mode` if you were relying on a non-default configuration.

Read `get_operating_mode` to confirm the reader's RF configuration before running an inventory with `control_operation` , and to verify a change applied by `set_operating_mode` . For the concept-level view, see [Operating Mode](#).

Related:  [set_operating_mode](#) ·  [control_operation](#) ·  [config_endpoint](#) ·  [reboot](#) ·  [Response Codes](#)

get_post_filter

REFERENCE

API consumer

read back the reader's current post-filter rules

Overview

`get_post_filter` reads back the reader's current post-filters — the rules that include or exclude scanned RFID tag data before it is reported to a data endpoint. Each rule matches tag data by pattern (prefix, suffix, or regex) and either reports (`INCLUDE`) or drops (`EXCLUDE`) it, scoped to a data endpoint. It is read-only. It is a **control-plane (CTRL) command**; configure filters with `set_post_filter`.

This is a **CTRL-endpoint command**: it is published on the **control** endpoint (`<tenantId>/CTRL/clients/cmdnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint** (`config_endpoint`, `epType: CTRL`, `activate: true`).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set_post_filter</code> , <code>get_operating_mode</code> , <code>config_endpoint</code>

When to Use

Use `get_post_filter` to inspect the reader's post-filters:

- Audit filtering** — see which patterns are included/excluded per data endpoint before relying on the tag-data stream.

- **After** `set_post_filter` — verify a filter was applied (the setter's response carries no configuration echo).
- **Confirm none are set** — an absent `postFilterPayload` means no filters are configured (see [Response Semantics](#)).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint** — publishing `get_post_filter` to the MGMT/MDM command topic produces no reply.

Post-filters act on **tag data** (post-singulation), distinct from the Gen2 `select` pre-filters in `set_operating_mode` that act on the air interface. If control commands time out, confirm a CTRL endpoint is active and activate one with `config_endpoint` if needed.

See [Topics and Channels](#).

Key Fields

- `postFilterPayload.postFilter[]` (response) — the configured rules; absent when none are set.
- `dataEpType` (response) — the data endpoint (`DATA_EP1` / `DATA_EP2`) each rule scopes to. `DATA_EP2` targets the `DATA2` endpoint, which is currently not supported.
- `matchPattern` + `matchPatternMethod` (response) — what to match and how (`PREFIX` / `SUFFIX` / `REGEX`).
- `reportOperation` (response) — `INCLUDE` (report matches) or `EXCLUDE` (drop matches).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` and `requestId`. It returns all configured post-filters and takes no parameters.

Request payload

```
1 {
2   "command" : "get_post_filter" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_post_filter	Fixed command name.
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) plus `postFilterPayload` when filters are configured. The **Example** tab shows one `DATA_EP1` regex filter.

Response payload

Example: with a filter set

```
1 {
2   "command" : "get_post_filter" ,
3   "requestId" : "abc123" ,
4   "apiVersion" : "V1.1" ,
5   "postFilterPayload" : {
6     "postFilter" : [
7       {
8         "dataEpType" : "DATA_EP1" ,
9         "matchPattern" : "FFFF" ,
10        "matchPatternMethod" : "REGEX" ,
11        "reportOperation" : "INCLUDE"
12      }
13    ]
14  } ,
15  "response" : {
16    "code" : 0 ,
17    "description" : "Success"
18  }
19 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_post_filter	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion [*]	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	postFilterPayload	object	—	—	The current post-filter configuration. Present only when at least one filter is configured — omitted entirely (not even an empty array) when none are set (see Response Semantics).
5	postFilter	array	—	—	The configured post-filter rules (one per entry).
6	dataEndpoint	enum	DATA_EP1 DATA_EP2	DATA_EP1	The data endpoint this filter applies to. DATA_EP2 targets the DATA2 endpoint, which is currently not supported.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
7	match pattern	string	—	FFFF	The pattern matched against scanned tag data.
8	match pattern Method	enum	PREFIX SUFFIX REGEX	REGEX	How the pattern is applied — prefix, suffix, or regular expression.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
9	reportOperation	enum	INCLUDE EXCLUDE	INCLUDE	Reports (INCLUDE) or drops (EXCLUDE) matching tag data.
10	response *	object	—	—	Status envelope for the read.
11	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Codes below.
12	description *	string	—	Success	Human-readable status.

When no filters are configured, the device omits `postFilterPayload` entirely:

Response — no filters configured

```
1  {
2    "command" : "get_post_filter" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }
```

Response Semantics

- `postFilterPayload` is absent when no filters are set. With no post-filters configured, the device omits `postFilterPayload` entirely — not even an empty `postFilter: []`. Treat its absence as "no filters." When filters exist, `postFilter[]` lists them.

Response Codes

`get_post_filter` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the post-filter configuration was retrieved (an empty configuration still succeeds).
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the post-filters at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

Examples

Publish `get_post_filter` over the CTRL endpoint (with an active CTRL endpoint); the device replies with the current filters. The [Response Payload](#) shows a configured filter; the no-filter reply is shown above it. There are no request variants — the command takes no parameters.

Notes and Caveats

- **Post-filters vs pre-filters.** `get_post_filter` reports **post-filters** — pattern rules applied to scanned **tag data** before it reaches a data endpoint (`INCLUDE` / `EXCLUDE`). These are separate from the Gen2 `select` **pre-filters** in `set_operating_mode`, which filter on the air interface during inventory. Configure post-filters with `set_post_filter`.

`get_post_filter` is a read-only, idempotent getter — safe to poll. Use it to verify a change applied by `set_post_filter`; it carries no credentials in request or response. For the concept-level view, see [Tag Filtering](#).

Related:  `set_post_filter` ·  `get_operating_mode` ·  `config_endpoint` ·  [Tag Filtering](#) ·  [Response Codes](#)

set_operating_mode

REFERENCE

API consumer

configure the reader's RFID operating mode

Overview

`set_operating_mode` configures the reader's RFID operating mode — the profile, advanced RF configuration, tag access operations, radio start/stop triggers, the Gen2 `query`, select pre-filters, and tag-metadata reporting flags. It is a state-changing **control-plane (CTRL)** command. It is a **partial, merging** update: you send only the sub-objects you want to change and they merge into the current mode. Read the result back with `get_operating_mode`.

`accessOperations` arm destructive tag operations (`WRITE` / `LOCK` / `KILL`) that execute on the next inventory — `KILL` and `PERMANENT_LOCK` are irreversible. See the danger callout in [Notes and Caveats](#) before sending.

This is a **CTRL-endpoint command**, published on the **control** endpoint (`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). This **requires an active CTRL endpoint** (`config_endpoint`, `epType: CTRL`, `activate: true`); without one, you get no response.

Set the [regulatory region](#) first. Before you start an RFID inventory, make sure the reader's regulatory region is configured: the reader does not transmit until it is, and the region is set outside MQTT with 123RFID Desktop or 123RFID Mobile. Use `get_current_region` to verify the current region.

Details

Property	Value
Interaction Pattern	Request / Response (partial-merge configuration)
State Change	STATE-CHANGING (arms access ops; changes RF config)
Reversibility	REVERSIBLE / VOLATILE (reset to default on <code>reboot</code> , or re-send to change)
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>operatingMode.operatingModes</code> is optional in the contract; send it with the aspects to set)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_operating_mode</code> , <code>control_operation</code> , <code>config_endpoint</code> , <code>reboot</code>

When to Use

Use `set_operating_mode` to configure how the reader reads (and acts on) tags:

- **Select a profile** — `CYCLE_COUNT` (read as many unique tags as possible), `DENSE_READERS` (multiple readers in proximity), `OPTIMAL_BATTERY` (best battery life), `BALANCED_PERFORMANCE` (default; balances performance and battery life), or `ADVANCED` (custom link-profile settings via `advancedConfigurations`). `FAST_READ` (maximum read speed within short range) is enumerated but not currently supported.
- **Tune the radio** — transmit power, link profile, session, dynamic power (`ADVANCED`).
- **Compose triggers** — `radioStartConditions` / `radioStopConditions` (trigger, delay, tag/time/cycle stops).
- **Shape inventory** — Gen2 `query` and up to 32 `select` pre-filters.
- **Choose reporting** — `tagMetaDataToEnable` flags.
- **Arm tag operations** — `accessOperations` (`READ`/`WRITE`/`ACCESS`/`LOCK`/`KILL`) that run on the next inventory. Handle with care (see the danger note).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint**.

If control commands time out, confirm a CTRL endpoint is active (`get_endpoint_config` → `activeEndpoints`) and activate one with `config_endpoint` if needed. The mode cannot be

changed while an inventory is running ([code 11](#)).

See [Topics and Channels](#).

Key Fields

- `operatingModes.profiles` (request) — the RF profile; `ADVANCED` requires `advancedConfigurations`. `FAST_READ` is the first member of the enum but is not currently supported, and data events are not supported while the sled is in it.
- `operatingModes.accessOperations` (request) — ⚠ armed tag operations (destructive; see the danger note).
- `operatingModes.select` (request) — up to 32 Gen2 pre-filters.
- `operatingModes.query` / `radioStartConditions` / `radioStopConditions` (request) — inventory shaping and trigger semantics.
- `operatingModes.tagMetadataToEnable` (request) — which metadata fields the reader reports on each read. The switch-to-field mapping is on [dataEVT](#).
- `response.code` (response) — read this to determine success vs failure; the contract requires it; note the reported success-path omission (see [Response Payload](#)) and the partial-apply-on-error behavior (see [Request Semantics](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `operatingMode` payload. Send only the `operatingModes` sub-objects you want to change. For a basic inventory, start with **Profile setting**: it sets only the RF profile. **Set operating mode** also sets advanced RF settings and arms tag access operations (`READ`, `WRITE`, `ACCESS`, `LOCK`), which run on the next inventory `START`; send it only when you want those operations. Pick a **configuration section** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Configuration section

Profile setting ▼

Request payload

Example: basic inventory: profile only

```
1  {
2    "command" : "set_operating_mode" ,
3    "requestId" : "1286" ,
4    "operatingMode" : {
5      "operatingModes" : {
6        "profiles" : "BALANCED_PERFORMANCE"
7      }
8    }
9  }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAM
1	command *	enum	set_operating_mode	set
2	requestId *	string	—	128
3	operatingMo de	object	—	—
4	operating Modes	object	—	—
5	profil es	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	ADV
6	advanc edConf igurat ions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
7	transmitPower	number	0-300	270
8	linkProfile	enum	M4_256K M2_240K M2_256K M2_320K M4_240K M4_320K FM0_0K FM0_320K M8_240K M8_256K M8_320K	M2_
9	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
10	dynamicPower	boolean	—	true
11	inventoryProtocol	enum	GEN2 GEN2X HYBRID	GEN
12	accessOperations	array	—	—
13	operationType	enum	READ WRITE ACCESS LOCK KILL	REA
14	config	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
15	m e m o r y B a n k	enum	EPC TID USER RESERVED	EPC
16	o f f s e t	integer	—	1
17	l e n g t h	integer	—	2
18	d a t a	string	—	FFF
19	p a s s w o r d	string	—	000

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
20	lockMemBank	enum	EPC ACCESS_PWD KILL_PWD TID USER ALL	TID
21	lockActivation	enum	READ_AND_WRITE PERMANENT_LOCK PERMANENT_UNLOCK UNLOCK	UNLOCK
22	radioStartConditions	object	—	—
23	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEDIATE
24	startDelay	integer	—	300
25	repeat	boolean	—	false
26	radioStopConditions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
27	tri gge r	enum	PRESSED RELEASED IMMEDIATE	IMM
28	tag Cou nt	integer	—	10
29	sto pTi meo ut	integer	—	100
30	inv ent ory Cou nt	integer	—	0
31	query	object	—	—
32	ses sio n	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
33	inv ent ory Sta te	enum	STATE_A STATE_B STATE_AB	STA
34	slF lag	enum	ALL ASSERTED DEASSERTED	ALL
35	tag Pop ula tio n	integer	min 0	30
36	select	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
37	enable *	boolean	—	true
38	tagParameter *	string	—	222
39	memoryBank *	enum	EPC TID USER	EPC
40	offset *	integer	—	32
41	action *	enum	INV_A_NOT_INV_B_OR_ASRT_SL_NOT_DSRT_SL INV_A_OR_ASRT_SL NOT_INV_B_OR_NOT_DSRT_SL INV_A2BB2A_NOT_INV_A_OR_NEG_SL_NOT_ASRT_SL INV_B_NOT_INV_A_OR_DSRT_SL_NOT_ASRT_SL INV_B_OR_DSRT_SL NOT_INV_A_OR_NOT_ASRT_SL NOT_INV_A2BB2A_OR_NOT_NEG_SL	INV
42	tagSetting *	enum	SESSION_SL SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
43	length *	integer	—	16
44	tagMetadataObjectEnable	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
45	RSSI	boolean	—	false
46	PHASE	boolean	—	false
47	SEENCOUNT	boolean	—	false
48	CHANNEL	boolean	—	false
49	PC	boolean	—	false
50	XPC	boolean	—	true
51	CRC	boolean	—	true
52	EPC	boolean	—	true
53	TID	boolean	—	true
54	USER	boolean	—	true
55	MAC	boolean	—	true
56	HOSTNAME	boolean	—	true

Request Semantics

Merge. `set_operating_mode` is **cumulative**: it updates only the aspects present in the payload and leaves all others intact — it is **not** a full replacement. For example, sending only `radioStartConditions` changes the triggers while preserving `profiles`, `query`, `select`, `accessOperations`, and `tagMetadataToEnable`. Send `{profiles: "BALANCED_PERFORMANCE"}` alone to reset the profile without touching anything else. Because merges accumulate, **armed** `accessOperations` **persist** across later partial sets until you overwrite them or reboot. Confirm the resulting merged mode with `get_operating_mode`.

- The singular `operatingMode` wrapper around the plural `operatingModes` object is the schema-defined shape. The command's `operatingMode` property carries a wrapper holding exactly one member, `operatingModes`, and every worked example nests it that way — the double nesting is intentional and unique to this command (see the Glossary). The contract marks neither level `required` and does not forbid unrecognized properties, so a flattened or renamed payload still validates — but it does not carry the operating mode where the schema defines it. Send the nested shape exactly as shown.
- An error response can leave the mode partially applied. An error response (e.g. `101` "Error in processing command") can leave the mode **partially applied** — always verify the result with `get_operating_mode` rather than assuming a `101` changed nothing.
- **ADVANCED** requires `advancedConfigurations`; **limits are enforced**. `profiles: ADVANCED` without `advancedConfigurations` returns `code 22`. `select` supports up to 32 pre-filters (`code 24` if exceeded); an overlong `select.tagPattern` returns `28`; WRITE `data` that is too large or mismatched returns `31` / `32`. `FAST_READ` is enumerated but **not currently supported**, and configuring `FAST_READ` disables data events.

Response Payload

`set_operating_mode` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`.

Response payload

```
1  {
2    "command" : "set_operating_mode" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command * mand d	string	—	set_operating_mode	Echoes the executed command.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — set_operating_mode echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.
6	description *	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful set can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```
1  {
2    "command" : "set_operating_mode" ,
3    "requestId" : "1286" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`set_operating_mode` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the mode was applied. (Readers have been reported to omit <code>code</code> on this reply — see above.)
11	Inventory in progress (<code>IOT_ERROR_INVENTORY_IN_PROGRESS</code>) — the mode cannot be changed while an inventory is running. Stop it with control_operation first.
22	Advanced configuration not set (<code>IOT_CTRL_ADVANCED_PROFILE_NOT_SET</code>) — <code>profiles:ADVANCED</code> was sent without <code>advancedConfigurations</code> .
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>profiles</code> , <code>operationType</code> , a Gen2 enum) is outside its allowed set.
24	Max 32 prefilters limit exceeded (<code>IOT_ERROR_PREFILTERS_LIMIT_EXCEEDED</code>) — more than 32 <code>select</code> entries.
28	Tag match pattern length exceeded (<code>IOT_ERROR_TAG_MATCH_PATTERN_LENGTH_EXCEEDED</code>) — a <code>select.tagPattern</code> is too long.
31	Max write length exceeded (<code>IOT_ERROR_MAX_WRITE_LENGTH_EXCEEDED</code>) — a WRITE <code>data</code> block is too large.
32	Write data length mismatch (<code>IOT_ERROR_WRITE_DATA_LENGTH_MISMATCH</code>) — a WRITE <code>data</code> length does not match its <code>length</code> .

Code	Description
101	Default error (<code>Error in processing command</code>) — a generic failure. Note: a 101 can still leave the mode <i>partially applied</i> — read back with <code>get_operating_mode</code> to see what took effect.

Examples

Publish `set_operating_mode` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). Every request example in the contract is in the **Configuration section** dropdown under [Request Payload](#): the full `ADVANCED` configuration (**Set operating mode**, which arms READ, WRITE, ACCESS and LOCK operations with the default password `00000000`), and one partial set per section. **Profile setting** is also the way to return the profile to its default.

A generic failure returns [code](#) `101` (which may still partially apply — read back to confirm):

Response — error (code 101)

```

1      {
2          "command" : "set_operating_mode" ,
3          "requestId" : "1286" ,
4          "apiVersion" : "V1.1" ,
5          "response" : {
6              "code" : 101 ,
7              "description" : "Error in processing command"
8          }
9      }
```

Notes and Caveats

⚠️ ARMED ACCESS OPERATIONS EXECUTE ON THE NEXT INVENTORY — AND THE PAYLOAD CARRIES THEIR PASSWORD

`accessOperations` (`WRITE` , `LOCK` , `KILL`) are stored by `set_operating_mode` and executed against tags **on the next inventory START** — not immediately. `KILL` permanently disables a tag; `LOCK` with `PERMANENT_LOCK` / `PERMANENT_UNLOCK` is irreversible; `WRITE` overwrites tag memory. Because partial sets **merge**, armed operations persist until you **overwrite the** `accessOperations` **array** (with a benign or empty set) or `reboot` . A profile-only set — or any partial set that omits `accessOperations` — does **not** clear them. Do this before running an inventory that should not trigger them. Each armed operation also carries `accessOperations[].config.password` , an 8-hex-char (32-bit) access/lock/kill password you send (default `00000000`) — mask it in shared artifacts. The device stores it but never echoes it back (`get_operating_mode` returns it empty). This command carries no broker/Wi-Fi credentials.

- **Volatile and CTRL-only.** The operating mode resets to default on `reboot` , and the command is answered only on the CTRL endpoint (see [Topic Mapping](#)).
- `FAST_READ` **is enumerated but not currently supported.** `FAST_READ` is the first member of the `profiles` enum, and the schema glosses it as "Configures the radio for maximum read speed within short range. (Currently not supported)". The command description repeats the restriction and extends it to the event plane: "When RFD40/90 sled is configured to FAST_READ profile, data events are not currently supported. On reboot the set configurations will be lost and the device will go back to default operating mode." Treat `FAST_READ` as unavailable, and expect no `dataEVT` traffic while the sled is in that profile.

Configure with `set_operating_mode` , then **verify with** `get_operating_mode` (especially after a `101`), and run the inventory with `control_operation` . For the concept-level view, see [Operating Mode](#).

Related:  `get_operating_mode` ·  `control_operation` ·  `config_endpoint` ·  `reboot` ·  [Response Codes](#)

set_post_filter

REFERENCE

API consumer

add, delete, or modify a post-filter rule on the reader

Overview

`set_post_filter` **configures a post-filter** — a rule that includes or excludes scanned RFID tag data before it is reported to a data endpoint. Each call operates on **one** filter, chosen by `postFilterPayload.operation` (`ADD` / `DELETE` / `MODIFY`): the rule matches tag data by pattern (prefix, suffix, or regex) and either reports (`INCLUDE`) or drops (`EXCLUDE`) it, scoped to a data endpoint. It is a state-changing **control-plane (CTRL)** command; read the current filters back with `get_post_filter`.

This is a **CTRL-endpoint command**, published on the **control** endpoint

(`<tenantId>/CTRL/clients/cmd/<serial>`) and answered only there (not on the

[MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint**

(`config_endpoint`, `epType: CTRL`, `activate: true`). Also: `dataEpType` is the **enum**

`DATA_EP1` / `DATA_EP2`, **not** the endpoint *name* — sending an endpoint name returns [code 23](#)

(see [Request Semantics](#)).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (adds/removes a filter)
Reversibility	REVERSIBLE (an <code>ADD</code> is undone by a <code>DELETE</code> on the same <code>dataEpType</code>)
Supported Operations	<code>ADD</code> (new), <code>DELETE</code> (existing), <code>MODIFY</code> (existing)
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>postFilterPayload</code> is optional in the contract; send it with the rule)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)

Property	Value
Related	<code>get_post_filter</code> , <code>set_operating_mode</code> , <code>config_endpoint</code>

When to Use

Use `set_post_filter` to control which scanned tag data reaches a data endpoint:

- **Include only matching tags** — `reportOperation: INCLUDE` reports only tags whose data matches; everything else is dropped.
- **Exclude noisy tags** — `reportOperation: EXCLUDE` drops matching tags from reporting.
- **Match by prefix, suffix, or regex** — choose `matchPatternMethod` to fit the tag-data pattern.
- **Manage the rule set** — `ADD` to install, `DELETE` to remove, `MODIFY` to update; verify with `get_post_filter`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint**.

Post-filters act on **tag data** (post-singulation), distinct from the Gen2 `select` pre-filters in `set_operating_mode` that act on the air interface. If control commands time out, confirm a CTRL endpoint is active and activate one with `config_endpoint` if needed.

See [Topics and Channels](#).

Key Fields

- `postFilterPayload.operation` (request) — `ADD` / `DELETE` / `MODIFY`; determines the action on the rule set.
- `dataEpType` (request) — the data-endpoint enum (`DATA_EP1` / `DATA_EP2`) the filter binds to — ⚠ not the endpoint name (see [Request Semantics](#)).
- `matchPattern` + `matchPatternMethod` (request) — what to match and how (`PREFIX` / `SUFFIX` / `REGEX`).
- `reportOperation` (request) — `INCLUDE` (report matches) or `EXCLUDE` (drop matches).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the single-rule `postFilterPayload`. Pick a **match method** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Post filter scenario

Filter tags using prefix



Request payload

Example: ADD, prefix

```
1  {
2    "command" : "set_post_filter" ,
3    "requestId" : "abc123" ,
4    "postFilterPayload" : {
5      "operation" : "ADD" ,
6      "dataEpType" : "DATA_EP1" ,
7      "matchPattern" : "FFFFBBBBBA500" ,
8      "matchPatternMethod" : "PREFIX" ,
9      "reportOperation" : "INCLUDE"
10   }
11 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	co mm an d *	string	—	set_post_filter	Fixed command name.
2	re qu es tI d *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	post Filt erPa yloa d	object	—	—	The single post-filter rule to add, delete, or modify. Optional in the contract, as is every field inside it, but a request without it carries no rule to apply.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	operation	enum	ADD DELETE MODIFY	ADD	ADD a new filter, DELETE an existing one, or MODIFY one. See Request Semantics .
5	dataEndpoint	enum	DATA_EP1 DATA_EP2	DATA_EP1	The data-endpoint TYPE the filter binds to. ⚠ This is the enum value DATA_EP1/DATA_EP2, not the endpoint name (see Request Semantics) — a wrong value returns code 23 . DATA_EP2 targets the DATA2 endpoint, which is currently not supported.
6	matchPattern	string	—	FFFFBBBBBA500	The value matched against scanned tag data. For PREFIX/SUFFIX: hex digits only, even count. For REGEX: a regular expression. See Request Semantics.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
7	m a t c h P a t t e r n M e t h o d	enum	PREFIX SUFFIX REGEX	REGEX	How the pattern is matched — beginning (PREFIX), end (SUFFIX), or regular expression (REGEX).
8	r e p o r t O p e r a t i o n	enum	INCLUDE EXCLUDE	INCLUDE	INCLUDE reports only matching tag data; EXCLUDE drops matching tag data from reporting.

Request Semantics

Modes. `postFilterPayload.operation` selects the action; each call handles one rule:

- `ADD` — install a new post-filter for the given `dataEpType`.
- `DELETE` — remove the post filter on a data endpoint. Send `operation: DELETE` with `dataEpType` alone; `matchPattern`, `matchPatternMethod` and `reportOperation` are not needed. Undoes a prior `ADD` on the same `dataEpType`.
- `MODIFY` — update an existing filter.

Confirm the resulting rule set with `get_post_filter`, which returns the stored `postFilter[]` (or nothing when none remain).

- `dataEpType` **is an enum, not the endpoint name.** `dataEpType` must be the enum value `DATA_EP1` or `DATA_EP2` — **not** the endpoint *name* you gave a data endpoint in `config_endpoint` (e.g. `DATA1_EP`). Sending the endpoint name is an enum violation and the device rejects it with [code 23](#) ("Invalid enum value"). Map your `DATA1` endpoint to `DATA_EP1` when setting a filter; `DATA_EP2` targets the `DATA2` endpoint, which is currently not supported.
- `matchPattern` **rules follow the match method.** For `PREFIX` and `SUFFIX` methods, `matchPattern` must be **hexadecimal digits only, with an even count** — the contract states this rule in the field description but does not enforce it with a pattern; for `REGEX` it is a free-form regular expression.
- `PREFIX` **post-filters and Gen2 `select` pre-filters are mutually exclusive.** A `PREFIX` post-filter precludes using Gen2 `select` pre-filters at the same time.

Response Payload

`set_post_filter` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. Confirm with `get_post_filter` — there is no filter echo.

Response payload

```

1  {
2      "command" : "set_post_filter" ,
3      "requestId" : "abc123" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 0 ,
7          "description" : "Success"
8      }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	set_post_filter	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — set_post_filter echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	d e s c r i p t i o n	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful set or delete can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```

1  {
2    "command" : "set_post_filter" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`set_post_filter` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the filter was added, deleted, or modified. (Readers have been reported to omit <code>code</code> on this reply — see above.)
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (<code>operation</code> , <code>dataEpType</code> , <code>matchPatternMethod</code> , Or <code>reportOperation</code>) is outside its allowed set. Commonly triggered by sending an endpoint <i>name</i> (e.g. <code>DATA1_EP</code>) as <code>dataEpType</code> instead of the enum <code>DATA_EP1</code> — see Request Semantics.
101	Default error — generic catch-all.

Examples

Publish `set_post_filter` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). The **Post filter scenario** dropdown under [Request Payload](#) shows each contract example: an `ADD` by prefix, regex and suffix, and a `DELETE`. In the regex example, `matchPattern` is free-form; the hex-digit rule applies only to prefix and suffix patterns.

Sending an endpoint **name** (not the enum) as `dataEpType` is rejected with [code](#) `23`:

Response — invalid enum value (code 23)

```

1  {
2      "command" : "set_post_filter" ,
3      "requestId" : "abc123" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 23 ,
7          "description" : "Invalid enum value"
8      }
9  }
```

Notes and Caveats

- Post-filters vs pre-filters.** `set_post_filter` configures **post-filters** — rules applied to scanned **tag data** before it reaches a data endpoint (`INCLUDE` / `EXCLUDE`). These are separate from the Gen2 `select` **pre-filters** in `set_operating_mode` , which filter on the air interface during inventory.

`set_post_filter` is **reversible** — an `ADD` is undone by a `DELETE` on the same `dataEpType`. It carries no credentials (match patterns are tag-data hex/regex strings). After setting a filter, **verify with** `get_post_filter` — the setter's response has no configuration echo. For the concept-level view, see [Tag Filtering](#).

Related:  `get_post_filter` ·  `set_operating_mode` ·  `config_endpoint` ·  [Tag Filtering](#) ·  [Response Codes](#)

alerts

Overview

`alerts` is a **device-emitted** event that delivers structured, categorized alert information — the reader publishes it **autonomously** when a monitored condition changes state or crosses a threshold; **no command triggers it**. Each event is a fixed **envelope** (`type` , `timestamp` , `state` , `id` , `priority`) plus a category-specific `alertDetails` body selected by `id` . It is read-only telemetry and carries no credentials.

The same envelope carries two message classes, distinguished by `type` :

- `ALERT` — an autonomous monitoring alert (battery, power, network).
- `NOTIFICATION` — a command side-effect: `FILE_DOWNLOAD` (see `install_certificate`) and the `FIRMWARE_UPDATE` start (see `set_os`).

This page documents the full envelope and **all seven** `alertDetails` **branch keys**. `config_events` toggles (`battery` , `power` , `network` , `firmwareUpdate` , `fileDownload`) govern which alerts publish.

The `id` enum has five categories, all emitted on current V1.1 firmware: `BATTERY` , `POWER` , `NETWORK_EVENT` , `FIRMWARE_UPDATE` , and `FILE_DOWNLOAD` . The schema also declares a `temperatureInfo` branch, but there is no `TEMPERATURE` id and no temperature alert is emitted on current firmware. See [Events](#).

Details

Property	Value
Event Type	Alert / notification (structured, categorized)
Trigger	State change / threshold (autonomous, on a monitored condition change or threshold crossing)
Enabled By	<code>config_events</code> with per-category <code>eventConfiguration</code> flags (<code>battery</code> , <code>power</code> , <code>network</code> , <code>firmwareUpdate</code> , <code>fileDownload</code>), applied on <code>reboot</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)

Property	Value
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>heartBeatEVT</code> , <code>dataEVT</code> , <code>mqttConnEVT</code>

When to Use

Subscribe to `alerts` when you want the reader to **tell you** about a monitored condition rather than asking for it — there is **no command that returns this data on demand** (no polling twin). Reach for it when you need to:

- **React to device-health changes as they happen** — battery state and health (`BATTERY`), power-source and power-mode transitions (`POWER`), and network interface connect/disconnect (`NETWORK_EVENT`).
- **Track long-running command side-effects** — the `FIRMWARE_UPDATE` stream during a `set_os` flash (a `type: NOTIFICATION` start, then in-progress notifications) and the `FILE_DOWNLOAD` result during an `install_certificate` transfer (`type: NOTIFICATION`), all on this same envelope.
- **Triage by severity** — the `priority` field (`CRITICAL` ... `LOW`) lets you route alerts without inspecting each branch body.

Enable only the categories you consume with `config_events` — a disabled category never publishes, so leaving one off is how you silence it. For a periodic health snapshot rather than change-driven alerts, use `heartBeatEVT`.

Topic Mapping

`alerts` is device-emitted and flows on the **management event topic** — it has no `cmd` / `resp` topics because no command produces it.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	The same event topic that carries <code>heartBeatEVT</code> and <code>NOTIFICATION</code> -type events.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the

reader's serial number, which the reader appends itself in its model-prefixed form (for example RFD40-212735201D0053). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `type` (event) — message class: `ALERT` (autonomous monitoring alert), `NOTIFICATION` (command side-effect), or the reserved `EVENT`. Route on this to separate alerts from notifications.
- `timestamp` (event) — ISO-8601 (`format: date-time`) instant the alert was generated.
- `state` (event) — `SET` (condition active/asserted), `CLEAR` (resolved), or `ONESHOT` (one-time, no matching `CLEAR`). Pair with `id` to track a condition's lifecycle.
- `id` (event) — the alert category (`BATTERY` , `POWER` , `NETWORK_EVENT` , `FIRMWARE_UPDATE` , `FILE_DOWNLOAD`); selects which `alertDetails` branch is populated.
- `priority` (event) — severity, `CRITICAL` / `HIGH` / `LOW` , for triage without inspecting the branch body.
- `alertDetails` (event) — the category-specific body (seven declared branch keys); exactly one branch is populated, matching `id` . Optional — not in the envelope `required` set.
- `config_events` **category toggles** (request) — the emission gate: the per-category `eventConfiguration` flags (`battery` , `power` , `network` , `firmwareUpdate` , `fileDownload`) set with `config_events` govern whether each `id` publishes at all. A disabled category is silent.

Event Payload

The envelope (`type` , `timestamp` , `state` , `id` , `priority`) is always present and **required**; `alertDetails` is optional and carries exactly one branch matching `id` . Pick an **alert category** below — the Example and Schema update together to show that category's `alertDetails` branch. Use the **Schema** tab for the field reference and the **Example** tab for the live payload.

Alert category

Battery alert



Event payload

Example: BATTERY (live)

```
1  {
2    "type": "ALERT",
3    "timestamp": "2026-04-29T12:33:34.279Z",
4    "state": "ONESHOT",
5    "id": "BATTERY",
6    "priority": "LOW",
7    "alertDetails": {
8      "batteryAlert": {
9        "status": "CHARGING",
10       "stateOfHealth": "GOOD",
11       "chargePercentage": 100
12     }
13   }
14 }
```

Event payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	type *	enum	EVENT NOTIFICATION ALERT	ALERT
2	timestamp *	string	date-time	2026-06-17T10:16:34.793Z
3	state *	enum	SET CLEAR ONESHOT	ONESHOT
4	id *	enum	BATTERY FILE_DOWNLOAD FIRMWARE_UPDATE NETWORK_EVENT POWER	BATTERY
5	priority *	enum	CRITICAL HIGH LOW	CRITICAL

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
6	alertDetails	object	—	—
7	fwUpdateStatus	object	—	—
8	updateStatus *	enum	Starting FW update FW update success updating failed skipping	FW update success
9	overallProgress	number	—	20
10	stage	string	—	updating scanner fw
11	downloadInfo	object	—	—
12	fileType *	enum	FILE_TYPE_FW FILE_TYPE_CERT	FILE_TYPE_FW
13	fileName *	string	—	mqtt_ca_cert
14	status *	enum	DOWNLOAD SUCCESS DOWNLOAD FAIL SAVE FAIL SAVE SUCCESS	DOWNLOAD SUCCESS

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
15	temperatureInfo	object	—	—
16	temperature *	number	—	—
17	pressure *	number	—	—
18	ambient *	number	—	—
19	networkInfo	object	—	—
20	networkInterface *	object	—	—
21	ethStatus	array	—	—
22	interface *	string	—	eth0
23	status *	enum	CONNECTED DISCONNECTED	CONNECTED

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
24	linkStatus	enum	UP DOWN	UP
25	linkSpeed	string	—	—
26	ipv4Address	string	—	192.168.0.165
27	ipv6Address	string	—	fe80::f299:9d51:864c:cff
28	wifiStatus	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
29	interface	string	—	wlan0
30	status	enum	CONNECTED DISCONNECTED	CONNECTED
31	ssid	string	—	IOT_TESTAP
32	ipV4Address	string	—	192.168.0.145
33	ipV6Address	string	—	fe80::f299:9d51:86c:cff14

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
34	btRemot eDevice	object	—	—
35	dev ice Nam e *	string	—	—
36	dev ice * Mac	string	—	00:11:22:A3:B1:D2
37	ntp	object	—	—
38	offset	number	—	—
39	reach	number	—	—
40	powerEvent	object	—	—
41	powerS ource *	enum	BATTERY USB CRADLE WALL_CHARGER	BATTERY
42	lldp	enum	success failed negotiating	—
43	powerM ode *	enum	ACTIVE LOWPOWER	LOWPOWER
44	batteryAle rt	object	—	—
45	status *	enum	LOW CRITICAL CHARGING FULL HIGH	LOW

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
46	stateOfHealth *	enum	GOOD CRITICAL MODERATE	GOOD
47	chargePercentage *	integer	max 100	100

`state` tells you whether a condition is active or resolved: `SET` = active/asserted, `CLEAR` = resolved, `ONESHOT` = a one-time event with no persistent state (it fires once and has no matching `CLEAR`). Every example on this page is `ONESHOT`, and so was every firmware-update notification recorded during one `set_os` run.

Event Semantics

Emission

The per-category `eventConfiguration` toggles you set with `config_events` govern which `id` categories publish: `battery` → `BATTERY`, `power` → `POWER`, `network` → `NETWORK_EVENT`, `firmwareUpdate` → `FIRMWARE_UPDATE`, and `fileDownload` → `FILE_DOWNLOAD`. A disabled category never publishes, so the *absence* of an alert is not proof of a healthy state. The `FIRMWARE_UPDATE` *start* and the `FILE_DOWNLOAD` result arrive as `type: NOTIFICATION` command side-effects (from `set_os` and `install_certificate`); the `BATTERY` / `POWER` / `NETWORK_EVENT` categories are autonomous `type: ALERT` events. The in-progress `FIRMWARE_UPDATE` events recorded during one `set_os` run were also `NOTIFICATION`s. The `config_events` schema has a `temperature` toggle, but there is no `TEMPERATURE` `id` and no temperature alert is emitted, so enabling it has no effect.

Branches

`alertDetails` declares seven branch keys and carries one of them; `id` selects the populated key. Two keys have no `id` value that selects them:

<code>id</code>	<code>alertDetails</code> key	Body fields	Emitted on V1.1
<code>BATTERY</code>	<code>batteryAlert</code>	<code>status</code> , <code>stateOfHealth</code> , <code>chargePercentage</code>	✓

id	alertDetails key	Body fields	Emitted on V1.1
POWER	powerEvent	powerSource , powerMode , (lldp)	✓
NETWORK_EVENT	networkInfo	networkInterface.{ethStatus[], wifiStatus[]} , btRemoteDevice	✓
FIRMWARE_UPDATE	fwUpdateStatus	updateStatus , overallProgress , stage	✓
FILE_DOWNLOAD	downloadInfo	fileType , fileName , status	✓
— (no TEMPERATURE id)	temperatureInfo	nge , pa , ambient	⚠ not emitted
— (no matching id)	ntp	offset , reach	⚠ no example

Field Support

- **type distinguishes alerts from notifications.** `type: ALERT` is an autonomous monitoring alert; `type: NOTIFICATION` is a command side-effect — `FILE_DOWNLOAD` (during `install_certificate` / firmware download) and the `FIRMWARE_UPDATE` *start* (during `set_os`). `EVENT` is in the enum but was not observed. All three share this one envelope.
- **FILE_DOWNLOAD fileName is the stored basename.** `downloadInfo.fileName` is the name the reader stores internally — the URL basename **stem** (e.g. a source `.../mqtt_ca_cert.pem` yields `fileName: "mqtt_ca_cert"`), not your request name. Do not expect a 1:1 match to a `get_installed_certificates` entry name without the source URLs.
- **The interface key is interface on these items.** Each `ethStatus` / `wifiStatus` item names its interface with the key `interface` (`"interface": "eth0"`), while the Wi-Fi and Ethernet commands and their read-backs use `interfaceName` for the same concept. The key is required: an `ethStatus` item requires `interface` , `status` , `linkStatus` and `ipV4Address` , a `wifiStatus` item `interface` , `status` , `ssid` and `ipV4Address` .
- **alertDetails is optional; envelope is required.** The envelope `required` set is `[type, timestamp, state, id, priority]`; `alertDetails` is not required. Validate the envelope strictly.
- **batteryAlert.stateOfHealth is the same health grade as get_status.** The value set here is `GOOD` / `CRITICAL` / `MODERATE` , and `get_status` reports `deviceStatus.batteryStatus.stateOfHealth` from that same enum. One grade, so nothing needs translating when you share battery logic between the command and the event stream. The battery's charge state is the separate `status` field. See [get_status](#).

Examples

Every contract example is in the **Alert category** dropdown under [Event Payload](#), and all of them are the reader's own live captures. For the `FIRMWARE_UPDATE` category, the dropdown shows the `NOTIFICATION` that starts an update (from `set_os`); the in-progress notifications that follow are under `set_os` [Progress](#). The two `FILE_DOWNLOAD` options come from an `install_certificate` transfer: one reports the download, the other the save.

Notes and Caveats

- **Validate against the long-form alert, not the [SOTI](#) short form.** Two alert shapes exist: the **long-form** envelope on this page (the one the reader publishes on the management event topic) and the **short-form (SOTI)** projection, a flat shape with granular ids like `BATTERY_LOW_SET` that carries neither `state` nor `alertDetails` and is reachable only on the `SOTICONNECT` endpoint — out of scope here. No `id` value is shared between the two vocabularies and no mapping between them is defined, so only the long form matches what the reader publishes here.
- **No response envelope.** `alerts` is a device-emitted event, so it carries **no** `response` / `code` — the command response-code table does not apply, and no response code is defined for any event.

`alerts` is **read-only telemetry** with no credentials. Remember emission is **gated by** `config_events` (see [Event Semantics](#)). For the concept-level view, see [Alerts](#) and [Event Management](#).

Related:  `config_events` ·  `set_os` ·  `install_certificate` ·  `heartBeatEVT` ·  [Events](#) ·  [Alerts](#) ·  [Events](#)

heartBeatEVT

Overview

`heartBeatEVT` is a **device-emitted** event — the reader publishes it on its own, with **no command triggering it**, to signal that it is alive and to report basic health: uptime, an event sequence number, and optional **inventory** and **battery** sub-payloads. It is read-only telemetry and carries no credentials.

You **enable** a heartbeat rather than requesting it. The reader publishes `heartBeatEVT` at the interval you set in `heartbeatConfiguration` once the heartbeat is enabled with `config_events` (see [Event Semantics](#)).

The reader publishes `heartBeatEVT` at the configured interval (at most 65,535 seconds) while the heartbeat is enabled with `config_events`; a reboot applies the setting. Monitoring can rely on it as its primary liveness signal: when `heartBeatEVT` stops arriving, the reader, its session, or the path to your application needs attention. `heartBeatEVT` is published at [QoS 0](#), so the MQTT layer does not resend one lost in transit. Choose how many missed intervals your application treats as an outage.

Details

Property	Value
Event Type	Heartbeat (periodic liveness / health)
Trigger	Timed interval (every <code>heartbeatConfiguration.interval</code> s while enabled)
Enabled By	<code>config_events</code> with <code>eventConfiguration.heartbeat: true</code> (plus <code>heartbeatConfiguration</code>), applied on <code>reboot</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	dataEVT , alerts , mqttConnEVT

When to Use

Subscribe to `heartBeatEVT` when you want **continuous, pushed telemetry of reader liveness and health** — uptime, an event sequence number, and optional inventory and battery status — rather than polling for it on demand. It is the asynchronous twin of `get_status` (`heartBeatEVT` ↔ `get_status`): the heartbeat *pushes* health at a fixed interval, while `get_status` *pulls* a correlated snapshot when you ask.

- **Receive:** periodic `heartBeatEVT` payloads on the management event topic once the heartbeat is enabled and the reader has rebooted.
- **Use it for:** liveness monitoring (the *absence* of an expected heartbeat is itself a signal), tracking RFID inventory progression, and routine battery monitoring — without issuing a command per check.
- **Prefer a different call when:** you need an immediate, on-demand reading — call `get_status` instead. It returns a synchronous device-health and battery snapshot in a correlated response (note it does not include the heartbeat's inventory scan counts or uptime).

Topic Mapping

`heartBeatEVT` is device-emitted and flows on the **management event topic** — it has no `cmd` / `resp` topics of its own because no command produces it.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	Published on the management endpoint's event topic.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The event arrives on the same management-event plane you enabled it on with `config_events`.

See [Topics and Channels](#).

Key Fields

- `eventName` (event) — names the event; the fixed value `heartbeat`. Required.
- `timestamp` (event) — ISO-8601 time the heartbeat was generated. Required.
- `eventNumber` (event) — a unique identifier or sequence number for the event. The schema does not guarantee it is monotonic or contiguous, so a gap between consecutive heartbeats is not necessarily a missed heartbeat. Required.
- `data.inventoryStatus` (event) — RFID scan state (`rfidStatus`), plus `tagCount` and `scanCount`. Present when `heartbeatConfiguration.inventoryStatus` is `true`.
- `data.batteryAlert` (event) — battery `status`, `stateOfHealth`, and `chargePercentage`. Present when `heartbeatConfiguration.batteryStatus` is `true`.
- `heartbeat` (request) — the gate flag in `config_events`; the heartbeat publishes only once `heartbeat: true` and `heartbeatConfiguration.interval` is set (at most 65,535 seconds) (see [Event Semantics](#)).

Event Payload

The top-level shape is `eventName`, `timestamp`, `eventNumber` (all **required**), an optional `upTime`, and an optional `data` object carrying the inventory and/or battery sub-payloads. Use the **Schema** tab for the field reference and the **Example** tab for a full heartbeat.

Event payload

Example: inventory + battery

```
1  {
2    "eventName": "heartbeat",
3    "timestamp": "2019-08-24T14:15:22Z",
4    "eventNumber": 120,
5    "data": {
6      "inventoryStatus": {
7        "rfidStatus": "INPROGRESS",
8        "tagCount": 45,
9        "scanCount": 128
10     },
11     "batteryAlert": {
12       "status": "HIGH",
13       "stateOfHealth": "GOOD",
14       "chargePercentage": 85
15     }
16   }
17 }
```

Event payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	eventName *	string	—	heartbeat	Names the event — the fixed value heartbeat for this event.
2	timestamp *	string	—	2019-08-24T14:15:22Z	ISO-8601 time the event was generated.
3	eventNumber *	integer	—	120	A unique identifier or sequence number for the event. The schema does not guarantee it is monotonic or contiguous per heartbeat.
4	uptime	string	—	5 days 4hr 3min	Device uptime when the event was generated. Free-form human-readable string (not a duration type).
5	data	object	—	—	Optional health payload. The schema declares it as anyOf two flat shapes — the inventory fields or the battery fields below — while the contract's example nests them as data.inventoryStatus and data.batteryAlert (see Field Support). Which appear is gated by heartbeatConfiguration (see Event Semantics).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	r f i d S t a t u s	enum	INPROGRESS STOPPED	—	The current status of the RFID scanning process.
7	t a g C o u n t	integer	—	—	The total number of RFID tags scanned in the current operation.
8	s c a n C o u n t	integer	—	—	The total number of scan attempts made during the operation.
9	s t a t u s	enum	LOW CRITICAL CHARGING FULL HIGH	LOW	The current status of the battery, which indicates its operational state.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
10	status	enum	GOOD CRITICAL MODERATE	—	The overall health of the battery, which reflects its long-term capacity and performance.
11	chargePercentage	integer	max 100	—	The current charge level of the battery, represented as a percentage.

In the examples, `data` nests its sub-payloads, and a heartbeat may carry **one, both, or neither** of `inventoryStatus` and `batteryAlert` (the schema declares a flatter shape — see [Field Support](#)). The `heartbeatConfiguration` inclusion flags you set when enabling the heartbeat control which ones appear:

- `inventoryStatus` — RFID scan state (`rfidStatus`), plus `tagCount` and `scanCount` . Present when `heartbeatConfiguration.inventoryStatus` is `true` .

- `batteryAlert` — `battery` `status`, `stateOfHealth`, and `chargePercentage`. Present when `heartbeatConfiguration.batteryStatus` is `true`.

Event Semantics

Emission

Enable the heartbeat with `config_events` under `eventConfiguration`: set `heartbeat` to `true` and `heartbeatConfiguration.interval` to a value of at most 65,535 seconds. The contract documents no default for either. After the reader has rebooted, it publishes `heartBeatEVT` every `heartbeatConfiguration.interval` seconds on the management event topic.

Field	Type	Meaning
<code>heartbeat</code>	boolean	Set <code>true</code> to enable the heartbeat stream.
<code>heartbeatConfiguration.interval</code>	integer (seconds)	Seconds between heartbeats. Lower = more telemetry, higher battery cost.

Enable the heartbeat via config_events

```
1  {
2    "command" : "config_events",
3    "requestId" : "abc123",
4    "eventConfiguration" : {
5      "terminalConnection" : true,
6      "firmwareUpdate" : true,
7      "network" : true,
8      "heartbeat" : true,
9      "power" : true,
10     "battery" : true,
11     "temperature" : true,
12     "fileDownload" : true,
13     "heartbeatConfiguration" : {
14       "interval" : 100,
15       "inventoryStatus" : true,
16       "batteryStatus" : true
17     }
18   }
19 }
```

`config_events` stores the heartbeat configuration immediately (confirm it with `get_endpoint_config`), but the reader begins emitting only after a `reboot`. Also note `heartbeatConfiguration` appears in a `get_endpoint_config` read only after `config_events` sets it — it is absent from a default read. See `config_events` for the full enable workflow.

- **If no heartbeat arrives.** Read the endpoint configuration with `get_endpoint_config` and confirm `heartbeat: true` and an `interval` of at most 65,535. Confirm that the reader has rebooted since you sent `config_events`, and that you subscribed to the management event topic; a heartbeat published before your subscription is active is not delivered later.

Shaping

The `heartbeatConfiguration` inclusion flags shape each heartbeat's optional `data` object — they select which sub-payloads a heartbeat carries, independently of the enabling above.

Field	Type	Meaning
<code>heartbeatConfiguration.inventoryStatus</code>	boolean	Include the <code>data.inventoryStatus</code> block in each heartbeat.
<code>heartbeatConfiguration.batteryStatus</code>	boolean	Include the <code>data.batteryAlert</code> block in each heartbeat.
<code>heartbeatConfiguration.userApps</code>	boolean	Declared for a <code>data.userApps</code> block. Not supported: firmware does not implement user-application status, so no heartbeat carries it.

Because every member of `data` is optional, a heartbeat carries one, both, or neither of `inventoryStatus` and `batteryAlert`, depending on these flags. Set them alongside the enable flags via `config_events`.

Field Support

- **The schema and the example disagree on the shape of `data`.** The schema declares `data` as `anyOf` two flat objects — `rfidStatus` / `tagCount` / `scanCount`, or `status` / `stateOfHealth` / `chargePercentage` — so those fields would sit directly under `data`. The contract's own example, like every example on this page, nests them as

`data.inventoryStatus` and `data.batteryAlert`; it validates only because neither branch forbids extra keys. Parse the nested form shown here, and tolerate the flat form.

- `stateOfHealth` is not a charge band. `batteryAlert.stateOfHealth` is enumerated `GOOD` / `CRITICAL` / `MODERATE`. `FULL` is a value of `status`, not of `stateOfHealth`, so a payload carrying `stateOfHealth: "FULL"` is outside the enum. Trust the enum.
- `batteryAlert.stateOfHealth` is the same health grade as `get_status`. The value set here is `GOOD` / `CRITICAL` / `MODERATE`, and `get_status` reports `deviceStatus.batteryStatus.stateOfHealth` from that same enum. One grade, so nothing needs translating when you share battery logic between the heartbeat and the command. The battery's charge state is the separate `status` field. See [get_status](#).
- **Required fields and** `upTime`. `eventName`, `timestamp`, and `eventNumber` are required; `upTime` and `data` are optional. `upTime` is a **free-form string** (e.g. `"5 days 4hr 3min"`), not a machine-parseable duration — do not parse it as a number.

Examples

A heartbeat carrying **both** sub-objects appears in the **Example** tab of the payload above. The variants below show the minimal required-only shape and an inventory-only heartbeat.

The minimum a heartbeat must carry (the three required fields; `data` and `upTime` omitted):

heartBeatEVT — minimal (required fields only)

```
1  {
2    "eventName" : "heartbeat" ,
3    "timestamp" : "2019-08-24T14:15:22Z" ,
4    "eventNumber" : 121
5  }
```

An **inventory-only** heartbeat (`heartbeatConfiguration.inventoryStatus: true`, `batteryStatus: false`):

heartBeatEVT — inventory status only

```
1  {
2    "eventName" : "heartbeat" ,
3    "timestamp" : "2019-08-24T14:15:37Z" ,
4    "eventNumber" : 122 ,
5    "data" : {
6      "inventoryStatus" : {
7        "rfidStatus" : "STOPPED" ,
8        "tagCount" : 45 ,
9        "scanCount" : 128
10     }
11   }
12 }
```

Notes and Caveats

- **No response envelope.** `heartBeatEVT` is a device-emitted event, so it carries no `response` / `code` — the command response-code table does not apply. No response code is defined for any event.

`heartBeatEVT` is **read-only telemetry** and carries no credentials. Use it for liveness checks (the *absence* of expected heartbeats is itself a signal), inventory progression, and routine battery monitoring. For the concept-level view, see [MQTT Connectivity](#) and Monitor Reader Health and Battery.

Related:  [config_events](#) ·  [get_endpoint_config](#) ·  [Events](#) ·  [dataEVT](#) ·  [MQTT Connectivity](#) ·  [Events](#)

mqttConnEVT

Overview

`mqttConnEVT` is a **device-emitted** event that reports an MQTT endpoint's **connection-state transition** — `CONNECTED` or `DISCONNECTED` — along with device identity (`deviceModel` , `deviceSerialNo`) and protocol-version metadata (`apiVersion` , `mqttVersion`). The reader publishes it **autonomously** on every connection-state change; **no command triggers it and no configuration enables it**. It is read-only telemetry and carries no credentials.

It behaves like standard MQTT connection lifecycle messages: a `CONNECTED` event is a *birth* message published when an endpoint (re)connects, and a `DISCONNECTED` event is a [Last Will](#) message the broker publishes when the device session drops. **Each active endpoint emits its own `mqttConnEVT`.**

`mqttConnEVT` is published on each endpoint's `{EP}/clients/rfid` topic (e.g. `<tenantId>/MGMT/clients/rfid/<serial>` , `<tenantId>/CTRL/clients/rfid/<serial>`) — **not** the `{EP}/clients/event` topic that carries `alerts` and `heartBeatEVT` , and not the DATA topic that carries `dataEVT` . Subscribe to the `rfid` topic of each endpoint you want connection events for.

Details

Property	Value
Event Type	Connection-state transition (birth / Last Will)
Trigger	Connection transition (on every MQTT connect / disconnect; autonomous, with no command or config)
Applies To	Endpoint: Any endpoint (MGMT , CTRL , DATA). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	alerts , heartBeatEVT , dataEVT

When to Use

Subscribe to `mqttConnEVT` when you need a **reliable, per-endpoint presence/liveness signal** — to learn the moment an MQTT endpoint (re)connects or drops, without polling. The reader emits it **autonomously** on every connection-state change; there is nothing to enable and no command to issue.

- **Get:** each endpoint's connect / disconnect transitions (`connectionState` = `CONNECTED` / `DISCONNECTED`), with device identity (`deviceModel`, `deviceSerialNo`) and protocol-version metadata (`apiVersion`, `mqttVersion`).
- **Use it for:** presence/liveness dashboards, triggering reconnect and re-subscribe logic, and detecting a dropped device even when the device itself cannot report it — the `DISCONNECTED` Last Will is broker-published.
- **Prefer a different approach when:** you want an on-demand, correlated read rather than a push — poll `get_status`, whose reply itself confirms the device is reachable and reports `deviceStatus.radioConnection`. Use the event for continuous presence monitoring and `get_status` for a point-in-time check.

Topic Mapping

`mqttConnEVT` is device-emitted and has no `cmdnd` / `resp` topics. It is published on each active endpoint's own `rfid` topic.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/rfid/<serial></code>	The management endpoint's connection events. <code>CONNECTED</code> is the MQTT birth message; <code>DISCONNECTED</code> is the broker Last Will. Published <code>retain=false</code> — a subscriber that connects after a transition will not see the last state replayed.
Event	Reader → App	<code><tenantId>/CTRL/clients/rfid/<serial></code>	The CTRL endpoint's connection events — each active endpoint emits its own (<code>retain=false</code>).

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Here the `rfid` tail is the endpoint's configured topic name, not tag data. On a reboot-triggered reconnect, each active endpoint (MGMT, CTRL, ...) publishes its own `CONNECTED` event on its own `rfid` topic.

See [Topics and Channels](#) and [Broker Connection](#).

Key Fields

- `connectionState` (event) — `CONNECTED` (birth message: the endpoint (re)connected) or `DISCONNECTED` (Last Will: the session was lost). This is the field you route and act on.
- `timestamp` (event) — ISO-8601 instant of the transition (`format: date-time`). Present on `CONNECTED` ; typically omitted on `DISCONNECTED` , because the device is gone when the broker publishes the Last Will.
- `deviceModel` / `deviceSerialNo` (event) — device identity: model (`RFD40` / `RFD90`) and the unique serial number, so you can attribute each transition to a specific reader.
- `apiVersion` (event) — IOTC API version as a bare JSON number (e.g. `1.21`); note it differs from the `v1.21` string command responses return (see [Notes and Caveats](#)).
- `mqttVersion` (event) — MQTT protocol version (MQTT 3.1.1), serialized as the float `3.09999999` (≈ `3.1` , a lossy JSON-number encoding); treat any value ≈ `3.1` as MQTT 3.1.1.

Event Payload

Six top-level fields; the schema requires all of them except `timestamp` , which may be absent (notably on `DISCONNECTED`). Pick a **connection state** below; the Example and Schema update together. Use the **Schema** tab for the field reference and the **Example** tab for the concrete payload.

Connection state

Connected event



Event payload

Example: `CONNECTED` (live)

```
1 {
2   "connectionState" : "CONNECTED" ,
3   "timestamp" : "2026-06-15T13:09:33.810Z" ,
4   "deviceModel" : "RFD40" ,
5   "deviceSerialNo" : "RFD40-211545201D0034" ,
6   "apiVersion" : 1.21 ,
7   "mqttVersion" : 3.09999999
8 }
```

Event payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	co nn ec ti on St at e *	enum	CONNECTED DISCONNECTED	CONNECTED	Endpoint connection status. CONNECTED = the endpoint (re)connected (birth message); DISCONNECTED = the session was lost (Last Will). See Event Semantics .
2	time stam p	string	date-time	2026-06-15T13:09:33.810Z	ISO-8601 time of the transition (format: date-time). Present on CONNECTED; typically omitted on DISCONNECTED (see Event Semantics).
3	de vi ce Mo de l *	enum	RFD40 RFD90	RFD40	Reader hardware model.
4	de vi ce Se ri al No *	string	—	RFD40-24190525100354	Unique device serial number.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	apiVersion *	number	v1.0 v1.1 v1.2 v1.21 v1.22	1.21	IOTC API version, emitted as a bare JSON number (e.g. 1.21). Note this differs from the v1.21 string that command responses return, and the schema pairs type: number with an enum of v... strings that no number can match — see Notes and Caveats .
6	mqttVersion *	number	—	3.0999999	MQTT protocol version — MQTT 3.1.1. Emitted as a JSON number; the device serializes it as the float 3.0999999 (≈ 3.1), a lossy encoding (see Event Semantics).

Event Semantics

States. `connectionState` carries one of two values, each tied to an MQTT lifecycle role:

connectionState	MQTT role	When	timestamp
CONNECTED	Birth message	Published by the endpoint when it (re)connects to the broker.	Present (ISO-8601).
DISCONNECTED	Last Will (LWT)	Published by the broker when the device session drops (detected via keepalive).	Usually omitted — the device is gone, so it cannot stamp it.

Because `DISCONNECTED` is a broker-published Last Will, it can arrive some time after the actual drop, once the broker detects the lost session through the keep-alive, and it carries no `timestamp`. `CONNECTED` events follow on reconnect, each with its own ISO-8601 `timestamp`.

- The device serializes `mqttVersion` as the JSON number `3.0999999`, a lossy floating-point encoding of `3.1`. The MQTT protocol in use is **3.1.1**; treat any `mqttVersion` value ≈ 3.1 as MQTT 3.1.1, and do not test it for exact equality to `3.1`. (This is a known firmware serialization quirk.)
- `timestamp` **is optional and format** `date-time`. When present it is a full ISO-8601 instant (e.g. `2026-06-15T13:09:33.810Z`). It is typically **omitted on** `DISCONNECTED` (a broker Last Will published after the device is gone), which the schema permits: `timestamp` is the one field outside its `required` set.
- Per-endpoint emission**, `retain=false`. Each active endpoint publishes its own `mqttConnEVT` on its own `{EP}/clients/rfid` topic, so a single reconnect yields one event per endpoint (e.g. MGMT and CTRL each publish a `CONNECTED`). All are published with `retain=false`, so a late subscriber will not see the last state replayed — subscribe before the transition you want to observe.

Examples

The **Connection state** dropdown under [Event Payload](#) shows the contract's `CONNECTED` event and its `DISCONNECTED` Last Will, which has no `timestamp`. Each endpoint publishes its own on its own `rfid` topic.

A second `CONNECTED` event on the **CTRL** endpoint's `rfid` topic (per-endpoint emission — the MGMT endpoint published its own moments earlier):

mqttConnEVT — CONNECTED on the CTRL endpoint (live)

```
1 {
2   "connectionState" : "CONNECTED" ,
3   "timestamp" : "2026-06-14T12:54:51.411Z" ,
4   "deviceModel" : "RFD40" ,
5   "deviceSerialNo" : "RFD40-24190525100255" ,
6   "apiVersion" : 1.21 ,
7   "mqttVersion" : 3.0999999
8 }
```

Notes and Caveats

- `apiVersion` is a JSON number here, a `v...` string in command responses. `mqttConnEVT` declares `apiVersion` as a **number**, so the event emits it bare — `"apiVersion": 1.21`. Every command response declares it a **string** with a leading `v` — `get version`, for instance, returns `"apiVersion": "v1.21"`. A shared envelope parser that assumes a string will break on this event (a strict decoder throws; a lenient one silently coerces `1.21` to `"1.21"`, which never matches `"v1.21"`). Parse the event's value as a number, and normalize both forms before you compare them — same version family, two representations. The schema also gives this number an enum of `v...` strings (`v1.0` ... `v1.22`), which no JSON number can equal, so its own examples fail that enum — a known issue in the source schema. Validate the type, not the enum.
- No response envelope; automatic.** `mqttConnEVT` is a device-emitted event, so it carries **no response / code** — the command response-code table does not apply, and no response code is defined for any event. The reader always emits it on a connection-state change; it needs no `config events` toggle. (The contract names this message `epConnectionEvent`, a cosmetic mismatch with the event name — the on-the-wire payload is unaffected.)

`mqttConnEVT` is **read-only telemetry** with no credentials (`deviceSerialNo` is an identifier; `apiVersion` / `mqttVersion` are version values). Use the `rfd`-topic connection events for a reliable presence/liveness signal — the `DISCONNECTED` Last Will tells you a device dropped even when the device itself cannot. For the concept-level view, see [MQTT Connectivity](#) and [Broker Connection](#).

Related:  [alerts](#) ·  [heartBeatEVT](#) ·  [dataEVT](#) ·  [Events](#) ·  [MQTT Connectivity](#) ·  [Broker Connection](#)

Overview

`dataEVT` is the reader's **tag-data** event — the device publishes it **autonomously** while an inventory is running, carrying the RFID tag reads (and any barcode reads) it produces. It is **device-emitted** (no command is in the payload path), but it only flows once you **start an inventory** with `control_operation` `START`, using whatever the `set_operating_mode` profile configured. It is read-only tag output.

Each event is a small envelope — `type` (the active profile), `timestamp`, and a `data` object holding a `tagData[]` array (and/or a `barcodeData[]` array). Reads are **batched** — the contract declares no limit on records per event — and the **operating mode's** `tagMetaDataToEnable` **flags gate which fields each tag carries** — of a record's twenty declared fields, only two are always there. See [Field Support](#).

`dataEVT` is published on the **DATA endpoint's** data topic (e.g. `<tenantId>/DATA/clients/dataevent/<serial>` for `DATA1`), not the `event` topic (`alerts` / `heartBeatEVT`) or the `rfid` topic (`mqttConnEVT`). Receiving it requires an **active** `DATA1` **endpoint** (`config_endpoint`, `epType: DATA1`, `activate: true`). The contract also declares a second data channel, `DATA2`, but it is currently not supported.

Details

Property	Value
Event Type	Tag-data (and barcode) read output
Trigger	Inventory (started with <code>control_operation</code> <code>START</code> ; stops on <code>STOP</code>)
Applies To	Endpoint: Data (DATA). Reader: RFD40 series, RFD90 series
Credentials	None device-side; a <code>RESERVED</code> -bank read can expose a tag's access and kill passwords (see Notes)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>alerts</code> , <code>heartBeatEVT</code> , <code>mqttConnEVT</code> , <code>set_operating_mode</code>

When to Use

Consume `dataEVT` to receive the tags an active inventory finds — it is the reader's sole tag-output stream:

- **Collect tag reads** — subscribe to the DATA endpoint's data topic and process each event's `data.tagData[]` (the always-present `EPCid` plus any enabled memory-bank and telemetry fields).
- **Collect barcode reads** — the same stream carries `data.barcodeData[]` when a barcode is scanned during the operation.
- **Read access-operation results** — when `accessOperations` are armed, each read's `accessResults[]` reports the outcome of the configured READ/WRITE/LOCK/KILL.

`dataEVT` is **device-emitted** — there is no request/response and **no polling equivalent**; you cannot fetch a tag read on demand. Configure it first with `set_operating_mode`, start the stream with `control_operation` `START`, and stop it with `STOP`. If it is already running and nothing arrives, work through [Emission](#).

Topic Mapping

`dataEVT` is device-emitted and has no `cmdnd` / `resp` topics. It is published on the DATA endpoint's data topic.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/DATA/clients/data1event/<serial></code>	The first data channel's tag stream.
Event	Reader → App	<code><tenantId>/DATA/clients/data2event/<serial></code>	The second data channel. Currently not supported: the reader does not yet support a <code>DATA2</code> endpoint, so nothing is published here.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the

reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The exact middle segment is the topic configured in the DATA endpoint's `publishTopics[1]` — the slot, not the topic text, is what routes tag data. See [Emission](#).

See [Topics and Channels](#).

Key Fields

Every `dataEVT` payload field below is read-only device output; the final **Emission gate** bullet is a request-side control (what arms the stream), not a payload field.

- `type` (event) — the active operating-mode profile echoed on the read (e.g. `BALANCED_PERFORMANCE`); the reader emits the same profile on every event of a run.
- `timestamp` (event) — ISO-8601 time the event was generated; the only required top-level field.
- `data.tagData[]` (event) — the batch of RFID reads. Each read always carries `EPCid` (hex tag identifier) plus the required `eventNum` (per-record sequence number).
- `EPC / TID / USER / RESERVED` (event) — memory-bank contents as hex, each present only when its bank is enabled; a `RESERVED` read can expose the tag's access/kill passwords (see Notes).
- `peakRssi / phase / channel / seenCount` (event) — per-read telemetry: RSSI in dBm, phase in degrees, the raw channel value, and the seen-count — each gated by its `tagMetaDataToEnable` flag.
- `accessResults[]` (event) — one string per armed access operation (READ/WRITE/LOCK/KILL); present only when `accessOperations` are configured.
- `data.barcodeData[]` (event) — barcode reads (`decodedBarcode`, `symbology`) when a barcode is scanned during the operation.
- Emission gate** (request) — the stream flows only when an endpoint, an operating mode and a running inventory all line up. No active inventory, no `dataEVT`. The full chain, with a check at each link, is [Emission](#).

Emission

Every command in this chain can return `code: 0` while the stream stays silent. The response-code catalogue is a closed 39-member set and **not one member describes a data endpoint that is missing, inactive or wrongly addressed** — so a silent stream produces no error to read. Silence is the signal, and this section is the checklist.

Work down it in order. Each link has something you can verify.

1. Subscribe Before You Start

Tag data is published with `retain: false` on all three telemetry channels, so **anything published before your subscription is live is gone and is never redelivered**. A client that sends `START` and *then* subscribes sees an empty stream that is not a configuration fault at all.

2. A DATA Endpoint Must Exist

`epType` is a closed seven-member enum — `MGMT`, `MGMT_EVT`, `CTRL`, `DATA1`, `DATA2`, `SOTI`, `MDM` — and only the data types carry tag and barcode data. `DATA2` is currently not supported, so create a `DATA1` endpoint with `config_endpoint` using `operation: add`.

Verify: `get_endpoint_config` and read the returned `activeEndpoints`.

3. It Must Be Activated

`activate` is a boolean and **its default is `false`**, so an endpoint you did not explicitly activate is not carrying anything. Send `activate: true` for the data endpoint of type `DATA1` — on the `add`, or with an `update` if the endpoint was saved dormant. The reader checks the parameters and, if they are valid, connects the endpoint without a reboot. A `CONNECTED` `mqttConnEVT` on the endpoint's connection-state topic confirms it.

⚠ THE TRAP THAT PRODUCES A SILENT STREAM

`publishTopics` is a **three-slot positional array**, and it is the position that routes — slot 0 is command replies, slot 1 is asynchronous events, slot 2 is the connection event and the Last Will. On a data endpoint **slot 1 carries the tag data**. The conventional names `resp` / `event` / `rfid` are a convention and are **not** auto-detected.

So a single-entry payload — one topic, in slot 0 — lands in the **reply** slot and streams nothing. The endpoint is created, the command succeeds, and no tag data ever arrives. A DATA endpoint needs **two** entries: a filler in slot 0 and the tag-data topic in slot 1.

4. An Operating Mode Must Be Configured, and It Does Not Survive a Reboot

Set the profile and the metadata flags with `set_operating_mode`. Two things to know:

- **The mode is not persistent.** A reboot discards it and the reader returns to its default operating mode (see `set_operating_mode`), so re-apply the mode after any reboot.
- **Do not configure `FAST_READ`.** The contract states that data events are not currently supported under that profile, and describes the profile itself as not currently supported — pick another profile.

5. Start the Inventory

The reader does not transmit until its [regulatory region](#) is set, and the region is set outside MQTT with 123RFID Desktop or 123RFID Mobile. Confirm it with [get_current_region](#), then send [control_operation](#) `START`. Stop with `STOP`.

! CHECK WHAT THE MODE HAS ARMED BEFORE YOU SEND `START`

`START` executes whatever [set_operating_mode](#) last configured, including destructive [accessOperations](#) — see the caution in [Notes and Caveats](#) before starting a production inventory.

Still Silent?

Two causes that look like a broken stream and are not:

- An `INCLUDE` post filter that matches nothing. A filter reporting only matching tags will report none if the pattern matches none. Read it back with [get_post_filter](#) — see [Shaping](#).
- A reconfiguration that was refused. [Code 35](#) rejects a change to an *active* endpoint and advises deactivating it first. The contract does not document how to deactivate an endpoint directly, so instead of updating the active endpoint in place, do one of the following:
 - Delete the endpoint, then add it again with the new settings.
 - Activate a different endpoint first, then update the original endpoint.

One thing not to rely on: watching for an [mqttConnEVT](#) as proof the endpoint came up. A connection event shows a session, not that tag data is routed to slot 1, so use [get_endpoint_config](#) instead.

Telemetry Payload

The envelope is `type`, `timestamp` (the only required field), and `data`. Pick a **read-detail** level below; the Example and Schema update together — from a base tag batch, to reads enriched with EPC/TID and reader telemetry, to access-operation results and a barcode read. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete batch.

Read detail

Tag data event

Telemetry payload

Example: base tag data (abridged)

```
1  {
2    "type": "BALANCED_PERFORMANCE" ,
3    "timestamp": "2026-06-16T06:03:36.894Z" ,
4    "data": {
5      "tagData": [
6        {
7          "EPCid": "E28011B0A506007D3DD07C0F" ,
8          "eventNum": 41 ,
9          "peakRssi": -73 ,
10         "seenCount": 256
11       },
12       {
13         "EPCid": "311100000000000000000000" ,
14         "eventNum": 42 ,
15         "peakRssi": -50 ,
16         "seenCount": 256
17       },
18       {
19         "EPCid": "11113333880600890630DBEC" ,
20         "eventNum": 43 ,
21         "peakRssi": -53 ,
22         "seenCount": 256
23       },
24       {
25         "EPCid": "000011C1A5000062F7952926" ,
26         "eventNum": 44 ,
27         "peakRssi": -65 ,
28         "seenCount": 256
29       },
30       {
31         "EPCid": "E200001B880600870630DBEB" ,
32         "eventNum": 45 ,
33         "peakRssi": -44 ,
34         "seenCount": 256
35       },
36       {
37         "EPCid": "1111333300004001B043ACB2" ,
38         "eventNum": 46 ,
39         "peakRssi": -36 ,
40         "seenCount": 256
41       },
42       {
43         "EPCid": "11113333333344445555AAAA" ,
44         "eventNum": 47 ,
45         "peakRssi": -32 ,
46         "seenCount": 256
47       },

```

```

48     {
49         "EPCid" : "11113333A5000062F798E5B6" ,
50         "eventNum" : 48 ,
51         "peakRssi" : -34 ,
52         "seenCount" : 256
53     },
54     {
55         "EPCid" : "11113333333344445555AAAA" ,
56         "eventNum" : 49 ,
57         "peakRssi" : -36 ,
58         "seenCount" : 256
59     },
60     {
61         "EPCid" : "11115555222233334444A9B6" ,
62         "eventNum" : 50 ,
63         "peakRssi" : -43 ,
64         "seenCount" : 256
65     },
66     {
67         "EPCid" : "111133332222333344445555" ,
68         "eventNum" : 51 ,
69         "peakRssi" : -44 ,
70         "seenCount" : 256
71     },
72     {
73         "EPCid" : "111133332222333344445550" ,
74         "eventNum" : 52 ,
75         "peakRssi" : -43 ,
76         "seenCount" : 256
77     },
78     {
79         "EPCid" : "FFFFFFFFE3435363742426666" ,
80         "eventNum" : 53 ,
81         "peakRssi" : -51 ,
82         "seenCount" : 256
83     },
84     {
85         "EPCid" : "11113333333344445555AAAA" ,
86         "eventNum" : 54 ,
87         "peakRssi" : -46 ,
88         "seenCount" : 256
89     },
90     {
91         "EPCid" : "11113333880600900630DBF1" ,
92         "eventNum" : 55 ,
93         "peakRssi" : -52 ,
94         "seenCount" : 256
95     },
96     {
97         "EPCid" : "E200001B880600840630DBE2" ,
98         "eventNum" : 56 ,

```

```

99         "peakRssi" : -57 ,
100         "seenCount" : 256
101     },
102     {
103         "EPCid" : "111133332222333344445555" ,
104         "eventNum" : 57 ,
105         "peakRssi" : -63 ,
106         "seenCount" : 256
107     },
108     {
109         "EPCid" : "111133332222333344445555" ,
110         "eventNum" : 58 ,
111         "peakRssi" : -63 ,
112         "seenCount" : 256
113     },
114     {
115         "EPCid" : "111133332222333344445555" ,
116         "eventNum" : 59 ,
117         "peakRssi" : -62 ,
118         "seenCount" : 256
119     },
120     {
121         "EPCid" : "E200001B880600870630DBEB" ,
122         "eventNum" : 60 ,
123         "peakRssi" : -44 ,
124         "seenCount" : 256
125     },
126     {
127         "EPCid" : "000011C1A5000062F7952926" ,
128         "eventNum" : 61 ,
129         "peakRssi" : -66 ,
130         "seenCount" : 256
131     },
132     {
133         "EPCid" : "11113333B22FB22FB22FB22F" ,
134         "eventNum" : 62 ,
135         "peakRssi" : -65 ,
136         "seenCount" : 256
137     },
138     {
139         "EPCid" : "AAAABBBB11112222CCCCDDDD" ,
140         "eventNum" : 63 ,
141         "peakRssi" : -60 ,
142         "seenCount" : 256
143     },
144     {
145         "EPCid" : "BEDD11112222333344445555" ,
146         "eventNum" : 64 ,
147         "peakRssi" : -45 ,
148         "seenCount" : 256
149     },

```

```

150     {
151         "EPCid" : "311100000000000000000000" ,
152         "eventNum" : 65 ,
153         "peakRssi" : -51 ,
154         "seenCount" : 256
155     },
156     {
157         "EPCid" : "11113333880600890630DBEC" ,
158         "eventNum" : 66 ,
159         "peakRssi" : -53 ,
160         "seenCount" : 256
161     },
162     {
163         "EPCid" : "1111FB63AC1F3841DADA00000000CAFEDADACAFEDADACAFE00001111" ,
164         "eventNum" : 67 ,
165         "peakRssi" : -57 ,
166         "seenCount" : 256
167     },
168     {
169         "EPCid" : "E28011B0A506007D3DD07D01" ,
170         "eventNum" : 68 ,
171         "peakRssi" : -72 ,
172         "seenCount" : 256
173     },
174     {
175         "EPCid" : "E28011B0A506007D3DD07C0F" ,
176         "eventNum" : 69 ,
177         "peakRssi" : -69 ,
178         "seenCount" : 256
179     },
180     {
181         "EPCid" : "E28011B0A506007D3DD07D00" ,
182         "eventNum" : 70 ,
183         "peakRssi" : -70 ,
184         "seenCount" : 256
185     },
186     {
187         "EPCid" : "11115555222233334444A9B6" ,
188         "eventNum" : 71 ,
189         "peakRssi" : -41 ,
190         "seenCount" : 256
191     },
192     {
193         "EPCid" : "111133332222333344445550" ,
194         "eventNum" : 72 ,
195         "peakRssi" : -45 ,
196         "seenCount" : 256
197     },
198     {
199         "EPCid" : "311100000000000000000000" ,
200         "eventNum" : 73 ,

```

```

201         "peakRssi" : -65 ,
202         "seenCount" : 256
203     },
204     {
205         "EPCid" : "111133332222333344445555" ,
206         "eventNum" : 74 ,
207         "peakRssi" : -61 ,
208         "seenCount" : 256
209     },
210     {
211         "EPCid" : "A1B2222BBBB" ,
212         "eventNum" : 75 ,
213         "peakRssi" : -65 ,
214         "seenCount" : 256
215     }
216 ]
217 }
218 }

```

Telemetry payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	type	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	BALANCED_PERFORMANCE
2	timestamp*	string	—	2026-06-16T06:03:36.894Z
3	data	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
4	tagData	array	—	—
5	EP C * id	string	—	000011C1A5000062F7952926
6	EP C	string	—	D98A3000AAAABBBB11112222CCCCD
7	TI D	string	—	e20060030178529d
8	US ER	string	—	12343333123456781234567812345 12345678123456781234567812345 12345678123456781234567812345 12345678
9	RE SE RV ED	string	—	12343333123456781234567812345 12345678123456781234567812345 12345678123456781234567812345 12345678
10	PC	string	—	3000
11	CR C	string	—	b06a

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
12	chan nel	number	—	926750
13	even t [*] Num	number	—	13
14	format	string	—	epc
15	peak Rs si	number	—	-59
16	phase	number	—	0
17	seen Count	number	—	256

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
18	XP C	string	—	—
19	ac ce ss Re su lt s	array	—	["READ-EPC-Success", ...]
20	us er De fi ne d	string	—	—
21	fi rs tS ee nT im e	number	—	—
22	la st Se en Ti me	number	—	—
23	MA C	string	—	AC:3F:A4:00:2B:15
24	HO ST NA ME	string	—	RFD40-211545201D0034

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
25	barcodeData	array	—	—
26	symbolology	string	—	0x03
27	decodedBarcode*	string	—	PWR-BGA24V78W1WW 183213949

Field Support

A tag record declares twenty fields. Two are always there, twelve are switched, and six have no switch at all.

Always present: `EPCid` and `eventNum`, the two fields the schema marks `required`. Everything else is conditional.

Switched by `tagMetadataToEnable` in `set_operating_mode`. The switch names are ALL-CAPS on purpose — they mirror the EPC Gen2 air-protocol spelling and are wire-fixed — and **four of the twelve do not match the field they enable:**

Switch	Field it enables
<code>RSSI</code>	<code>peakRssi</code>

Switch	Field it enables
PHASE	phase
CHANNEL	channel
SEENCOUNT	seenCount
EPC	EPC — or EPCid ; see below
PC , XPC , CRC , TID , USER , MAC , HOSTNAME	the identically named field

Six fields have no switch: `RESERVED` , `firstSeenTime` , `lastSeenTime` , `accessResults` , `userDefined` , `format` . The contract does not say which of them firmware can control. Two of them do arrive in practice — `RESERVED` and `accessResults` both appear when `accessOperations` are configured — so the absence of a switch is not the same as the absence of the field.

- `accessResults` arrives with configured access operations, not with metadata flags. It is populated only when you arm `accessOperations` (READ/WRITE/LOCK/KILL). A bank read enabled purely through `tagMetaDataSetToEnable` surfaces as the `EPC` / `TID` / `USER` metadata fields instead — which is why you can see bank contents with no `accessResults` at all. For how to read the strings, see [Telemetry Semantics](#).
- Which field the `EPC` switch controls is not documented. It could enable `EPC` or `EPCid` . Their descriptions differ — `EPC` is the memory bank's contents, `EPCid` the item identifier — and `EPCid` is required on every record anyway. **Code against `EPCid` and treat `EPC` as bank content you asked for.**
- No switch declares a default, and the contract does not say which metadata the device reports when you ask for none. What the contract's examples show is narrow and consistent: `EPCid` , `eventNum` , `peakRssi` and `seenCount` appear in **40 of 40** example records. Treat that as the observed floor, not a guarantee, and **treat every other field as optional.**

Telemetry Semantics

What each record carries. Two things determine the content of a `dataEVT` record — which fields are switched on, and which of the two reporting shapes you are receiving:

- Which fields are present is governed by `tagMetaDataSetToEnable` in `set_operating_mode` . The full switch-to-field mapping, the two always-present fields and the six with no switch at all are in [Field Support](#).
- Two reporting shapes — and no command in this API selects between them. A `dataEVT` arrives in one of two shapes. **Per-read:** every individual read is reported, so `channel` and `phase` are present and `seenCount` is `1` . **Aggregated:** reads are combined and reported at intervals, so

`channel` and `phase` are absent, `seenCount` counts the reads since the previous report, and `peakRssi` is the peak since that report. The reader's firmware governs which shape you get; **the contract declares no field for it and no operation carries one**, so treat the shape as something you *detect*, not something you set — **the presence of `channel` is the discriminator**.

- Despite the schema describing `channel` as a value "in MHz", the device emits a **raw channel value/index** (e.g. `9216`), **not** a ~900 MHz frequency. Treat `channel` as an opaque channel index; do not interpret it as megahertz. `channel` (and `phase`) are present only in the per-read shape.
- **Reading `accessResults` strings.** When the array is present (see [Field Support](#) for when that is), each string is one operation's outcome — `READ-EPC-Success`, or an error, or a bare `SUCCESS` for writes. The shape is one string per armed operation, in the order the operations were configured.
- **`type` echoes the profile.** `type` carries the active operating-mode profile (e.g. `BALANCED_PERFORMANCE`), and the reader emits the same value on every event of a run — so it identifies the run, not the record. One value you will not see here is `FAST_READ`; [Emission](#) explains why, and why you should not configure it. How much arrives per event is [Volume and Limits](#).

Shaping

`tagMetadataToEnable` **changes what each record carries. Post filters change which records exist.** That is the whole distinction, and it is the one worth keeping straight.

A post filter is applied after inventory, on the stream's way out, and it is the only control that changes **which tag records reach a data endpoint**. Configure it with `set_post_filter` and read it back with `get_post_filter` — the rule shape, the match methods and the `INCLUDE` / `EXCLUDE` semantics are documented there and not repeated here.

Three things to know from the `dataEVT` side, because the contract leaves them open:

- **The endpoint vocabularies differ and are not mapped.** Filters scope to `dataEpType` (`DATA_EP1` / `DATA_EP2`); endpoints are configured with `epType` (`DATA1` / `DATA2` , which is currently not supported). The contract declares both and never states the correspondence, so `DATA_EP1` to `DATA1` is the necessary reading rather than a documented one.
- **The pattern matches the *tag ID*, and the contract does not name which payload field that is.** `EPCid` is the only always-present identifier, so it is the reasonable assumption — confirm against firmware before relying on it. Nothing ties post filtering to barcode reads on the same stream.
- **Do not use `eventNum` continuity as a loss detector while a filter is installed.** Nothing in the contract says whether a suppressed read still consumes a sequence number. See [Delivery and Loss](#).

Also undocumented, and worth a read-back rather than an assumption: whether filters survive a reboot, how many rules an endpoint accepts, and what happens when two rules on one endpoint disagree.

Delivery and Loss

`dataEVT` carries no `response` or `code` — it is device-emitted, so the command response-code table does not apply and no response code is defined for any event. The only thing that tells you something went wrong is what fails to arrive.

Tag data travels at the QoS you set on the data endpoint's tag-data topic, `publishTopics[1].qos`: `0` or `1`, the two levels the reader supports. The reader's default endpoint configurations use `0`, and the channels declare `retain: false`. At [QoS 0](#), loss is normal rather than exceptional. At either level, nothing is replayed to a subscriber that was not listening. [Delivery and Retry Semantics](#) has the full treatment. [QoS 1](#) changes the transport guarantee without making the reader re-send anything it already dropped.

`eventNum` is the only ordering signal on the wire. It is a per-record sequence counter, and the contract documents no wrap-around, no reset rule and no scope — so **do not assume the counter restarts between runs, or that it does not**. A break in the sequence is a prompt to investigate, not proof of loss: it cannot distinguish a dropped publish from a record the reader never emitted. And a batch arriving shortly after `STOP` is a late buffered batch, not a gap — see [Notes and Caveats](#).

Volume and Limits

Firmware caps a whole `dataEVT` message at 6144 bytes, and there is no record-count limit. The current contract states neither limit; the byte cap is a firmware limit that the device owners confirmed, and firmware notes it can be increased to support batching, so treat it as a current-build number. An over-size message is **dropped by the reader, with no error, and the connection stays up**: the symptom is a message that never arrives. Size a buffer against 6144 bytes and a variable record count — not against a tag count. The serialisation is not stated: any tags-per-event arithmetic assumes minified JSON.

The declared controls live on the endpoint as `batching`: `reportingInterval` (1000–60000 ms) and `maxPayloadSizePerReport` (1024–5120 bytes); a `0` in either falls back to the minimum. A report is flushed on **whichever comes first, and the contract does not document which wins** when both thresholds are near. Send both members together — whether setting one and omitting the other leaves the other at its default is undocumented. The contract's examples use `1000` / `1024`.

Examples

Every contract example is in the **Read detail** dropdown under [Telemetry Payload](#), and which fields appear reflects the operating-mode configuration:

- **Tag data event** — the default fields only: `EPCid`, `eventNum`, `peakRssi` and `seenCount`.
- **Tag data with additional details** — EPC and TID memory plus `channel`, `phase`, `MAC` and `HOSTNAME`, with their `tagMetaDataToEnable` flags on. In this per-read shape `channel` and `phase` are present and `seenCount` is `1`.
- **Tag events for access operations** — a configured `READ` of the EPC, TID and RESERVED banks, with the outcomes in `accessResults`.
- **Barcode data event** — a scanned barcode, reported in `data.barcodeData[]` instead of `tagData`. Barcode reads were not exercised live; this is the contract's example.

Notes and Caveats

RESERVED BANK READS EXPOSE TAG PASSWORDS

The `RESERVED` memory bank holds a tag's **access and kill passwords**. If you configure a `READ` of the RESERVED bank, those passwords appear in `dataEVT` as the `RESERVED` hex field — treat that data as sensitive. (`EPCid` and the other bank contents are physical-tag data, not device or broker credentials.)

- `STOP` **is not instantaneous for buffered reads**. `control_operation` `STOP` halts the radio, but a buffered in-flight `dataEVT` batch may still be delivered a few seconds after `STOP` (~3–5 s observed on RFD40). Do not treat a `dataEVT` arriving shortly after `STOP` as an error, and do not stop listening the instant you send `STOP`.

Start the stream with `control_operation` `START` and stop it with `STOP`. Because `dataEVT` reflects the operating mode, **any armed destructive** `accessOperations` (**WRITE/LOCK/KILL**) **execute on the tags read during the run** — confirm the mode is read-only before starting (see `set_operating_mode` [§ Notes](#)). For the concept-level view, see [Tag Data Stream](#), Choose Shared or Dedicated Data Endpoints, and `dataEVT`.

Related:  `control_operation` ·  `set_operating_mode` ·  `config_endpoint` ·  [Events](#) ·  [Tag Data Stream](#) ·  Choose Shared or Dedicated Data Endpoints